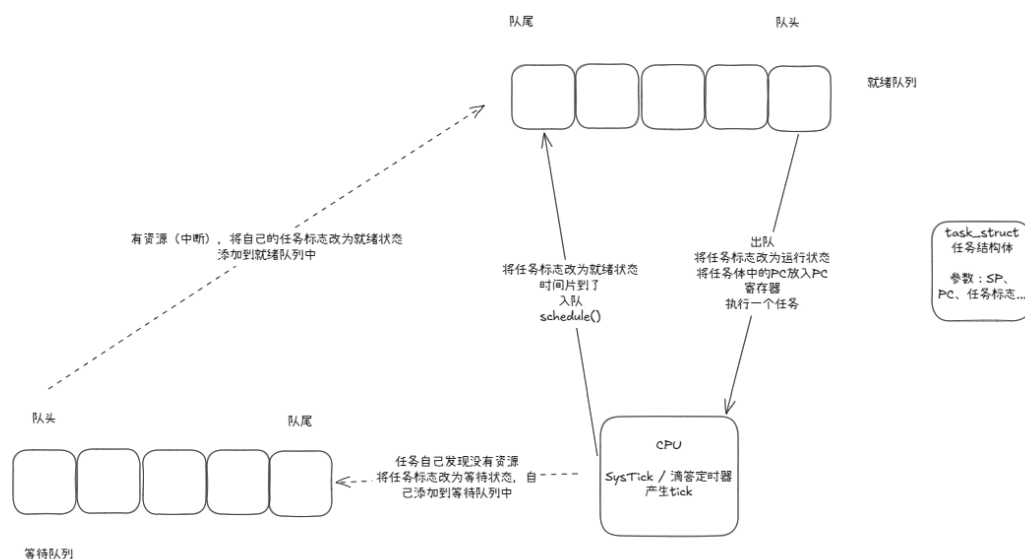
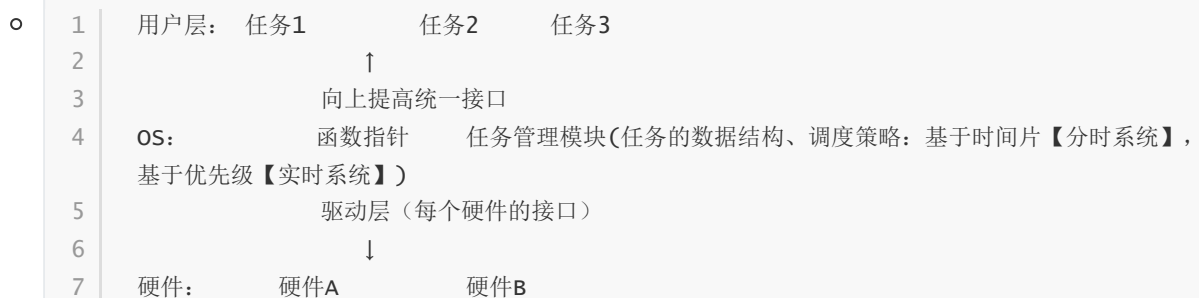


1. 操作系统移植

什么是OS?

- 是一个管理软件。main函数中进行“任务管理”，前后台设计方式、多任务分时设计方式、多任务抢占（优先级）设计方式。
- 在不同的硬件，运行相同的任务，每个系统都能运行多个任务
- 架构思路



- 自己编写OS，需要考虑怎么定义任务（task_struct中保存，如：PC、SP、任务状态标志...）
- 等待 → 就绪，靠的中断处理函数（需要硬件配合）
- 内存管理：
 - 物理内存管理（总内存管理，需要软件进行管理用户区和内核区，以及栈【保证不越界】、堆【需要策略管理内存碎片】）
 - 虚拟内存管理（必须有硬件单元支撑 MMU）
- 基于时间片设计思路
 - 需要定时器中断。即需要一个tick(节拍)，单次 Tick 时间 (秒) = $\frac{1}{\text{定时器频率 (Hz)}}$ ，一个时间片为多少tick由当前OS的设计决定。时间片到了就去就绪队列“调度一个任务”
 - 多个等待队列，一个就绪队列

- 基于优先级的设计思路
 - 多个就绪队列（一个就绪队列就是一个优先级），一个等待队列

2. FreeRTOS(源码安装)

书籍推荐：FreeRTOS内核实现与应用开发实战基于stm32 —— 野火出版

官网地址：<https://www.freertos.org/zh-cn-cmn-s>

GitHub地址：<https://github.com/FreeRTOS/FreeRTOS> + <https://github.com/FreeRTOS/FreeRTOS-Kernel>

FreeRTOS在提供了两套代码，一个是最新更新版本，一个是LTS版本（长期维护版本），在LTS版本中是没有Demo参考。

GitHub上的为全家桶，所有的信息和示例都放到了一起，Github上的外设库和内核代码是分开的

```
1  FreeRTOS/
2  |— FreeRTOS/
3  |   |— Source/                                <-- 核心! FreeRTOS 内核源码
4  |   |   |— include/                          <-- 内核头文件 (task.h, queue.h 等)
5  |   |   |— portable/                        <-- 硬件移植层代码 (不同 CPU 架构的接口)
6  |   |   |   |— GCC/                          <-- 使用 GCC 编译器的各芯片移植代码
7  |   |   |   |— RVDS/                        <-- 使用 Keil/ARMCC 编译器的移植代码
8  |   |   |   |— MemMang/                     <-- 5 种官方内存管理算法 (heap_1 ~ heap_5)
9  |   |   |— tasks.c                          <-- 任务管理核心实现
10 |   |   |— queue.c                          <-- 队列/信号量/互斥锁实现
11 |   |   |— list.c                           <-- 链表数据结构实现
12 |   |   |— timers.c                         <-- 软件定时器实现
13 |   |
14 |   |— Demo/                                <-- 各大厂商开发板的官方示例工程
15 |   |   |— CORTEX_STM32F103_Keil/          <-- 针对 STM32F1 的现成示例
16 |   |   |— ...                             <-- 包含几百个不同芯片/开发板的 Demo
17 |   |
18 |   |— Test/                                <-- 自动化测试与内核验证代码
```

- FreeRTOS/Demo目录
 - 主要包含了大量预先制作好的示例工程，比如保留CORTEX_STM32F103_Keil目录，为stm32F103的参考工程
 - 还有Common目录，独立于demo的通用代码，大部分已经废弃，比如像drivers里其实就是stm32的标准库，可以删除
- FreeRTOS/Source目录
 - 该目录即为FreeRTOS的内核文件，去掉非.c的文件，保证目录的纯粹
 - 这些.c文件是通用的，和平台无关的代码，核心的文件有2个： task.c和list.c

FreeRTOS/Source/*.c	作用
tasks.c	必需，任务操作
list.c	必需，列表操作

FreeRTOS/Source/*.c	作用
queue.c	基本必需，提供队列、信号量（semaphore）的操作
timer.c	可选， software timer
event_groups.c	可选，提供event group功能
croutine.c	可选，过时了

- FreeRTOS/Source/portable目录
 - portable目录下是移植时需要实现的文件
 - 目录命名规则： [compiler]/[architecture]
 - 比如RVDS/ARM_CM3，表示cortexM3架构在RVDS工具上的移植文件
 - 这个目录原本应该保留RVDS下的内容，以支持Keil，但最新版本的Keil编译器AC6对这些语法不支持了
 - 保留目录下的**GCC目录**，该目录保留ARM_CM3，以支持stm32F103
 - 其中该目录下有一个名为MemMang的目录，里面包含了堆内存管理代码，可以在Keil建立工程时，选择其中一个作为堆内存管理接口即可
- FreeRTOS-Plus目录： FreeRTOS的生态文件，不需要，可以删除
- tools目录：被亚马逊收购后，新增的云服务业务，可以删除

官网的LTS：面向**企业级商业产品**的极简模块化发布。只要核心库和稳定的外围库

1	FreeRTOS-LTS/	
2	— FreeRTOS/	
3	— FreeRTOS-Kernel/	<-- 纯粹的内核源码（即主版本中的 Source 文件夹）
4	— include/	<-- 内核头文件
5	— portable/	<-- 硬件移植层（GCC，IAR，RVDS 等）
6	— *.c	<-- tasks.c，queue.c 等内核源文件
7		
8	— coreMQTT/	<-- 轻量级 MQTT 协议栈（已通过 LTS 安全认证）
9	— coreHTTP/	<-- 轻量级 HTTP 客户端库
10	— coreJSON/	<-- 适合嵌入式系统的 JSON 解析器
11	— coreSNTP/	<-- 网络对时协议库
12	— corePKCS11/	<-- 密码学安全接口规范
13	— backoffAlgorithm/	<-- 指数退避重试算法库
14	— FreeRTOS-Cellular-Interface/	<-- 蜂窝网络（如 4G/5G 模组）抽象层
15	— FreeRTOS-Plus-TCP/	<-- 官方 TCP/IP 网络协议栈

移植时需要的文件

在自己项目的内核目录，应该包含以下内容

1	FreeRTOS Core Folder Structure
2	
3	— CMakeLists.txt // CMake 编译构建配置文件

4	— croutine.c	// [可选] 协程 (Co-routine) 实现文件, 适用于极小内存芯片 (通常不使用)
5	— event_groups.c	// [可选] 事件标志组 (Event Groups) 实现, 用于多任务间的事件同步
6	— History.txt	// 版本更新与变更历史日志
7	— list.c	// [必选] 双向链表数据结构实现, FreeRTOS 调度器和队列的基础
8	— manifest.yml	// 包元数据配置文件
9	— queue.c	// [必选] 消息队列与信号量 (Semaphore/Mutex) 核心实现
10	— stream_buffer.c	// [可选] 流缓冲区 (Stream Buffer) 与消息缓冲区实现 (单生产者/单消费者场景)
11	— tasks.c	// [必选] 任务管理与核心调度器 (Scheduler) 实现
12	— timers.c	// [可选] 软件定时器 (Software Timers) 实现
13		
14	—inc	// 内核 API 接口头文件目录
15	atomic.h	// 原子操作接口 (用于多核或无锁操作)
16	croutine.h	// 协程头文件
17	deprecated_definitions.h	// 旧版本废弃宏定义的兼容层
18	event_groups.h	// 事件标志组 API 头文件
19	FreeRTOS.h	// [必选] FreeRTOS 核心配置与基础定义头文件
20	list.h	// [必选] 链表数据结构头文件
21	message_buffer.h	// 消息缓冲区 API 头文件
22	mpu_prototypes.h	// MPU (内存保护单元) API 函数原型
23	mpu_syscall_numbers.h	// MPU 系统调用号定义
24	mpu_wrappers.h	// MPU 封装函数定义
25	newlib-freertos.h	// Newlib C 标准库适配头文件
26	picolibc-freertos.h	// Picolibc 轻量级 C 标准库适配头文件
27	portable.h	// [必选] 硬件移植层统一接口抽象头文件
28	projdefs.h	// [必选] 系统通用宏定义 (如 pdPASS, pdFAIL)
29	queue.h	// [必选] 队列/信号量 API 头文件
30	semphr.h	// 信号量/互斥锁 API 头文件
31	stack_macros.h	// 栈溢出检测辅助宏定义
32	stack_macros.h	// 栈分析辅助宏定义
33	stdint.readme	// C99 stdint.h 类型的说明文件
34	stream_buffer.h	// 流缓冲区 API 头文件
35	task.h	// [必选] 任务管理 API 头文件
36	timers.h	// 软件定时器 API 头文件
37		
38	—portable	// 平台移植层 (Porting Layer)
39	—GCC	// 选择的编译器工具链: GCC
40	—ARM_CM3	// 选择的 CPU 架构: ARM Cortex-M3 (如 STM32F103)
41	port.c	// [必选] Cortex-M3 的硬件细节实现 (上下文切换、PendSV/SysTick 中断)
42	portmacro.h	// [必选] Cortex-M3 的架构定义 (如临界区控制、数据类型定义)
43		
44	—MemMang	// 动态内存管理策略选型 (5种策略选择1种)
45	heap_4.c	// [必选] 最常用堆管理算法 (支持动态内存分配/释放, 带碎片自动合并)
46		
47	FreeRTOSConfig.h	// 配置文件, 放在自己相应放的目录下, 我放在App下

• MemMang下的5种策略:

◦	文件	优点	缺点
	heap_1.c	分配简单, 时间确定	只分配、不回收

文件	优点	缺点
heap_2.c	动态分配，最佳匹配	碎片，时间不定
heap_3.c	调用标准库函数	速度慢、时间不定
heap_4.c	相邻空闲内存可合并	可解决碎片问题、时间不定
heap_5.c	在heap_4基础上支持分隔的内存块	可解决碎片问题、时间不定

- `FreeRTOSConfig.h`: FreeRTOS的配置文件，比如选择调度算法: `configUSE_PREEMPTION` 每个demo都必定含有`FreeRTOSConfig.h`,建议去修改demo中的 `FreeRTOSConfig.h`, 而不是从头写一个。【需要自己去demo拷贝一份，并且修改参数】

因为FreeRTOS需要时间片，进行任务的调度，使用的是SysTick定时器；所以自己写代码时，不能使用这个定时器

在 `FreeRTOSConfig.h` 内容：

```

1  /*
2   * FreeRTOS V202411.00
3   * Copyright (C) 2020 Amazon.com, Inc. or its affiliates. All Rights Reserved.
4   *
5   * Permission is hereby granted, free of charge, to any person obtaining a copy of
6   * this software and associated documentation files (the "Software"), to deal in
7   * the Software without restriction, including without limitation the rights to
8   * use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of
9   * the Software, and to permit persons to whom the Software is furnished to do so,
10  * subject to the following conditions:
11  *
12  * The above copyright notice and this permission notice shall be included in all
13  * copies or substantial portions of the Software.
14  *
15  * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
16  * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS
17  * FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR
18  * COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER
19  * IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
20  * CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
21  *
22  * https://www.FreeRTOS.org
23  * https://github.com/FreeRTOS
24  *
25  */
26
27 #ifndef FREERTOS_CONFIG_H
28 #define FREERTOS_CONFIG_H
29
30 /*-----
31  * Application specific definitions.
32  *
33  * These definitions should be adjusted for your particular hardware and
34  * application requirements.

```

```

35  *
36  * THESE PARAMETERS ARE DESCRIBED WITHIN THE 'CONFIGURATION' SECTION OF THE
37  * FreeRTOS API DOCUMENTATION AVAILABLE ON THE FreeRTOS.org WEB SITE.
38  *
39  * See http://www.freertos.org/a00110.html
40  *-----*/
41
42  #define configUSE_PREEMPTION          1
43  #define configUSE_IDLE_HOOK          0
44  #define configUSE_TICK_HOOK          0
45  #define configCPU_CLOCK_HZ           ( ( unsigned long ) 72000000 )
46  #define configTICK_RATE_HZ           ( ( TickType_t ) 1000 )
47  #define configMAX_PRIORITIES         ( 5 )
48  #define configMINIMAL_STACK_SIZE     ( ( unsigned short ) 128 )
49  #define configTOTAL_HEAP_SIZE        ( ( size_t ) ( 17 * 1024 ) )
50  #define configMAX_TASK_NAME_LEN      ( 16 )
51  #define configUSE_TRACE_FACILITY     0
52  #define configUSE_16_BIT_TICKS       0
53  #define configIDLE_SHOULD_YIELD      1
54
55  /* Co-routine definitions. */
56  #define configUSE_CO_ROUTINES         0
57  #define configMAX_CO_ROUTINE_PRIORITIES ( 2 )
58
59  /* Set the following definitions to 1 to include the API function, or zero
60  to exclude the API function. */
61
62  #define INCLUDE_vTaskPrioritySet       1
63  #define INCLUDE_uxTaskPriorityGet      1
64  #define INCLUDE_vTaskDelete           1
65  #define INCLUDE_vTaskCleanUpResources 0
66  #define INCLUDE_vTaskSuspend          1
67  #define INCLUDE_vTaskDelayUntil       1
68  #define INCLUDE_vTaskDelay            1
69
70  /* This is the raw value as per the Cortex-M3 NVIC.  Values can be 255
71  (lowest) to 0 (1?) (highest). */
72  #define configKERNEL_INTERRUPT_PRIORITY 255
73  /* !!!! configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to zero !!!!
74  See http://www.FreeRTOS.org/RTOS-Cortex-M3-M4.html. */
75  #define configMAX_SYSCALL_INTERRUPT_PRIORITY 191 /* equivalent to 0xb0, or
76  priority 11. */
77
78  /* This is the value being used as per the ST library which permits 16
79  priority values, 0 to 15.  This must correspond to the
80  configKERNEL_INTERRUPT_PRIORITY setting.  Here 15 corresponds to the lowest
81  NVIC value of 255. */
82  #define configLIBRARY_KERNEL_INTERRUPT_PRIORITY 15
83
84  /* 添加内容 */
85  /* 必须添加： 将 FreeRTOS 的中断处理函数映射到 Cortex-M 对应的标准向量入口 */
86  #define vPortSVCHandler SVC_Handler

```

```

87 #define xPortPendSVHandler PendSV_Handler
88 #define xPortSysTickHandler SysTick_Handler
89
90
91 /* -- 下面可选，还有其他选项没有列出 -- */
92 /* 互斥量、递归互斥量、计数信号量 */
93 #define configUSE_MUTEXES 1
94 #define configUSE_RECURSIVE_MUTEXES 1
95 #define configUSE_COUNTING_SEMAPHORES 1
96
97 /* 软件定时器（很常用） */
98 #define configUSE_TIMERS 1
99 #define configTIMER_TASK_PRIORITY ( configMAX_PRIORITIES - 1 )
100 #define configTIMER_QUEUE_LENGTH 10
101 #define configTIMER_TASK_STACK_DEPTH ( configMINIMAL_STACK_SIZE * 2 )
102
103 /* 任务通知（现代 FreeRTOS 强烈推荐） */
104 #define configUSE_TASK_NOTIFICATIONS 1
105 #define configTASK_NOTIFICATION_ARRAY_ENTRIES 1
106
107 /* 静态/动态分配支持（根据需要） */
108 #define configSUPPORT_STATIC_ALLOCATION 0 /* 需要静态创建任务时改为 1 */
109 #define configSUPPORT_DYNAMIC_ALLOCATION 1
110
111 /* 断言（调试时非常有用） */
112 #define configASSERT( x ) if( ( x ) == 0 ) { taskDISABLE_INTERRUPTS(); for( ;; ); }
113
114 /* 栈溢出检测 */
115 #define configCHECK_FOR_STACK_OVERFLOW 2 /* 0关闭, 1简单, 2完整 */
116
117 #endif /* FREERTOS_CONFIG_H */

```

对于 `stm32f10x_it.c` 中的修改

```

1  /**
2      *****
3      * @file    Project/STM32F10x_StdPeriph_Template/stm32f10x_it.c
4      * @author  MCD Application Team
5      * @version V3.6.0
6      * @date    20-September-2021
7      * @brief   Main Interrupt Service Routines.
8      *          This file provides template for all exceptions handler and
9      *          peripherals interrupt service routine.
10     *****
11     * @attention
12     *
13     * Copyright (c) 2011 STMicroelectronics.
14     * All rights reserved.
15     *
16     * This software is licensed under terms that can be found in the LICENSE file
17     * in the root directory of this software component.
18     * If no LICENSE file comes with this software, it is provided AS-IS.
19     *

```

```

20  ****
21  */
22
23  /* Includes -----*/
24  /** @addtogroup STM32F10x_StdPeriph_Template
25      * @{
26      */
27
28  /* Private typedef -----*/
29  /* Private define -----*/
30  /* Private macro -----*/
31  /* Private variables -----*/
32  /* Private function prototypes -----*/
33  /* Private functions -----*/
34
35  /**
36      *          Cortex-M3 Processor Exceptions Handlers          */
37  /**
38
39  /**
40      * @brief This function handles NMI exception.
41      * @param None
42      * @retval None
43      */
44  void NMI_Handler(void)
45  {
46  }
47
48  /**
49      * @brief This function handles Hard Fault exception.
50      * @param None
51      * @retval None
52      */
53  void HardFault_Handler(void)
54  {
55      /* Go to infinite loop when Hard Fault exception occurs */
56      while (1)
57      {
58      }
59  }
60
61  /**
62      * @brief This function handles Memory Manage exception.
63      * @param None
64      * @retval None
65      */
66  void MemManage_Handler(void)
67  {
68      /* Go to infinite loop when Memory Manage exception occurs */
69      while (1)
70      {
71      }
72  }

```



```

73
74 /**
75  * @brief This function handles Bus Fault exception.
76  * @param None
77  * @retval None
78  */
79 void BusFault_Handler(void)
80 {
81     /* Go to infinite loop when Bus Fault exception occurs */
82     while (1)
83     {
84     }
85 }
86
87 /**
88  * @brief This function handles Usage Fault exception.
89  * @param None
90  * @retval None
91  */
92 void UsageFault_Handler(void)
93 {
94     /* Go to infinite loop when Usage Fault exception occurs */
95     while (1)
96     {
97     }
98 }
99
100 /* 屏蔽这3个中断函数 */
101 #ifndef FREERTOS
102 /**
103  * @brief This function handles SVCall exception.
104  * @param None
105  * @retval None
106  */
107 void SVC_Handler(void)
108 {
109 }
110
111 /**
112  * @brief This function handles PendSVC exception.
113  * @param None
114  * @retval None
115  */
116 void PendSV_Handler(void)
117 {
118 }
119
120 /**
121  * @brief This function handles SysTick Handler.
122  * @param None
123  * @retval None
124  */
125 void SysTick_Handler(void)

```

```

126 {
127 }
128 #endif
129
130 /**
131  * @brief This function handles Debug Monitor exception.
132  * @param None
133  * @retval None
134  */
135 void DebugMon_Handler(void)
136 {
137 }
138
139
140 /*****
141  /*          STM32F10x Peripherals Interrupt Handlers          */
142  /* Add here the Interrupt Handler for the used peripheral(s) (PPP), for the */
143  /* available peripheral interrupt handler's name please refer to the startup */
144  /* file (startup_stm32f10x_xx.s). */
145  *****/
146
147 /**
148  * @brief This function handles PPP interrupt request.
149  * @param None
150  * @retval None
151  */
152 /*void PPP_IRQHandler(void)
153 {
154 }*/
155
156 /**
157  * @}
158  */
159

```

对于 `stm32f10x_it.h` 中的修改

```

1  /**
2      ****
3      * @file    Project/STM32F10x_StdPeriph_Template/stm32f10x_it.h
4      * @author  MCD Application Team
5      * @version V3.6.0
6      * @date    20-September-2021
7      * @brief   This file contains the headers of the interrupt handlers.
8      ****
9      * @attention
10     *
11     * Copyright (c) 2011 STMicroelectronics.
12     * All rights reserved.
13     *
14     * This software is licensed under terms that can be found in the LICENSE file
15     * in the root directory of this software component.
16     * If no LICENSE file comes with this software, it is provided AS-IS.

```

```

17      *
18      ****
19      */
20
21  /* Define to prevent recursive inclusion -----*/
22  #ifndef __STM32F10x_IT_H
23  #define __STM32F10x_IT_H
24
25  #ifdef __cplusplus
26      extern "C" {
27  #endif
28
29  /* Includes -----*/
30  #include "stm32f10x.h"
31
32  /* Exported types -----*/
33  /* Exported constants -----*/
34  /* Exported macro -----*/
35  /* Exported functions ----- */
36
37  void NMI_Handler(void);
38  void HardFault_Handler(void);
39  void MemManage_Handler(void);
40  void BusFault_Handler(void);
41  void UsageFault_Handler(void);
42  void DebugMon_Handler(void);
43  #ifndef FREERTOS
44  void SVC_Handler(void);
45  void PendSV_Handler(void);
46  void SysTick_Handler(void);
47  #endif
48
49  /* 自定义中断 */
50  void EXTI0_IRQHandler();
51  void TIM6_IRQHandler();
52  void USART1_IRQHandler();
53
54  #ifdef __cplusplus
55  }
56  #endif
57
58  #endif /* __STM32F10x_IT_H */
59
60

```

在keil中将所有.c添加到项目树中，所有头文件都需要在魔术棒设置中告知keil头文件位置。

测试例子

```

1  #include <FreeRTOS.h>
2  #include <stm32f10x.h>
3  #include <task.h>           // 任务管理接口
4

```

```

5 volatile static int task1Flags = 0;
6 volatile static int task2Flags = 0;
7 volatile static int task3Flags = 0;
8
9 void task1(void * arg) {
10     BaseType_t xReturn = pdPASS;
11     while (1) {
12         task1Flags = 1;
13         task2Flags = 0;
14         task3Flags = 0;
15     }
16 }
17
18 void task2(void * arg) {
19     while (1) {
20         task1Flags = 0;
21         task2Flags = 1;
22         task3Flags = 0;
23     }
24 }
25
26 TaskHandle_t task1_handle;          // 任务PCB信息
27 TaskHandle_t task2_handle;
28
29 int main() {
30     // 创建任务
31     BaseType_t xReturn = pdPASS;
32
33     xReturn = xTaskCreate(task1, "task1", 100, NULL, 0, &task1_handle);
34     if (xReturn != pdPASS) {
35         goto failed;
36     }
37     xReturn = xTaskCreate(task2, "task2", 100, NULL, 0, &task2_handle);
38     if (xReturn == pdPASS) {
39         vTaskStartScheduler();          // 使用系统调用，让CPU在异常中执行该任务调度，之后交替
    执行这两个任务
40     }
41 failed:
42     while (1);
43 }
44

```

任务相关的函数

API 函数	功能描述	核心参数说明
<code>xTaskCreate()</code>	动态创建任务并分配内存	任务函数名, 任务名称, 栈深度, 传递参数, 优先级, 任务句柄
<code>xTaskCreateStatic()</code>	静态创建任务（需手动指定内存）	任务函数名, 任务名称, 栈深度, 传递参数, 优先级, 栈数组, TCB结构体

API 函数	功能描述	核心参数说明
<code>vTaskDelete()</code>	删除指定的任务（传 <code>NULL</code> 删除自身）	<code>xTaskToDelete</code> : 目标任务句柄
<code>vTaskSuspend()</code>	挂起指定任务	<code>xTaskToSuspend</code> : 目标任务句柄
<code>vTaskResume()</code>	恢复挂起的任务	<code>xTaskToResume</code> : 目标任务句柄

```
1 BaseType_t xTaskCreate(  
2     TaskFunction_t pxTaskCode,          /* [参数1] 任务函数入口指针：指向任务的主体函  
    数（函数签名必须为 void vFunction(void *pvParameters)） */  
3     const char * const pcName,          /* [参数2] 任务的可读名称：字符串形式，主要用  
    于调试和追踪（最大长度由 configMAX_TASK_NAME_LEN 定义） */  
4     const configSTACK_DEPTH_TYPE uxStackDepth, /* [参数3] 任务栈深度：分配给该任务的栈空间大  
    小（注意：单位是 word 字节数/字长，例如在 32 位系统下，传入 128 代表 128 * 4 = 512 字节） */  
5     void * const pvParameters,          /* [参数4] 传递给任务的参数：通用指针，会作为  
    参数传递给 pxTaskCode 任务函数（若无参数传 NULL） */  
6     UBaseType_t uxPriority,              /* [参数5] 任务优先级：数值越大优先级越高，范  
    围为 0 ~ (configMAX_PRIORITIES - 1) */  
7     TaskHandle_t * const pxCreatedTask    /* [参数6] 任务句柄指针：用于传出刚创建好的任  
    务句柄（后续挂起、删除或修改该任务时需要用到；若不需要可传 NULL） */  
8 );
```

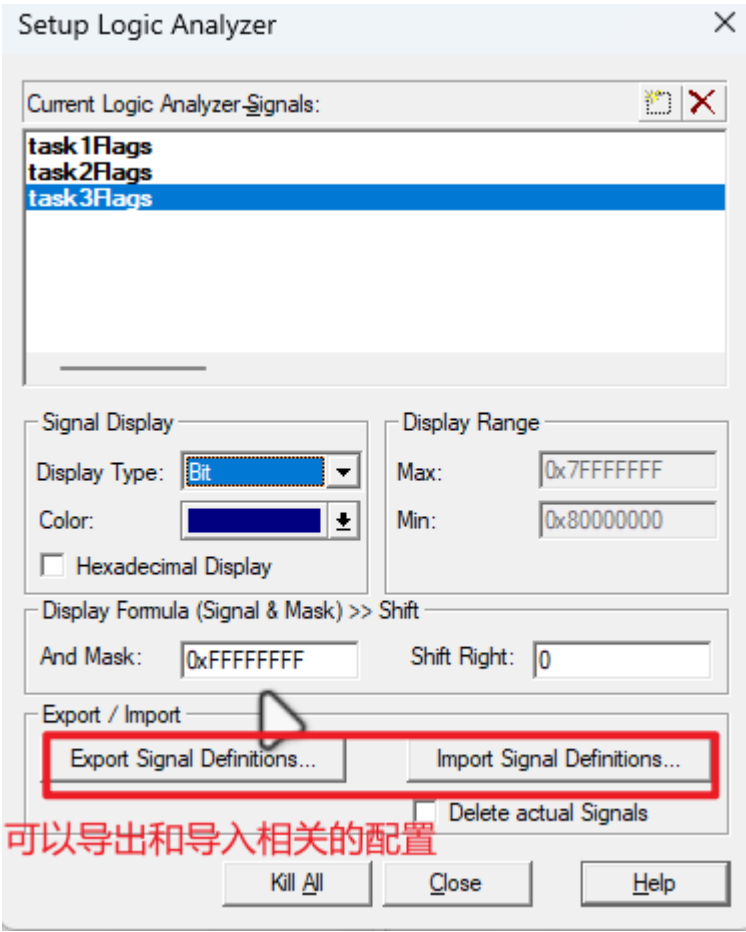
测试方式（软件的模拟逻辑分析仪测试）：

- 1. 先创建一个项目目标，用于区分RAM、flash的配置。
- 2. 在Debug选项框中，选择“use simulator”，在“Dialog DLL”中改为“DARMSTM.DLL”，然后对应的parameter改为“-pSTM32F103ZE”，勾选Load Application at Startup
- 3. 软件模拟，默认是12MHZ，需要修改频率，让它9倍频之后等于72MHZ

- ```
1 /* PLL configuration: PLLCLK = HSE * 9 = 72 MHz */
2 RCC->CFGR &= (uint32_t)((uint32_t)~(RCC_CFGR_PLLSRC | RCC_CFGR_PLLXTPRE |
3 RCC_CFGR_PLLMULL));
```

```
1 /* 软件模拟 HSE = 12MHZ 需要修改,添加宏方便切换 */
2 #ifndef FREERTOS_SIM
3 RCC->CFGR |= (uint32_t)(RCC_CFGR_PLLSRC_HSE | RCC_CFGR_PLLMULL9);
4 #else
5 RCC->CFGR |= (uint32_t)(RCC_CFGR_PLLSRC_HSE | RCC_CFGR_PLLMULL6); // 6倍频
6 #endif
7 ````
```

4.



### 3. Linux移植

Linux的移植流程：

| 步骤 | 阶段名称          | 核心任务与实现细节                                                                                                                                                                 |
|----|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 工具与环境搭建       | 搭建 Linux 开发主机环境，安装交叉编译工具链（如 <code>aarch64-none-linux-gnu-gcc</code> 或 <code>arm-linux-gnueabi-gcc</code> ），配置 TFTP/NFS 网络服务以便调试。                                          |
| 2  | Bootloader 阶段 | 硬件复位后首先运行（如 U-Boot）。初始化 DDR 内存、串口等基础硬件；配置启动参数（ <code>bootargs</code> / 传参给内核）；将 Flash/SD卡/eMMC 中的 <code>zImage</code> / <code>uImage</code> 以及设备树（DTB）加载到内存指定位置，并跳转到内核入口。 |
| 3  | 内核移植与启动       | 解压内核，进行架构相关初始化（如 MMU 开启、中断控制器配置）；挂载根文件系统（Rootfs）；解析设备树；建立虚拟内存与物理内存映射。                                                                                                     |

| 步骤 | 阶段名称        | 核心任务与实现细节                                                                                                                                                                           |
|----|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4  | 初始化根文件系统    | 基于 VFS（虚拟文件系统）将存储介质上的块设备挂载到挂载点；利用 Ext4/UBIFS 等文件系统建立文件名/路径（字符串）到磁盘 Block/Inode 的物理映射，使操作系统能够识别目录结构。                                                                                 |
| 5  | 用户态首个程序     | 内核在空间初始化完成后，从根文件系统中寻找并启动 <b>PID 1</b> 的 <code>init</code> 进程（如 BusyBox init、Systemd 或 SysVinit），完成从内核态（Kernel Space）到用户态（User Space）的跨越。                                            |
| 6  | Shell 与人机交互 | <code>init</code> 脚本通过 <code>/etc/inittab</code> 或服务配置文件拉起 <code>getty/login</code> 终端，进而加载并运行 <code>Shell</code> （如 <code>ash</code> 、 <code>bash</code> ）可执行程序，为用户提供指令解析与内核交互的窗口。 |

相关的工作岗位：BSP工程师

### 3.1 工具与环境搭建

1. 【面试常问】：交叉编译工具链（Cross-Compilation Toolchain）

回答：之所以需要交叉编译，是因为嵌入式目标机的计算性能和存储资源有限，不具备在本地直接编译复杂大型代码的能力；因此我们需要借用性能强悍的 x86 宿主机，配合针对目标板架构定制的交叉编译工具链来完成编译。（本质：）

编译工具：将高级语言→ 机器语言（各个CPU架构有自己独立的指令集），同时集成了某款OS的初始化代码（调用 `main` 函数之前，该OS初始化执行指令）

工具依赖于具体CPU架构

交叉编译工具链的本质：**指运行在某种架构的宿主机（Host）上，能够生成另一种不同架构目标机（Target）所能执行的机器码的编译器集合**

- 宿主机：Host、主机端，编写代码、执行编译的机器
- 目标机：Target，最终运行可执行程序的机器

如何证明工具链运行在哪个宿主机上

```
1 # 查看命令的位置
2 siguyuan@debian:~$ which gcc
3 /usr/bin/gcc
4
5 # 使用file命令探测和识别文件真正类型
6 siguyuan@debian:~$ file /usr/bin/gcc
7 /usr/bin/gcc: symbolic link to gcc-14 # 是一个软链接，链接到gcc-14
8
9 siguyuan@debian:~$ file /usr/bin/gcc-14
```

```

10 /usr/bin/gcc-14: symbolic link to x86_64-linux-gnu-gcc-14
11
12 # 链接的真实文件
13 siguyuan@debian:~$ file /usr/bin/x86_64-linux-gnu-gcc-14
14 /usr/bin/x86_64-linux-gnu-gcc-14: ELF 64-bit LSB executable, x86-64,
version 1 (GNU/Linux), dynamically linked, interpreter /lib64/ld-linux-x86-
64.so.2, BuildID[sha1]=a162828956e216e665b1216fd8d6199f4e9b78b4, for
GNU/Linux 3.2.0, stripped
15
16 # 文件绝对路径: ELF格式 64位架构 小端 可执行程序, 运行在x86-64CPU上, ELF文件头版本,
动态链接, 指定的动态链接器/加载器路径, 编译时生成的独一无二的 SHA-1 校验指纹, 最低内核版
本要求, 符号表已被剥离/去除

```

如何证明工具链 生成哪个目标机Target上

```

1 siguyuan@debian:~$ gcc -v
2 Using built-in specs.
3 COLLECT_GCC=gcc
4 COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-linux-gnu/14/lto-wrapper
5 OFFLOAD_TARGET_NAMES=nvptx-none:amdgc-n-amdhsa
6 OFFLOAD_TARGET_DEFAULT=1
7 Target: x86_64-linux-gnu # 编译的程序运行到 x86_64-linux上
8 Configured with: ../src/configure -v --with-pkgversion='Debian 14.2.0-19' -
--with-bugurl=file:///usr/share/doc/gcc-14/README.Bugs --enable-
languages=c,ada,c++,go,d,fortran,objc,obj-c++,m2,rust --prefix=/usr --with-
gcc-major-version-only --program-suffix=-14 --program-prefix=x86_64-linux-
gnu- --enable-shared --enable-linker-build-id --libexecdir=/usr/libexec --
without-included-gettext --enable-threads=posix --libdir=/usr/lib --enable-
nls --enable-bootstrap --enable-clocale=gnu --enable-libstdcxx-debug --
enable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-
libstdcxx-backtrace --enable-gnu-unique-object --disable-vtable-verify --
enable-plugin --enable-default-pie --with-system-zlib --enable-libphobos-
checking=release --with-target-system-zlib=auto --enable-objc-gc=auto --
enable-multiaich --disable-werror --enable-cet --with-arch-32=i686 --with-
abi=m64 --with-multilib-list=m32,m64,mx32 --enable-multilib --with-
tune=generic --enable-offload-targets=nvptx-none=/build/reproducible-
path/gcc-14-14.2.0/debian/tmp-nvptx/usr,amdgc-n-amdhsa=/build/reproducible-
path/gcc-14-14.2.0/debian/tmp-gcn/usr --enable-offload-defaulted --without-
cuda-driver --enable-checking=release --build=x86_64-linux-gnu --
host=x86_64-linux-gnu --target=x86_64-linux-gnu --with-build-
config=bootstrap-lto-lean --enable-link-serialization=3
9 Thread model: posix
10 Supported LTO compression algorithms: zlib zstd
11 gcc version 14.2.0 (Debian 14.2.0-19)
12

```

## 2. 工具链获取方式

1. 芯片厂家提供 —— 最优解 (性能优化最强, 需要花钱, 工程上推荐)
2. CPU厂家提供 —— 最通用 (泛用最强, 部分需要花钱, Linux不需要, 学习推荐)



- arm开发者网站下载: <https://developer.arm.com/tools-and-software/gnu-toolchain>
- 本次学习使用的是野火的 EBF6ULL 板子, 需要下载 gun-toolchain 的9版本 (新版的gcc工具不支持低版本的语法)

1 |

```
1 # 下载9版本的32位 需要添加-L允许重定向, 没有科学上网很慢
2 curl -L -O
 https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu-a/9.2-
 2019.12/binrel/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz
3
4 # 使用国内镜像源下载, 下载的大小为264181856
5 wget https://mirrors.tuna.tsinghua.edu.cn/armbian-releases/_toolchain/gcc-
 arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz
6
7 # 解压到/opt目录 (防止有些工具没有使用相对路径, 导致报错)
8 sudo tar -Jxvf gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf.tar.xz -C
 /opt/
9 ...
```

- 权限修改

1 |

```
1 siguyuan@debian:~/EBF_6ULL/packages$ ls -l /opt/
2 总计 4
3 drwxr-xr-x 9 siguyuan siguyuan 4096 2019年12月13日 gcc-arm-9.2-2019.12-x86_64-
 arm-none-linux-gnueabihf
4
5 # 如果中的所有组和所有者不是当前用户, 而是root, 需要修改权限
6 sudo chown -R 当前用户组:当前用户名 路径
```

```
1 # 文件重命名 (可选)
2 mv gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf arm-linux-gnueabihf
3
4 # 创建软连接
5 sudo ln -s /opt/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf /opt/arm-
 linux-gnueabihf
6
7 ...
8
9 -
```

- 配置环境变量

- export PATH=\$PATH:/opt/arm-linux-gnueabihf/bin

- 直接运行, 已经运行的终端无法知道环境变量的改变
- 把这个配置信息写到一个脚本里, 等需要时, 激活脚本 (推荐)

```

1 siguyuan@debian:~/EBF_6ULL/packages/env$ cat arm_path.sh
2 #!/bin/bash
3
4 # 交叉编译工具链的环境变量配置
5 export PATH=$PATH:/opt/arm-linux-gnueabi/bin
6 # export 把当前 shell 进程中的“局部变量”提升为“环境变量”，使其可以
 被当前进程后续创建的所有“子进程”继承和使用。
7 siguyuan@debian:~/EBF_6ULL/packages/env$ sudo chmod +x
 arm_path.sh
8

```

- 
- `source` 激活形式（推荐）
  - 在当前进程激活脚本，会修改当前的环境变量
  - `source arm_path.sh`
- 脚本直接执行
  - 只有子进程才有这个变量，父进程的环境变量没有执行
  - `./arm_path.sh`

○ 常用的连接介质：串口、网线（JTAG太贵、USB需要自己写驱动）

### 3. 自己构建 不推荐

- Buildroot：开源自动化嵌入式 Linux 系统构建工具。下载链接：<https://buildroot.org/download.html>

## 常见的二进制工具

在嵌入式与 C/C++ 开发中，这些属于 GNU Binutils 工具链的核心命令。以下是为你补充完整并加上高频实用参数的参考字典：

### `readelf`（解析 ELF 文件结构）

- `-h`：查看 ELF 文件头（显示目标架构如 ARM/x86、大小端、程序入口地址等）。
- `-S`：查看段表（Section headers），例如代码段 `.text`、数据段 `.data` 和 `.bss`。
- `-s`：查看详细的符号表。
- `-d`：查看动态段信息（可用于查看该程序依赖了哪些 `.so` 动态库）。

### `od` / `hexdump`（查看二进制原始数据）

- `od -t x1 文件名`：以单字节十六进制格式输出。
- `hexdump -C 文件名`（补充）：以左侧十六进制、右侧 ASCII 字符对应的经典排版显示，比 `od` 更适合人类阅读。

### `size`（查看内存分段占用）

- `size 文件名`：直接输出 `.text`（代码区）、`.data`（已初始化全局/静态变量）、`.bss`（未初始化全局/静态变量）的独立大小以及总大小。用于评估程序放入 RAM/ROM 的尺寸。

### `nm`（查看符号表）

- `nm` 文件名：列出程序中的函数名、全局变量等符号及其内存地址。
- `-C`：解开 (Demangle) C++ 的符号名，将乱码般的底层符号还原为带参数列表的 C++ 源码函数名。
- `-u`：只显示未定义符号（查找“为什么编译提示 undefined reference”时的利器）。

#### `strip` (文件瘦身/去除调试信息)

- `strip` 文件名：剔除符号表、重定位信息和调试信息。**开发时不能 strip，否则无法用 GDB 调试；部署到板子上前必须 strip，可大幅缩小文件体积。**
- `--strip-debug`：仅剔除调试信息，保留普通的符号表。
- 不同CPU架构的gcc生成的汇编，需要对应架构的strip进行识别

#### `objdump` (反汇编与分析，实际工程会做乱序处理，防止反汇编)

- `-d`：仅反汇编包含可执行指令的段（如 `.text`）。
- `-S`：将 C/C++ 源代码与对应的汇编代码交替显示（前提是编译时加了 `-g` 参数）。
- `-R`：显示动态重定位入口（查看对外部动态库函数的调用劫持点）。
- 不同CPU架构的gcc生成的汇编，需要对应架构的objdump进行识别

#### `objcopy` (文件格式转换/提纯)

- **提取纯二进制**：ELF 文件包含头部和地址信息，通常用此命令剥离格式，生成可以直接烧录到单片机或内存运行的裸二进制 (bin) 文件：`objcopy -O binary 原文件.elf 输出.bin`

#### `strings` (提取可打印字符)

- `strings` 文件名：打印文件中所有的文本字符串常量（例如代码里写死的密码、路径、版本号、报错提示等）。即使经过 `strip`，字符串常量也依然存在。

`addr2line`：程序发生段错误 (Segmentation fault) 死机时，配合内核打印的崩溃地址，将地址翻译为具体的**源码文件名和报错行号**。`addr2line -e 文件名 崩溃内存地址`

`file`：查看文件基础属性，快速确认这个程序是 32 位还是 64 位，是 x86 还是 ARM 架构。

`ldd` (仅限板端/主机端)：查看可执行文件运行所需的所有动态共享库 (`.so`) 及其绝对路径。

```

1 # 先创建例子
2 siguyuan@debian:~/EBF_6ULL/test$ gcc -o a_x86 aa.c
3 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-gcc -o a_arm aa.c
4 siguyuan@debian:~/EBF_6ULL/test$ ls
5 aa.c a_arm a_x86
6 siguyuan@debian:~/EBF_6ULL/test$ cat aa.c
7 #include <stdio.h>
8
9 void test01(){}
10
11 void test02(){}
12
13 int main(){
14 printf("=====\n");
15 test01();
16 test02();
17 return 0;

```

```

18 }
19
20 siguyuan@debian:~/EBF_6ULL/test$ ls -l
21 总计 32
22 -rw-rw-r-- 1 siguyuan siguyuan 134 8月11日 17:30 aa.c
23 -rwxrwxr-x 1 siguyuan siguyuan 12052 8月11日 17:38 a_arm
24 -rwxrwxr-x 1 siguyuan siguyuan 16008 8月11日 17:38 a_x86
25

```

```

1 # file查看ELF头区别
2
3 # 运行在x86-64的环境，没有剔除符号表
4 siguyuan@debian:~/EBF_6ULL/test$ file a_x86
5 a_x86: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
6 interpreter /lib64/ld-linux-x86-64.so.2,
7 BuildID[sha1]=71a94870abdb3570f4f832cf36deee88f311e1ed, for GNU/Linux 3.2.0, not
8 stripped
9
10 # 运行在32位 arm的环境，没有剔除符号表
11 siguyuan@debian:~/EBF_6ULL/test$ file a_arm
12 a_arm: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked,
13 interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, with debug_info, not
14 stripped
15

```

```

1 # 查看结构头的区别
2 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-hf-readelf -h a_arm
3 ELF Header:
4 Magic: 7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
5 Class: ELF32
6 Data: 2's complement, little endian
7 Version: 1 (current)
8 OS/ABI: UNIX - System V
9 ABI Version: 0
10 Type: EXEC (Executable file)
11 Machine: ARM
12 Version: 0x1
13 Entry point address: 0x10319 # 程序运行 最开始的地方
14 Start of program headers: 52 (bytes into file)
15 Start of section headers: 10572 (bytes into file)
16 Flags: 0x5000400, version5 EABI, hard-float ABI
17 Size of this header: 52 (bytes)
18 Size of program headers: 32 (bytes)
19 Number of program headers: 9
20 Size of section headers: 40 (bytes)
21 Number of section headers: 37

```

```

22 Section header string table index: 36
23
24
25 siguyuan@debian:~/EBF_6ULL/test$ od -t x1 a_arm
26 00000000 7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
27 00000020 02 00 28 00 01 00 00 00 19 03 01 00 34 00 00 00
28 00000040 4c 29 00 00 00 04 00 05 34 00 20 00 09 00 28 00
29 00000060 25 00 24 00 01 00 00 70 6c 04 00 00 6c 04 01 00
30
31
32 siguyuan@debian:~/EBF_6ULL/test$ readelf -h a_x86
33 ELF 头:
34 Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00 # 这个ELF头标志对应的二进制
35 类别: ELF64
36 数据: 2 补码, 小端序 (little endian)
37 Version: 1 (current)
38 OS/ABI: UNIX - System V
39 ABI 版本: 0
40 类型: DYN (Position-Independent Executable file)
41 系统架构: Advanced Micro Devices X86-64
42 版本: 0x1
43 入口点地址: 0x1050
44 程序头起点: 64 (bytes into file)
45 Start of section headers: 14024 (bytes into file)
46 标志: 0x0
47 Size of this header: 64 (bytes)
48 Size of program headers: 56 (bytes)
49 Number of program headers: 14
50 Size of section headers: 64 (bytes)
51 Number of section headers: 31
52 Section header string table index: 30
53
54 siguyuan@debian:~/EBF_6ULL/test$ od -t x1 a_x86
55 00000000 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
56 00000020 03 00 3e 00 01 00 00 00 50 10 00 00 00 00 00 00
57 00000040 40 00 00 00 00 00 00 00 c8 36 00 00 00 00 00 00
58 00000060 00 00 00 00 40 00 38 00 0e 00 40 00 1f 00 1e 00
59

```

入口地址：ELF 头部记录的一个内存指针，指示 CPU 或操作系统加载程序后**执行第一条指令的位置**。

- **操作系统/加载器**：Linux 下由标准链接脚本或动态随机基址（ASLR + PIE）决定（如 0x1000 或 0x10300 偏移）。（相对地址）
- **裸机/板端硬件**：由芯片 SRAM/Flash 的物理内存布局决定（如 i.MX6ULL 的 0x80000000、STM32 的 0x08000000）。（真实物理地址）

```

1 # 查看大小，不太编译器添加的main之前添加的内容不同
2 siguyuan@debian:~/EBF_6ULL/test$ size a_x86
3 text data bss dec hex filename
4 1423 584 8 2015 7df a_x86
5
6 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-hf-size a_arm
7 text data bss dec hex filename
8 799 288 4 1091 443 a_arm
9

```

```

1 # 查看二进制符号表
2 siguyuan@debian:~/EBF_6ULL/test$ nm a_x86
3 0000000000002134 r __abi_tag
4 0000000000004018 B __bss_start
5 0000000000004018 b completed.0
6 w __cxa_finalize@GLIBC_2.2.5
7 0000000000004008 D __data_start
8 0000000000004008 w data_start
9 0000000000001080 t deregister_tm_clones
10 00000000000010f0 t __do_global_dtors_aux
11 0000000000003dd8 d __do_global_dtors_aux_fini_array_entry
12 0000000000004010 D __dso_handle
13 0000000000003de0 d _DYNAMIC
14 0000000000004018 D _edata
15 0000000000004020 B _end
16 0000000000001178 T _fini
17 0000000000001130 t frame_dummy
18 0000000000003dd0 d __frame_dummy_init_array_entry
19 0000000000002130 r __FRAME_END__
20 0000000000003fe8 d _GLOBAL_OFFSET_TABLE_
21 w __gmon_start__
22 000000000000200c r __GNU_EH_FRAME_HDR
23 0000000000001000 T _init
24 0000000000002000 R _IO_stdin_used
25 w _ITM_deregisterTMCloneTable
26 w _ITM_registerTMCloneTable
27 U __libc_start_main@GLIBC_2.34
28 0000000000001147 T main
29 U puts@GLIBC_2.2.5
30 00000000000010b0 t register_tm_clones
31 0000000000001050 T _start
32 0000000000001139 T test01
33 0000000000001140 T test02
34 0000000000004018 D __TMC_END__
35
36 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-hf-nm a_arm
37 U abort@GLIBC_2.4
38 0002102c B __bss_end__
39 0002102c B _bss_end__
40 00021028 B __bss_start
41 00021028 B _bss_start__
42 00010348 t call_weak_fn

```

```

43 00021028 b completed.10857
44 00021020 D __data_start
45 00021020 w data_start
46 0001036c t deregister_tm_clones
47 000103bc t __do_global_dtors_aux
48 00020f0c d __do_global_dtors_aux_fini_array_entry
49 00021024 D __dso_handle
50 00020f10 d _DYNAMIC
51 00021028 D _edata
52 0002102c B __end__
53 0002102c B _end
54 00010458 T _fini
55 000103d4 t frame_dummy
56 00020f08 d __frame_dummy_init_array_entry
57 00010474 r __FRAME_END__
58 00021000 d _GLOBAL_OFFSET_TABLE_
59 w __gmon_start__
60 000102c8 T _init
61 00020f0c d __init_array_end
62 00020f08 d __init_array_start
63 00010460 R _IO_stdin_used
64 00010454 T __libc_csu_fini
65 00010414 T __libc_csu_init
66 U __libc_start_main@@GLIBC_2.4
67 000103f4 T main
68 U puts@@GLIBC_2.4
69 00010390 t register_tm_clones
70 00010318 T _start
71 000103d8 T test01
72 000103e6 T test02
73 00021028 D __TMC_END__
74

```

| 符号类型                        | 符号含义描述                                  | 常见示例                                                |
|-----------------------------|-----------------------------------------|-----------------------------------------------------|
| <b>T / t</b>                | <b>Text (代码段)</b> 符号                    | 函数、子程序 (如 <code>main</code> ) , t: 局部有效             |
| <b>D / d</b>                | <b>Data (数据段)</b> 符号                    | 已初始化的全局变量或静态变量 (如 <code>int a = 10;</code> )        |
| <b>B / b</b> 或 <b>S / s</b> | <b>BSS / Uninitialized (未初始化数据段)</b> 符号 | 未初始化的全局变量或静态变量 (如 <code>int a;</code> )             |
| <b>R / r</b>                | <b>ReadOnly (只读数据段)</b> 符号              | 常量数据、只读变量 (如 <code>const int a = 5;</code> 或字符串字面量) |
| <b>U</b>                    | <b>Undefined (未定义)</b> 符号               | 当前文件使用但未定义的符号 (如引用了外部库函数或未包含主体的声明)                  |
| <b>N</b>                    | <b>Debug (调试)</b> 符号                    | 编译时加入的调试信息符号                                        |

```

1 # 剔除符号表
2 siguyuan@debian:~/EBF_6ULL/test$ strip a_x86
3 siguyuan@debian:~/EBF_6ULL/test$ strip a_arm # 需要对应架构的strip才能识别
4 strip: Unable to recognise the format of the input file `a_arm'
5
6 siguyuan@debian:~/EBF_6ULL/test$ file a_x86
7 a_x86: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
 interpreter /lib64/ld-linux-x86-64.so.2,
 BuildID[sha1]=71a94870abdb3570f4f832cf36deee88f311e1ed, for GNU/Linux 3.2.0, stripped
 # 已经剔除
8 siguyuan@debian:~/EBF_6ULL/test$ ls -l a_x86
9 -rwxrwxr-x 1 siguyuan siguyuan 14472 8月11日 18:11 a_x86 # 对比之前少了一些
10 siguyuan@debian:~/EBF_6ULL/test$ nm a_x86
11 nm: a_x86: 无符号
12

```

```

1 # 反汇编
2 # 剔除符号表之后
3 siguyuan@debian:~/EBF_6ULL/test$ objdump -d a_x86
4
5 a_x86: 文件格式 elf64-x86-64
6
7
8 Disassembly of section .init:
9
10 0000000000001000 <.init>:
11 1000: 48 83 ec 08 sub $0x8,%rsp
12 1004: 48 8b 05 c5 2f 00 00 mov 0x2fc5(%rip),%rax # 3fd0
 <__cxa_finalize@plt+0x2f90>
13 100b: 48 85 c0 test %rax,%rax
14 100e: 74 02 je 1012 <puts@plt-0x1e>
15 1010: ff d0 call *%rax
16 1012: 48 83 c4 08 add $0x8,%rsp
17 1016: c3 ret
18
19 Disassembly of section .plt:
20
21 0000000000001020 <puts@plt-0x10>:
22 1020: ff 35 ca 2f 00 00 push 0x2fca(%rip) # 3ff0
 <__cxa_finalize@plt+0x2fb0>
23 1026: ff 25 cc 2f 00 00 jmp *0x2fcc(%rip) # 3ff8
 <__cxa_finalize@plt+0x2fb8>
24 102c: 0f 1f 40 00 nopl 0x0(%rax)
25
26 0000000000001030 <puts@plt>:
27 1030: ff 25 ca 2f 00 00 jmp *0x2fca(%rip) # 4000
 <__cxa_finalize@plt+0x2fc0>
28 1036: 68 00 00 00 00 push $0x0
29 103b: e9 e0 ff ff ff jmp 1020 <puts@plt-0x10>
30

```



```

31 Disassembly of section .plt.got:
32
33 0000000000001040 <__cxa_finalize@plt>:
34 1040: ff 25 9a 2f 00 00 jmp *0x2f9a(%rip) # 3fe0
 <__cxa_finalize@plt+0x2fa0>
35 1046: 66 90 xchg %ax,%ax
36
37 Disassembly of section .text:
38
39 0000000000001050 <.text>:
40 1050: 31 ed xor %ebp,%ebp
41 1052: 49 89 d1 mov %rdx,%r9
42 1055: 5e pop %rsi
43 1056: 48 89 e2 mov %rsp,%rdx
44 1059: 48 83 e4 f0 and $0xfffffffffffffff0,%rsp
45 105d: 50 push %rax
46 105e: 54 push %rsp
47 105f: 45 31 c0 xor %r8d,%r8d
48 1062: 31 c9 xor %ecx,%ecx
49 1064: 48 8d 3d dc 00 00 00 lea 0xdc(%rip),%rdi # 1147
 <__cxa_finalize@plt+0x107>
50 106b: ff 15 4f 2f 00 00 call *0x2f4f(%rip) # 3fc0
 <__cxa_finalize@plt+0x2f80>
51 1071: f4 hlt
52 1072: 66 2e 0f 1f 84 00 00 cs nopw 0x0(%rax,%rax,1)
53 1079: 00 00 00
54 107c: 0f 1f 40 00 nopl 0x0(%rax)
55 1080: 48 8d 3d 91 2f 00 00 lea 0x2f91(%rip),%rdi # 4018
 <__cxa_finalize@plt+0x2fd8>
56 1087: 48 8d 05 8a 2f 00 00 lea 0x2f8a(%rip),%rax # 4018
 <__cxa_finalize@plt+0x2fd8>
57 108e: 48 39 f8 cmp %rdi,%rax
58 1091: 74 15 je 10a8 <__cxa_finalize@plt+0x68>
59 1093: 48 8b 05 2e 2f 00 00 mov 0x2f2e(%rip),%rax # 3fc8
 <__cxa_finalize@plt+0x2f88>
60 109a: 48 85 c0 test %rax,%rax
61 109d: 74 09 je 10a8 <__cxa_finalize@plt+0x68>
62 109f: ff e0 jmp *%rax
63 10a1: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
64 10a8: c3 ret
65 10a9: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
66 10b0: 48 8d 3d 61 2f 00 00 lea 0x2f61(%rip),%rdi # 4018
 <__cxa_finalize@plt+0x2fd8>
67 10b7: 48 8d 35 5a 2f 00 00 lea 0x2f5a(%rip),%rsi # 4018
 <__cxa_finalize@plt+0x2fd8>
68 10be: 48 29 fe sub %rdi,%rsi
69 10c1: 48 89 f0 mov %rsi,%rax
70 10c4: 48 c1 ee 3f shr $0x3f,%rsi
71 10c8: 48 c1 f8 03 sar $0x3,%rax
72 10cc: 48 01 c6 add %rax,%rsi
73 10cf: 48 d1 fe sar $1,%rsi
74 10d2: 74 14 je 10e8 <__cxa_finalize@plt+0xa8>

```

```

75 10d4: 48 8b 05 fd 2e 00 00 mov 0x2efd(%rip),%rax # 3fd8
 <__cxa_finalize@plt+0x2f98>
76 10db: 48 85 c0 test %rax,%rax
77 10de: 74 08 je 10e8 <__cxa_finalize@plt+0xa8>
78 10e0: ff e0 jmp *%rax
79 10e2: 66 0f 1f 44 00 00 nopw 0x0(%rax,%rax,1)
80 10e8: c3 ret
81 10e9: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
82 10f0: f3 0f 1e fa endbr64
83 10f4: 80 3d 1d 2f 00 00 00 cmpb $0x0,0x2fd(%rip) # 4018
 <__cxa_finalize@plt+0x2fd8>
84 10fb: 75 2b jne 1128 <__cxa_finalize@plt+0xe8>
85 10fd: 55 push %rbp
86 10fe: 48 83 3d da 2e 00 00 cmpq $0x0,0x2eda(%rip) # 3fe0
 <__cxa_finalize@plt+0x2fa0>
87 1105: 00
88 1106: 48 89 e5 mov %rsp,%rbp
89 1109: 74 0c je 1117 <__cxa_finalize@plt+0xd7>
90 110b: 48 8b 3d fe 2e 00 00 mov 0x2efe(%rip),%rdi # 4010
 <__cxa_finalize@plt+0x2fd0>
91 1112: e8 29 ff ff ff call 1040 <__cxa_finalize@plt>
92 1117: e8 64 ff ff ff call 1080 <__cxa_finalize@plt+0x40>
93 111c: c6 05 f5 2e 00 00 01 movb $0x1,0x2ef5(%rip) # 4018
 <__cxa_finalize@plt+0x2fd8>
94 1123: 5d pop %rbp
95 1124: c3 ret
96 1125: 0f 1f 00 nopl (%rax)
97 1128: c3 ret
98 1129: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
99 1130: f3 0f 1e fa endbr64
100 1134: e9 77 ff ff ff jmp 10b0 <__cxa_finalize@plt+0x70>
101 1139: 55 push %rbp
102 113a: 48 89 e5 mov %rsp,%rbp
103 113d: 90 nop
104 113e: 5d pop %rbp
105 113f: c3 ret
106 1140: 55 push %rbp
107 1141: 48 89 e5 mov %rsp,%rbp
108 1144: 90 nop
109 1145: 5d pop %rbp
110 1146: c3 ret
111 1147: 55 push %rbp
112 1148: 48 89 e5 mov %rsp,%rbp
113 114b: 48 8d 05 b2 0e 00 00 lea 0xeb2(%rip),%rax # 2004
 <__cxa_finalize@plt+0xfc4>
114 1152: 48 89 c7 mov %rax,%rdi
115 1155: e8 d6 fe ff ff call 1030 <puts@plt>
116 115a: b8 00 00 00 00 mov $0x0,%eax
117 115f: e8 d5 ff ff ff call 1139 <__cxa_finalize@plt+0xf9>
118 1164: b8 00 00 00 00 mov $0x0,%eax
119 1169: e8 d2 ff ff ff call 1140 <__cxa_finalize@plt+0x100>
120 116e: b8 00 00 00 00 mov $0x0,%eax
121 1173: 5d pop %rbp

```

```

122 1174: c3 ret
123
124 Disassembly of section .fini:
125
126 0000000000001178 <.fini>:
127 1178: 48 83 ec 08 sub $0x8,%rsp
128 117c: 48 83 c4 08 add $0x8,%rsp
129 1180: c3 ret
130
131
132 # 没有剔除符号表, main函数位置清楚
133 siguyuan@debian:~/EBF_6ULL/test$ gcc -o a_x86 aa.c
134 siguyuan@debian:~/EBF_6ULL/test$ objdump -d a_x86
135
136 a_x86: 文件格式 elf64-x86-64
137
138
139 Disassembly of section .init:
140
141 0000000000001000 <_init>:
142 1000: 48 83 ec 08 sub $0x8,%rsp
143 1004: 48 8b 05 c5 2f 00 00 mov 0x2fc5(%rip),%rax # 3fd0
144 <__gmon_start__@Base>
145 100b: 48 85 c0 test %rax,%rax
146 100e: 74 02 je 1012 <_init+0x12>
147 1010: ff d0 call *%rax
148 1012: 48 83 c4 08 add $0x8,%rsp
149 1016: c3 ret
150
151 Disassembly of section .plt:
152
153 0000000000001020 <puts@plt-0x10>:
154 1020: ff 35 ca 2f 00 00 push 0x2fca(%rip) # 3ff0
155 <_GLOBAL_OFFSET_TABLE_+0x8>
156 1026: ff 25 cc 2f 00 00 jmp *0x2fcc(%rip) # 3ff8
157 <_GLOBAL_OFFSET_TABLE_+0x10>
158 102c: 0f 1f 40 00 nopl 0x0(%rax)
159
160 0000000000001030 <puts@plt>:
161 1030: ff 25 ca 2f 00 00 jmp *0x2fca(%rip) # 4000
162 <puts@GLIBC_2.2.5>
163 1036: 68 00 00 00 00 push $0x0
164 103b: e9 e0 ff ff ff jmp 1020 <_init+0x20>
165
166 Disassembly of section .plt.got:
167
168 0000000000001040 <__cxa_finalize@plt>:
169 1040: ff 25 9a 2f 00 00 jmp *0x2f9a(%rip) # 3fe0
170 <__cxa_finalize@GLIBC_2.2.5>
171 1046: 66 90 xchg %ax,%ax
172
173 Disassembly of section .text:

```

```

170 0000000000001050 <_start>:
171 1050: 31 ed xor %ebp,%ebp
172 1052: 49 89 d1 mov %rdx,%r9
173 1055: 5e pop %rsi
174 1056: 48 89 e2 mov %rsp,%rdx
175 1059: 48 83 e4 f0 and $0xfffffffffffffffff0,%rsp
176 105d: 50 push %rax
177 105e: 54 push %rsp
178 105f: 45 31 c0 xor %r8d,%r8d
179 1062: 31 c9 xor %ecx,%ecx
180 1064: 48 8d 3d dc 00 00 00 lea 0xdc(%rip),%rdi # 1147 <main>
181 106b: ff 15 4f 2f 00 00 call *0x2f4f(%rip) # 3fc0
 <__libc_start_main@GLIBC_2.34>
182 1071: f4 hlt
183 1072: 66 2e 0f 1f 84 00 00 cs nopw 0x0(%rax,%rax,1)
184 1079: 00 00 00
185 107c: 0f 1f 40 00 nopl 0x0(%rax)
186
187 0000000000001080 <deregister_tm_clones>:
188 1080: 48 8d 3d 91 2f 00 00 lea 0x2f91(%rip),%rdi # 4018
 <__TMC_END__>
189 1087: 48 8d 05 8a 2f 00 00 lea 0x2f8a(%rip),%rax # 4018
 <__TMC_END__>
190 108e: 48 39 f8 cmp %rdi,%rax
191 1091: 74 15 je 10a8 <deregister_tm_clones+0x28>
192 1093: 48 8b 05 2e 2f 00 00 mov 0x2f2e(%rip),%rax # 3fc8
 <__ITM_deregisterTMCloneTable@Base>
193 109a: 48 85 c0 test %rax,%rax
194 109d: 74 09 je 10a8 <deregister_tm_clones+0x28>
195 109f: ff e0 jmp *%rax
196 10a1: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
197 10a8: c3 ret
198 10a9: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
199
200 00000000000010b0 <register_tm_clones>:
201 10b0: 48 8d 3d 61 2f 00 00 lea 0x2f61(%rip),%rdi # 4018
 <__TMC_END__>
202 10b7: 48 8d 35 5a 2f 00 00 lea 0x2f5a(%rip),%rsi # 4018
 <__TMC_END__>
203 10be: 48 29 fe sub %rdi,%rsi
204 10c1: 48 89 f0 mov %rsi,%rax
205 10c4: 48 c1 ee 3f shr $0x3f,%rsi
206 10c8: 48 c1 f8 03 sar $0x3,%rax
207 10cc: 48 01 c6 add %rax,%rsi
208 10cf: 48 d1 fe sar $1,%rsi
209 10d2: 74 14 je 10e8 <register_tm_clones+0x38>
210 10d4: 48 8b 05 fd 2e 00 00 mov 0x2efd(%rip),%rax # 3fd8
 <__ITM_registerTMCloneTable@Base>
211 10db: 48 85 c0 test %rax,%rax
212 10de: 74 08 je 10e8 <register_tm_clones+0x38>
213 10e0: ff e0 jmp *%rax
214 10e2: 66 0f 1f 44 00 00 nopw 0x0(%rax,%rax,1)
215 10e8: c3 ret

```

```

216 10e9: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
217
218 000000000000010f0 <__do_global_dtors_aux>:
219 10f0: f3 0f 1e fa endbr64
220 10f4: 80 3d 1d 2f 00 00 00 cmpb $0x0,0x2f1d(%rip) # 4018
 <__TMC_END__>
221 10fb: 75 2b jne 1128 <__do_global_dtors_aux+0x38>
222 10fd: 55 push %rbp
223 10fe: 48 83 3d da 2e 00 00 cmpq $0x0,0x2eda(%rip) # 3fe0
 <__cxa_finalize@GLIBC_2.2.5>
224 1105: 00
225 1106: 48 89 e5 mov %rsp,%rbp
226 1109: 74 0c je 1117 <__do_global_dtors_aux+0x27>
227 110b: 48 8b 3d fe 2e 00 00 mov 0x2efe(%rip),%rdi # 4010
 <__dso_handle>
228 1112: e8 29 ff ff ff call 1040 <__cxa_finalize@plt>
229 1117: e8 64 ff ff ff call 1080 <deregister_tm_clones>
230 111c: c6 05 f5 2e 00 00 01 movb $0x1,0x2ef5(%rip) # 4018
 <__TMC_END__>
231 1123: 5d pop %rbp
232 1124: c3 ret
233 1125: 0f 1f 00 nopl (%rax)
234 1128: c3 ret
235 1129: 0f 1f 80 00 00 00 00 nopl 0x0(%rax)
236
237 00000000000001130 <frame_dummy>:
238 1130: f3 0f 1e fa endbr64
239 1134: e9 77 ff ff ff jmp 10b0 <register_tm_clones>
240
241 00000000000001139 <test01>:
242 1139: 55 push %rbp
243 113a: 48 89 e5 mov %rsp,%rbp
244 113d: 90 nop
245 113e: 5d pop %rbp
246 113f: c3 ret
247
248 00000000000001140 <test02>:
249 1140: 55 push %rbp
250 1141: 48 89 e5 mov %rsp,%rbp
251 1144: 90 nop
252 1145: 5d pop %rbp
253 1146: c3 ret
254
255 00000000000001147 <main>:
256 1147: 55 push %rbp
257 1148: 48 89 e5 mov %rsp,%rbp
258 114b: 48 8d 05 b2 0e 00 00 lea 0xeb2(%rip),%rax # 2004
 <_IO_stdin_used+0x4>
259 1152: 48 89 c7 mov %rax,%rdi
260 1155: e8 d6 fe ff ff call 1030 <puts@plt>
261 115a: b8 00 00 00 00 mov $0x0,%eax
262 115f: e8 d5 ff ff ff call 1139 <test01>
263 1164: b8 00 00 00 00 mov $0x0,%eax

```

```

264 1169: e8 d2 ff ff ff call 1140 <test02>
265 116e: b8 00 00 00 00 mov $0x0,%eax
266 1173: 5d pop %rbp
267 1174: c3 ret
268
269 Disassembly of section .fini:
270
271 0000000000001178 <_fini>:
272 1178: 48 83 ec 08 sub $0x8,%rsp
273 117c: 48 83 c4 08 add $0x8,%rsp
274 1180: c3 ret
275

```

```

1 # 提取可打印字符 (strip对这个没有影响)
2 siguyuan@debian:~/EBF_6ULL/test$ strings a_x86
3 /lib64/ld-linux-x86-64.so.2
4 puts
5 __libc_start_main
6 __cxa_finalize
7 libc.so.6
8 GLIBC_2.2.5
9 GLIBC_2.34
10 _ITM_deregisterTMCloneTable
11 __gmon_start__
12 _ITM_registerTMCloneTable
13 PTE1
14 u+UH
15 ===== # 自己在printf输出的字符串常量
16 ;*3$"
17 GCC: (Debian 14.2.0-19) 14.2.0
18 Scrt1.o
19 __abi_tag
20 crtstuff.c
21 deregister_tm_clones
22 __do_global_dtors_aux
23 completed.0
24 __do_global_dtors_aux_fini_array_entry
25 frame_dummy
26 __frame_dummy_init_array_entry
27 aa.c
28 __FRAME_END__
29 _DYNAMIC
30 __GNU_EH_FRAME_HDR
31 _GLOBAL_OFFSET_TABLE_
32 __libc_start_main@GLIBC_2.34
33 _ITM_deregisterTMCloneTable
34 test01
35 puts@GLIBC_2.2.5
36 _edata
37 _fini
38 __data_start

```

```
39 __gmon_start__
40 __dso_handle
41 _IO_stdin_used
42 _end
43 test02
44 __bss_start
45 main
46 __TMC_END__
47 _ITM_registerTMCloneTable
48 __cxa_finalize@GLIBC_2.2.5
49 _init
50 .symtab
51 .strtab
52 .shstrtab
53 .note.gnu.property
54 .note.gnu.build-id
55 .interp
56 .gnu.hash
57 .dynsym
58 .dynstr
59 .gnu.version
60 .gnu.version_r
61 .rela.dyn
62 .rela.plt
63 .init
64 .plt.got
65 .text
66 .fini
67 .rodata
68 .eh_frame_hdr
69 .eh_frame
70 .note.ABI-tag
71 .init_array
72 .fini_array
73 .dynamic
74 .got.plt
75 .data
76 .bss
77 .comment
78
```

```
1 # 创建汇编文件用于测试
2 # .S支持C的宏，.s不支持
3 siguyuan@debian:~/EBF_6ULL/test$ cat a.s
4 .text
5 SWI 0x66 // 软中断命令
6 loop:
7 b loop
8
9 .end
```

```

10
11 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-gcc -c -o a.o a.S
12 siguyuan@debian:~/EBF_6ULL/test$ ls -l
13 -rw-rw-r-- 1 siguyuan siguyuan 600 8月11日 18:28 a.o
14 -rw-rw-r-- 1 siguyuan siguyuan 98 8月11日 18:28 a.S
15 siguyuan@debian:~/EBF_6ULL/test$ file a.o
16 a.o: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), not stripped # 未剔除
17
18 # 进行连接
19 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-gcc -o a a.o
20 /opt/gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi/bin/../lib/gcc/arm-none-
21 linux-gnueabi/9.2.1/../../../../arm-none-linux-gnueabi/bin/ld: /opt/gcc-arm-9.2-
22 2019.12-x86_64-arm-none-linux-gnueabi/bin/../arm-none-linux-
23 gnueabi/libc/usr/lib/crt1.o: in function `_start':
24 /tmp/dgboter/bbs/rhev-vm7--rhe6x86_64/buildbot/rhe6x86_64--arm-none-linux-
25 gnueabi/build/src/glibc/csu/../sysdeps/arm/start.S:119: undefined reference to
26 `main'
27 collect2: error: ld returned 1 exit status # 缺少main 且使用gcc会将一些别的库和OS初始
28 化代码也放入
29
30 # 使用ld减少不必要的库的引入（生成一个ARM的二进制文件，用于在OS内核成功加载之前运行来处理事务）
31 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-ld -o a a.o
32 arm-none-linux-gnueabi-ld: warning: cannot find entry symbol _start; defaulting to
33 000000000010054 # 缺少开始标志，需要告知ld从哪里开始
34 siguyuan@debian:~/EBF_6ULL/test$ cat a.S
35
36 .text
37 _start: // 添加gcc的ld需要的开始标志
38 SWI 0x66 // 软中断命令
39
40 loop:
41 b loop
42
43 .end
44
45 # 重新编译之后
46 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-nm a.o
47 00000004 t loop
48 00000000 t _start # t局部有效，只能在本文件才识别需要弄成全局
49 siguyuan@debian:~/EBF_6ULL/test$ cat a.S
50
51 .text
52 .global _start // 添加全局表示，.global为gcc汇编语法
53 _start: // 添加gcc的ld需要的开始标志
54 SWI 0x66 // 软中断命令
55
56 loop:
57 b loop
58
59 .end
60
61 # 重新编译之后
62 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-nm a.o
63 00000004 t loop
64 00000000 T _start
65
66 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-ld -o a a.o
67 siguyuan@debian:~/EBF_6ULL/test$ ls -l a
68 -rwxrwxr-x 1 siguyuan siguyuan 712 8月11日 18:44 a

```



```

56 # 此时生成的可执行文件是根据OS来生成的虚拟地址，在没有OS加载之前，需要告知CPU绝对地址
57 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-hf-nm a
58 0002005c T __bss_end__
59 0002005c T _bss_end__
60 0002005c T __bss_start
61 0002005c T __bss_start__
62 0002005c T _edata
63 0002005c T __end__
64 0002005c T _end
65 00010058 t loop
66 00010054 T _start
67
68 # 划为实际的物理地址
69 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-hf-ld -Ttext=0x20008000 -o a
a.o
70 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-hf-nm a
71 20018008 T __bss_end__
72 20018008 T _bss_end__
73 20018008 T __bss_start
74 20018008 T __bss_start__
75 20018008 T _edata
76 20018008 T __end__
77 20018008 T _end
78 20008004 t loop
79 20008000 T _start
80
81 # 添加了一些ELF头 + 内存地址信息
82 siguyuan@debian:~/EBF_6ULL/test$ ls -l a
83 -rwxrwxr-x 1 siguyuan siguyuan 33396 8月11日 18:47 a
84 # 剔除符号表
85 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-hf-strip a
86 siguyuan@debian:~/EBF_6ULL/test$ ls -l a
87 -rwxrwxr-x 1 siguyuan siguyuan 33004 8月11日 18:49 a
88
89 # 将代码段提取出来 a.bin为代码段
90 siguyuan@debian:~/EBF_6ULL/test$ arm-none-linux-gnueabi-hf-objcopy -O binary a a.bin
91 siguyuan@debian:~/EBF_6ULL/test$ ls -l a a.bin
92 -rwxrwxr-x 1 siguyuan siguyuan 33004 8月11日 18:49 a
93 -rwxrwxr-x 1 siguyuan siguyuan 8 8月11日 18:51 a.bin
94

```

## 生成ELF可执行文件

- 1、把高级语言 .c .S生成 .o目标文件
- 2、不能ld命令，而是使用gcc，将.o目标文件 做为链接原材料进行连接，gcc会自动加载标准C库和OS初始化库的内容
- 3、在链接时，交给默认链接脚本实现链接，使用虚拟地址
- 4、默认生成ELF格式的可执行文件

生成ARM的二进制文件（二进制镜像文件）【剔除ELF格式头和其他冗余代码，boot loader 和内核都是这个二进制镜像文件】

1、把高级语言 .c .S生成 .o目标文件

```
arm-none-linux-gnueabi-gcc -c -o a.o a.S
```

2、使用ld命令，将.o目标文件 做为链接原材料进行连接，ld不会自动加载标准C库和OS初始化库的内容

3、在链接时，指定各个段的首地址

```
arm-none-linux-gnueabi-ld -T-Tmap.lds -o a a.o
```

4、默认生成ELF格式的可执行文件

5、利用objcopy实现 ELF去头效果，拷贝对应的段到二进制镜像文件

```
arm-none-linux-gnueabi-objcopy -O binary a a.bin
```

ld的连接脚本：

```
1 /* 示例: map.lds */
2 OUTPUT_FORMAT("elf32-littlearm", "elf32-littlearm", "elf32-littlearm")
3 /*OUTPUT_FORMAT("elf32-arm", "elf32-arm", "elf32-arm")*/
4 OUTPUT_ARCH(arm)
5 ENTRY(_start) /* 1. 显式指定入口标签为 _start */
6
7 SECTIONS
8 {
9 . = 0x80000000; /* 2. 将定位计数器设为物理内存/DDR的首地址 需要修改 */
10
11 .text : { /* 3. 定义代码段 */
12 start.o (.text) /* 确保 start.o 的代码段放在最前面（最先执行） 需要修改 */
13 *(.text) /* 其他所有 .o 文件的代码段紧随其后 */
14 }
15
16 . = ALIGN(4); /* 4. 4 字节对齐 */
17 .rodata : { *(.rodata) } /* 只读数据段 */
18
19 . = ALIGN(4);
20 .data : { *(.data) } /* 已初始化数据段 */
21
22 . = ALIGN(4);
23 __bss_start = .; /* 记录 BSS 段起始地址，供 C 语言清除内存使用 */
24 .bss : { *(.bss) } /* 未初始化数据段 */
25 __bss_end = .; /* 记录 BSS 段结束地址 */
26 }
```

生成makefile脚本

```
1 CROSS_COMPILE = arm-none-linux-gnueabi- # 需要烧入板子的对编译器 需要改
2 CC = $(CROSS_COMPILE)gcc
3 OBJCOPY = $(CROSS_COMPILE)objcopy
```

```

4 LD = $(CROSS_COMPILE)ld
5
6 OBJS += start.o
7 CFLGAS += -nostdinc
8 LDFLAGS += -nostdlib -Tmap.lds
9
10 TARGET := build.bin
11
12 all:$(TARGET)
13 $(TARGET):build
14 $(OBJCOPY) -S --gap-fill 0xff -O binary $^ $@ # -S: 剔除 (Strip) 目标文件中的所有
 符号表 (Symbol table) 和重定位信息 --gap-fill 0xff: 用指定的十六进制数值 0xFF 去填充各个段
 (section) 之间的空隙
15
16 build:$(OBJS)
17 $(LD) $(LDFLAGS) -o $@ $^
18
19 %.o:%.c
20 $(CC) -c $(CFLAGS) -o $@ $<
21 %.o:%.S
22 $(CC) -c $(CFLAGS) -o $@ $<
23 .PYTHON:clean
24 clean:
25 rm -f *.o build build.bin
26

```

## 3.2 bootloader搭建（最小系统搭建）

最小系统：保证微处理器（如 CPU、内存、串口、flash 等）**正常工作、运行最基础程序所需的最少硬件电路组合**。

Linux系统被设计成“被动加载”的形式，必须有一个引导程序——boot loader，来加载内核镜像文件（纯二进制文件，没有ELF头）。

boot loader程序的获取来源：

- 芯片厂商提供的原厂程序（BSPs / SDKs）：
  1. 根据开源项目，针对自家芯片进行移植和二次开发的程序。
  2. 完全自研的程序。
- 开源 Bootloader（开源社区）：支持成百上千种不同的芯片架构和单板。它们通过非常成熟的**配置机制**（如类似于 Linux 内核的 `kconfig` + `make menuconfig` / DeviceTree）来适配特定芯片与硬件板级外设。如：Uboot

boot loader按功能区分：

- 交互式bootloader（研发/调试阶段使用）：允许开发者通过串口（UART）、网口或 USB 输入指令，控制启动流程。
  - 典型行为：
    - **启动等待倒计时**：开机时停顿 1~3 秒，打印类似 `Press any key to stop autoboot` 的提示
    - **进入命令行**：按下任意键可打断自动引导，进入交互 Shell（如 U-Boot 命令行）。

- 非交互式bootloader（量产/发布阶段使用）：全自动静默运行，取消交互界面，追求**极速启动与系统安全**。
  - 典型行为：
    - 移除等待延时，上电即加载内核
    - 关闭/裁剪控制台
    - 启用安全校验，开启 Secure Boot，校验内核和设备树的签名，阻止未授权固件运行。

本次实验使用的是由野火公司提供的针对 EBF6ULL 的定制U-boot

如何移植Uboot到开发板上：

1. 下载对应板子的Uboot版本的源码， `uboot-ebf_v2020_10_imx.tar.gz`
2. 解压 `tar -zxvf uboot-ebf_v2020_10_imx.tar.gz -C ../uboot-ebf/`
3. 对Uboot编译前的配置，配置哪些文件参与编译

- 1 |

```
1 # 解压之后的目录
2 siguyuan@debian:~/EBF_6ULL/uboot-ebf/uboot-ebf_v2020_10_imx$ ls
3 api board common configs doc dtoverlay env fs kbuild lib
4 MAINTAINERS net README test
5
6 # 主Makefile中，发现需要定义CROSS_COMPILE才能使用交叉编译器
7 siguyuan@debian:~/EBF_6ULL/uboot-ebf/uboot-ebf_v2020_10_imx$ nvim Makefile +390
8 AS = $(CROSS_COMPILE)as
9 # Always use GNU ld
10 ifneq ($(shell $(CROSS_COMPILE)ld.bfd -v 2> /dev/null),)
11 LD = $(CROSS_COMPILE)ld.bfd
12 else
13 LD = $(CROSS_COMPILE)ld
14 endif
15 CC = $(CROSS_COMPILE)gcc
16 CPP = $(CC) -E
17 AR = $(CROSS_COMPILE)ar
18 NM = $(CROSS_COMPILE)nm
19 LDR = $(CROSS_COMPILE)ldr
20 STRIP = $(CROSS_COMPILE)strip
21 OBJCOPY = $(CROSS_COMPILE)objcopy
22 OBJDUMP = $(CROSS_COMPILE)objdump
23 LEX = flex
24 YACC = bison
25 AWK = awk
26 PERL = perl
27 PYTHON ?= python
28 PYTHON2 = python2
29 PYTHON3 = python3
30 DTC ?= $(objtree)/scripts/dtc/dtc
```

```

31 CHECK = sparse
32
33 ...

```

- 每个目录下都有自己的Makefile进行配置
- **arch**: CPU体系架构强相关的代码目录。架构初始化（如 `start.S` 汇编）、中断处理、CPU 相关的低级硬件初始化都在这里。
- **board**: 板卡（Board）的硬件配置与板级初始化代码目录。它会调用 `arch` 目录提供的底层接口，完成板载外设（如 PMIC、DDR 参数、电源引脚）的初始化。
- **cmd**: 定义 U-Boot 交互控制台下的指令集目录，决定有哪些命令。如果想在 U-Boot 中添加自定义的交互命令，就是在 `cmd/` 目录下新增 `.c` 文件并用 `U_BOOT_CMD` 宏注册。
- **configs**: 存放默认配置文件目录。执行 `make xxx_defconfig` 时，构建系统就会把 `configs/` 下对应的配置文件复制到根目录下的 `.config` 中，作为当前项目的全局编译开关。
- Makefile中的 `obj-y` 表示将对应的 `.o` 目标文件编译并打包进最终的镜像；`obj-`（或未指定）则不参与编译。通常配合配置文件（`.config`）中的宏使用

```

1

```

```

1 obj-$(CONFIG_CMD_NET) += net.o
2 ...
3
4 - 当 `.config` 中定义了 `CONFIG_CMD_NET=y` 时，这一行自动展开为 `obj-y += net.o`（编译）；若定义为 `n` 或未定义，则展开为 `obj-` 或 `obj-n`（不编译）。

```

- 从参考配置单目录configs下，找到一个适合我们开发板的配置单，再修改成独立（厂家会告知需要激活哪些配置单）—— `mx6ull_fire_mmc_defconfig` 由野火公司提供的配置单
- **步骤1: 在 `uboot-ebf_v2020_10_imx` 目录下make执行对应的配置单**

```

1 # 运行出现错误（注意：一次执行可能之后报出一个错误）
2 siguyuan@debian:~/EBF_6ULL/uboot-ebf/uboot-ebf_v2020_10_imx$ make
3 mx6ull_fire_mmc_defconfig
4 HOSTCC scripts/basic/fixdep
5 HOSTCC scripts/kconfig/conf.o
6 YACC scripts/kconfig/zconf.tab.c
7 /bin/sh: 1: bison: not found
8 make[1]: *** [scripts/Makefile.lib:222: scripts/kconfig/zconf.tab.c] 错误
9 127
10 make: *** [Makefile:565: mx6ull_fire_mmc_defconfig] 错误 2
11
12 # 根据错误提示安装对应的工具，如：bison工具
13 sudo apt update
14 sudo apt install bison

```

```

o 1 # 执行成功后的现象
2 siguyuan@debian:~/EBF_6ULL/u-boot-ebf/u-boot-ebf_v2020_10_imx$ make
 mx6ull_fire_mmc_defconfig
3 LEX scripts/kconfig/zconf.lex.c
4 HOSTCC scripts/kconfig/zconf.tab.o
5 HOSTLD scripts/kconfig/conf
6 #
7 # configuration written to .config
8 #
9
10 # 成功之后会在当前目录生成.config文件，Makefile会根据这个文件去看哪些文件编译，哪些文件不编译
11 # 如果想要在vscode中能自由跳转，需要使用bear工具生成clangd可以识别的json文件

```

## • 步骤2: 生成Uboot镜像文件

```
1 |
```

```

1 siguyuan@debian:~/EBF_6ULL/u-boot-ebf/u-boot-ebf_v2020_10_imx$ make
2 scripts/kconfig/conf --syncconfig Kconfig
3 UPD include/config.h
4 CFG u-boot.cfg
5 GEN include/autoconf.mk
6 GEN include/autoconf.mk.dep
7 CFGCHK u-boot.cfg
8 UPD include/config/u-boot.release
9 UPD include/generated/version_autogenerated.h
10 UPD include/generated/timestamp_autogenerated.h
11 UPD include/generated/dt.h
12 CC lib/asm-offsets.s
13 cc1: error: bad value 'generic-armv7-a' for '-mtune=' switch
14 cc1: note: valid arguments to '-mtune=' switch are: nocona core2 nehalem corei7
westmere sandybridge corei7-avx ivybridge core-avx-i haswell core-avx2 broadwell
skylake skylake-avx512 cannonlake icelake-client rocketlake icelake-server cascadelake
tigerlake cooperlake sapphirerapids emeraldrapids alderlake raptorlake meteorlake
graniterapids graniterapids-d arrowlake arrowlake-s lunarlake pantherlake bonnell atom
silvermont slm goldmont goldmont-plus tremont gracemont sierraforest grandridge
clearwaterforest knl knm intel x86-64 eden-x2 nano nano-1000 nano-2000 nano-3000 nano-
x2 eden-x4 nano-x4 lujiazui yongfeng k8 k8-sse3 opteron opteron-sse3 athlon64
athlon64-sse3 athlon-fx amdfam10 barcelona bdver1 bdver2 bdver3 bdver4 znver1 znver2
znver3 znver4 znver5 btver1 btver2 generic native
15 make[1]: *** [scripts/Makefile.build:155: lib/asm-offsets.s] 错误 1
16 make: *** [Makefile:1834: prepare0] 错误 2
17
18 # 这个错误是因为，当前终端的编译器是x86的gcc，不是交叉编辑器
19 siguyuan@debian:~/EBF_6ULL/u-boot-ebf/u-boot-ebf_v2020_10_imx$ cat
 ../../env/arm_path.sh
20 #!/bin/bash
21
22 # 交叉编译工具链的环境变量配置
23 export PATH=$PATH:/opt/arm-linux-gnueabihf/bin
24 # 告知交叉编译器的前缀

```

```

25 export CROSS_COMPILE=arm-none-linux-gnueabi-
26
27 # 将交叉编辑器的bin目录添加到环境变量中
28 siguyuan@debian:~/EBF_6ULL/u-boot-ebf/u-boot-ebf_v2020_10_imx$ source
 ../../env/arm_path.sh
29
30 # -j$(nproc) 结合系统的 CPU 核心数变量来实现自动分配多线程编译
31 siguyuan@debian:~/EBF_6ULL/u-boot-ebf/u-boot-ebf_v2020_10_imx$ make -j$(nproc)
32 # HOSTCC表示编译宿主主机使用的工具 tools/
33 HOSTCC tools/lpc32xximage.o
34 HOSTCC tools/mxsimage.o
35 HOSTCC tools/omapimage.o
36 HOSTCC tools/pblimage.o
37 HOSTCC tools/pbl_crc32.o
38 HOSTCC tools/vybridimage.o
39 HOSTCC tools/stm32image.o
40 WRAP tools/lib/rc4.c
41 ...
42 # 目标机的文件 arch/
43 LD u-boot # 生成ELF头的可执行文件
44 OBJCOPY u-boot-nodtb.bin
45 OBJCOPY u-boot.srec
46 DTC arch/arm/dts/imx6ull-14x14-evk.dtb
47 SYM u-boot.sym
48 DTC arch/arm/dts/imx6ull-fire-mmc.dtb
49 DTC arch/arm/dts/imx6ull-fire-nand.dtb
50 DTC arch/arm/dts/imx6ull-colibri.dtb
51 DTC arch/arm/dts/imx6ull-mys-6ulx-eval.dtb
52 DTC arch/arm/dts/imx6ull-phytec-segin-ff-rdk-emmc.dtb
53 DTC arch/arm/dts/imx6ull-dart-6ul.dtb
54 DTC arch/arm/dts/imx6ull-somlabs-visionsom.dtb
55 DTC arch/arm/dts/imx6ulz-14x14-evk.dtb
56 DTC arch/arm/dts/imx6-apalis.dtb
57 DTC arch/arm/dts/imx6-colibri.dtb
58 SHIPPED dts/dt.dtb
59 FDTGREP dts/dt-spl.dtb
60 CAT u-boot-dtb.bin
61 COPY u-boot.dtb
62 COPY u-boot.bin # 镜像文件
63 CFGS u-boot-dtb.cfgout
64 MKIMAGE u-boot-dtb.imx # 根据芯片内部厂家的bootloader的规则生成的镜像文件
65
66 ...
67
68 - 有可能会出现缺少openssl库和dev库
69
70 - `make distclean`: 清除目标文件和配置文件
71
72 - 编译成功后, 生成一个u-boot的可执行文件, (静态文件)、u-boot-nodtb.bin(镜像文件)、u-boot-
 dtb,imx(最后根据芯片内自动的芯片产生的bootloader规则生成的镜像文件)
73
74 - 将`u-boot-dtb.imx`拷贝到输出目录`cp u-boot-dtb.imx ../../out_dir/`

```

- 步骤3: 擦除SD卡数据 (擦除过程慢很正常)

- `lsblk`: 以树状结构列出系统中的所有块设备 (如硬盘、固态硬盘、SD 卡、U 盘、光驱等) 及其分区情况

- 1 |

```
1 siguyuan@debian:~/EBF_6ULL/uboot-ebf/uboot-ebf_v2020_10_imx$ lsblk
2 NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
3 sda 8:0 0 100G 0 disk
4 └─sda1 8:1 0 98G 0 part /
5 └─sda2 8:2 0 1K 0 part
6 └─sda5 8:5 0 2G 0 part [SWAP]
7 sdb 8:16 1 29.7G 0 disk # SD卡的目录
8 └─sdb1 8:17 1 50M 0 part
9 └─sdb2 8:18 1 148M 0 part
10 sr0 11:0 1 1024M 0 rom
11
12 ...
13
14 - `dd`: **按字节或数据块 (Block) 进行逐位 (Bit-by-Bit) 的低级底层复制**, 完全不在乎文件系统的类型
15
16 - ```bash
17 # 清空SD卡前4M
18 # if后为填充的值, of后面跟的是擦除设备
19 siguyuan@debian:~/EBF_6ULL$ sudo dd iflag=dsync oflag=dsync if=/dev/zero
20 of=/dev/sdb bs=512 count=8192
21 输入了 8192+0 块记录
22 输出了 8192+0 块记录
23 4194304 字节 (4.2 MB, 4.0 MiB) 已复制, 44.5842 s, 94.1 kB/s
24
25 siguyuan@debian:~/EBF_6ULL$ lsblk
26 NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
27 sda 8:0 0 100G 0 disk
28 └─sda1 8:1 0 98G 0 part /
29 └─sda2 8:2 0 1K 0 part
30 └─sda5 8:5 0 2G 0 part [SWAP]
31 sdb 8:16 1 29.7G 0 disk
32 sr0 11:0 1 1024M 0 rom
33
34 ...
35
36 - > [!CAUTION]
37 >
38 > 必须确认路径是否正常
39
40 - ```bash
41 # 烧写uboot
42 # 注意if输入和of输出
43 siguyuan@debian:~/EBF_6ULL/out_dir$ sudo dd iflag=dsync oflag=dsync
44 if=/home/siguyuan/EBF_6ULL/out_dir/u-boot-dtb.imx of=/dev/sdb seek=2
45 输入了 782+0 块记录
46 输出了 782+0 块记录
47 400384 字节 (400 kB, 391 KiB) 已复制, 4.23605 s, 94.5 kB/s
```



```
46
47 ...
48
49 -
```

#### 4. 烧写（斜体代码都是Uboot的命令，正常的是Linux命令）

- 连接USB串口，第一次启动，需要快速按下空格键。否则会进入野火厂家的Bootloader 的启动。
- 连接好以后，设置进入等待时间，
  - `setenv bootdelay 2`：设置将开机倒计时等待时间设置为 2 秒。
  - `saveenv`：保存设置

#### 5. 验证，界面最上面为当前时间，即为成功。

### 如何从宿主机里加载一段数据到目标机的指定内存上？

#### 1. 宿主机与目标机的连接方式（一般使用网络连接传输文件，串口传输太慢了），本次使用方式2 —— 网络层配置

##### 1. 方式1：双网卡直连（PC 扩展网卡）。板子 → PC 扩展网口 → 虚拟机网卡2（桥接）

- 虚拟扩展网口，和两根网线

##### 2. 方式2：路由器桥接（物理路由器 WAN/LAN）。板子 → 路由器 → PC 上网网口 → 虚拟机网卡2（桥接）。

- 一个扩展网口，一根网线，一根联网的路由器（构建局域网）
- 配置板子的IP
  - `*setenv ipaddr 192.168.50.50`
  - `printfenv ipaddr`:查看ip配置
  - `ping 192.168.50.51`: ping宿主机的IP，查看是否连通

#### ④ Note

- 因为板子资源有限，所以有些命令是精简命令，如：ping命令
- ping命令没有回应主机的代码
- 在虚拟机中运行 `ping <开发板IP>` 时，始终无法收到回应报文（显示超时或无响应）；但实际网络链路、TFTP 及 NFS 共享均正常工作。
  - **ICMP 响应机制被精简/裁减**：开发板运行的嵌入式系统（如 BusyBox 或精简版 U-Boot/Linux）裁剪了 ICMP Echo Reply（回显响应）逻辑，或防火墙/内核未使能对 Ping 请求的处理，导致其接收到包后**不予回发**。
  - **网络协议栈为单向主动设计**：在 U-Boot 阶段或极简网络栈中，网络驱动仅实现了“主动发送并接收回复”的单向逻辑，不具备作为 Ping 服务端回应外部 ICMP 请求的能力。
- 正确做法：在开发板终端运行 `ping <虚拟机IP>`，只要有数据包返回，即可**100% 确认双向物理链路和 IP 配置完全正确**。

```

1 => setenv ipaddr 192.168.50.50 # 设置本地IP
2 => printenv ipaddr # 查看设置的IP
3 ipaddr=192.168.50.50
4
5 => ping 192.168.50.51 # 与宿主机检查连接情况
6 Using ethernet@20b4000 device
7 host 192.168.50.51 is alive
8

```

- 为了防止虚拟机关机之后，虚拟机的IP丢失，写一个配置脚本 `net.sh` (在 `/env` 目录下),如果IP改变，运行这个脚本 `./net.sh`

```

1 #!/bin/bash
2
3 sudo ifconfig ens37 192.168.50.50

```

■

2. 板子传输层选择UDP协议，进行快速传输。

3. 板子上的应用层使用tftp客户（UDP协议），虚拟机上需要tftp服务 或者 atftpd

○ 虚拟机

```

1 # 安装tftpd服务端
2 sudo apt install tftpd-hpa
3
4 # 配置tftp 配置文件: /etc/default/tftpd-hpa
5 sudo nvim /etc/default/tftpd-hpa
6
7 # 修改内容
8 # -----
9 # /etc/default/tftpd-hpa
10
11 TFTP_USERNAME="siguyuan"
12 TFTP_DIRECTORY="/home/siguyuan/EBF_6ULL/out_dir" # 告知共享目录
13 TFTP_ADDRESS="0.0.0.0:69" # 使用IPv4, 板子不支持IPv6
14 TFTP_OPTIONS="--secure"
15
16 # -----
17
18 # 重启服务
19 sudo service tftpd-hpa restart
20
21 # 查看端口
22 sudo netstat -aun

```

```

1 # 错误: 目录不存在的错误
2 siguyuan@debian:~/EBF_6ULL/env$ sudo nvim /etc/default/tftpd-hpa
3 siguyuan@debian:~/EBF_6ULL/env$ sudo chown -R tftp:tftp
 /home/siguyuan/EBF_6ULL/out_dir

```

```

4 siguyuan@debian:~/EBF_6ULL/env$ sudo chmod 777
/home/siguyuan/EBF_6ULL/out_dir
5
6 # 重启服务出现报错
7 siguyuan@debian:~/EBF_6ULL/env$ sudo systemctl restart tftpd-hpa
8 Job for tftpd-hpa.service failed because the control process exited
with error code.
9 See "systemctl status tftpd-hpa.service" and "journalctl -xeu tftpd-
hpa.service" for details.
10 siguyuan@debian:~/EBF_6ULL/env$ cat nvim /etc/de
11 bconf.conf debian_version default/ deluser.conf
depmod.d/
12 siguyuan@debian:~/EBF_6ULL/env$ cat nvim /etc/default/tftpd-hpa
13 cat: nvim: 没有那个文件或目录
14 # /etc/default/tftpd-hpa
15 TFTP_USERNAME="tftp"
16 TFTP_DIRECTORY="/home/siguyuan/EBF_6ULL/out_dir"
17 TFTP_ADDRESS=":69"
18 TFTP_OPTIONS="--secure"
19
20 # 查看报错，提示目录不存在，即便目录存在，文件存在，权限正确
21 siguyuan@debian:~/EBF_6ULL/env$ sudo systemctl status tftpd-
hpa.service --no-pager -l
22 x tftpd-hpa.service - tftpd-hpa TFTP Server
23 Loaded: loaded (/usr/lib/systemd/system/tftpd-hpa.service;
enabled; preset: enabled)
24 Active: failed (Result: exit-code) since Wed 2026-08-12 19:11:00
CST; 46s ago
25 Duration: 19min 11.217s
26 Invocation: 9b09e48d654a402d868256fc200a18a7
27 Docs: man:in.tftpd
28 Process: 8314 ExecStart=/usr/sbin/in.tftpd --listen --user
$TFTP_USERNAME --address $TFTP_ADDRESS $TFTP_OPTIONS $TFTP_DIRECTORY
(code=exited, status=66)
29 Mem peak: 2.1M
30 CPU: 23ms
31 8月 12 19:11:00 debian systemd[1]: Starting tftpd-hpa.service -
tftpd-hpa TFTP Server...
32 8月 12 19:11:00 debian in.tftpd[8314]:
/home/siguyuan/EBF_6ULL/out_dir: No such file or directory
33 8月 12 19:11:00 debian systemd[1]: tftpd-hpa.service: Control process
exited, code=exited, status=66/NOINPUT
34 8月 12 19:11:00 debian systemd[1]: tftpd-hpa.service: Failed with
result 'exit-code'.
35 8月 12 19:11:00 debian systemd[1]: Failed to start tftpd-hpa.service
- tftpd-hpa TFTP Server.
36
37 siguyuan@debian:~/EBF_6ULL/env$ ls -l
/home/siguyuan/EBF_6ULL/out_dir/
38 总计 392
39 -rw-rw-r-- 1 tftp tftp 400384 8月12日 16:33 u-boot-dtb.imx
40 siguyuan@debian:~/EBF_6ULL/env$ ls -l /home/siguyuan/EBF_6ULL/
41 总计 20

```

```

42 drwxrwxr-x 2 siguyuan siguyuan 4096 8月12日 18:48 env
43 drwxrwxrwx 2 tftp tftp 4096 8月12日 16:33 out_dir
44 drwxrwxr-x 2 siguyuan siguyuan 4096 8月12日 15:32 packages
45 drwxrwxr-x 2 siguyuan siguyuan 4096 8月11日 19:08 test
46 drwxrwxr-x 3 siguyuan siguyuan 4096 8月12日 15:33 uboot-ebf
47 siguyuan@debian:~/EBF_6ULL/env$
48
49 # Debian默认给用户 home 目录设置为 750 (drwxr-x---) 权限，这是系统设计上的隐
 私保护机制，导致tftp无法访问
50 # 解决方式
51 # 在/opt目录下创建一个新的目录 或者 移动过去 ；
52 # 将家目录修改成777 【十分不推荐】
53 sudo mv /home/siguyuan/EBF_6ULL/out_dir /opt/
54 sudo mkdir /opt/out_dir/EBF_6ULL
55 sudo mv /opt/out_dir /opt/out_dir/EBF_6ULL
56
57 # 修改tftp配置文件（注意：不支持末尾注释，只能单独行注释）
58 siguyuan@debian:~/EBF_6ULL/env$ cat /etc/default/tftpd-hpa
59 # /etc/default/tftpd-hpa
60
61 TFTP_USERNAME="siguyuan"
62 # 告知共享目录
63 TFTP_DIRECTORY="/opt/out_dir/EBF_6ULL"
64 # 使用IPv4，板子不支持Ipv6
65 TFTP_ADDRESS="0.0.0.0:69"
66 TFTP_OPTIONS="--secure"
67
68 # 可以使用下面命令进行命令行写入
69 sudo tee /etc/default/tftpd-hpa > /dev/null <<'EOF'
70 # TFTP服务配置文件
71 # TFTP_USERNAME：运行tftpd的用户
72 TFTP_USERNAME="tftp"
73 # TFTP_DIRECTORY：共享目录
74 TFTP_DIRECTORY="/opt/out_dir/EBF_6ULL"
75 # TFTP_ADDRESS：监听地址，使用IPv4（板子不支持IPv6）
76 TFTP_ADDRESS="0.0.0.0:69"
77 TFTP_OPTIONS="--secure"
78 EOF
79
80
81 # 重启服务
82 sudo service tftpd-hpa restart
83
84 # 查看端口（69端口启动，且前面不是udp6）
85 siguyuan@debian:~/EBF_6ULL/env$ sudo netstat -aun
86 Active Internet connections (servers and established)
87 Proto Recv-Q Send-Q Local Address Foreign Address
88 State
89 udp 0 0 10.128.0.128:68 0.0.0.0:*
90 udp 0 0 0.0.0.0:69 0.0.0.0:*
91 udp6 0 0 fe80::fffc8:a1f3:f13:546 :::*
92
93 # 注意目录的权限问题

```

- 板子

- 确认tftp服务端IP是否配置正确

- ```

1  => printenv serverip
2  ## Error: "serverip" not defined
3  # 设置成宿主主的IP setenv把环境变量修改保存在了系统的临时内存（RAM）中
4  # saveenv将当前内存中的所有环境变量写入到物理存储介质
5  => setenv serverip 192.168.50.51
6  => printenv serverip
7  serverip=192.168.50.51
      
```

- 修改测试文件并编译

- ```

1 // led.c
2 // 寄存器写法
3 //时钟控制寄存器
4 #define CCM_CCGR1 (volatile unsigned long*)0x20C406C
5 //GPIO1_04复用功能选择寄存器
6 #define IOMUXC_SW_MUX_CTL_PAD_GPIO1_I004 (volatile unsigned
 long*)0x20E006C
7 //PAD属性设置寄存器
8 #define IOMUXC_SW_PAD_CTL_PAD_GPIO1_I004 (volatile unsigned
 long*)0x20E02F8
9 //GPIO方向设置寄存器（输入或输出）
10 #define GPIO1_GDIR (volatile unsigned long*)0x0209C004
11 //GPIO输出状态寄存器
12 #define GPIO1_DR (volatile unsigned long*)0x0209C000
13
14 typedef unsigned int uint32_t;
15
16 /*简单延时函数*/
17 void delay(uint32_t count) {
18 volatile uint32_t i = 0;
19 for (i = 0; i < count; ++i) {
20 __asm("NOP"); /* 调用nop空指令 */
21 }
22 }
23
24 int main() {
25 *(CCM_CCGR1) = (0x3 << 26) | (0x3 << 18); // 开启GPIO1
 的时钟
26 *(IOMUXC_SW_MUX_CTL_PAD_GPIO1_I004) = 0x5; // 设置PAD复
 用功能为GPIO
27 *(IOMUXC_SW_PAD_CTL_PAD_GPIO1_I004) = 0x1F838; // 设置PAD属
 性
28 *(GPIO1_GDIR) = 0x10; // 设置GPIO
 为输出模式
29 *(GPIO1_DR) = 0x0; // 设置输出电
 平为低电平
30

```

```

31 while(1) {
32 *(GPIO1_DR) = 0x0;
33 delay(0xaFFFFFFF);
34 *(GPIO1_DR) = 1<<4;
35 delay(0xaFFFFFFF);
36 }
37 }

```

```

1 /* led.lds 连接脚本 */
2 ENTRY(_start)
3 SECTIONS {
4 . = 0x80800000;
5
6 . = ALIGN(4);
7 .text : {
8 start.o (.text)
9 *(.text)
10 }
11
12 . = ALIGN(4);
13 .data : {
14 *(.data)
15 }
16
17 . = ALIGN(4);
18 .bss : {
19 *(.bss)
20 }
21 }
22

```

```

1 OBJS += led.o start.o
2
3 CROSS_COMPILE = arm-none-linux-gnueabi-
4 CC = $(CROSS_COMPILE)gcc
5 LD = $(CROSS_COMPILE)ld
6 OBJCOPY = $(CROSS_COMPILE)objcopy
7
8 CFLAGS += -Wall -O2
9 LDFLAGS += -Tled.lds
10 COPY_FLAGS += -S -g
11
12 TARGET = led.bin
13 ELF_TARGET = led.elf
14
15 all:$(TARGET)
16 $(TARGET):$(ELF_TARGET)
17 $(OBJCOPY) -O binary $(COPY_FLAGS) $^ $@
18 sudo cp $@ /opt/out_dir/EBF_6ULL # 复制到到别的目录下
19
20 $(ELF_TARGET):$(OBJS)
21 $(LD) $(LDFLAGS) -o $@ $^

```

```

22
23 %.o : %.c
24 $(CC) -c $(CFLAGS) -o $@ $^
25
26 %.o : %.S
27 $(CC) -c $(CFLAGS) -o $@ $^
28
29 clean:
30 rm -fr $(TARGET) $(ELF_TARGET) $(OBSJ)

```

- ```

1  # led.S
2  # 程序入口的汇编
3  .text                //代码段
4  .align 2             //设置字节对齐
5  .global _start       //定义全局变量
6  .global main
7
8  _start:              //程序的开始
9      ldr sp, =0x84000000 //暂时设置到这个位置
10     b main
11     // ldr pc, =main    //跳转到main函数执行
12 loop:
13     b loop
14

```

- ```

1 # 编译程序
2 siguyuan@debian:~/EBF_6ULL/test/led_test$ make
3 arm-none-linux-gnueabi-gcc -c -Wall -O2 -o led.o led.c
4 arm-none-linux-gnueabi-gcc -c -Wall -O2 -o start.o start.S
5 arm-none-linux-gnueabi-ld -Tled.lds -o led.elf led.o start.o
6 arm-none-linux-gnueabi-objcopy -O binary -S -g led.elf
 led.bin
7 sudo cp led.bin /opt/out_dir/EBF_6ULL/
8 siguyuan@debian:~/EBF_6ULL/test/led_test$ ls
/opt/out_dir/EBF_6ULL/
9 led.bin u-boot-dtb.imx
10

```

- 传输文件

- 板子上，执行获取文件数据的命令

- ```

1  => tftp 0x80800000 led.bin
2  Using ethernet@20b4000 device
3  TFTP from server 192.168.50.51; our IP address is
   192.168.50.50
4  Filename 'led.bin'.
5  Load address: 0x80800000
6  Loading: T #
7           0 Bytes/s
8  done

```

```

 9      Bytes transferred = 196 (c4 hex)
10      => md.b 0x80800000
11      80800000: 21 d3 a0 e3 09 00 00 ea fe ff ff ea 82 b0 00 23
                !.....#
12      80800010: 01 93 01 93 01 9b 98 42 06 d9 00 bf 01 9b 01 33
                .....B.....3
13      80800020: 01 93 01 9b 83 42 f8 d3 02 b0 70 47 00 00 00 00
                .....B....pG....
14      80800030: 04 f0 1f e5 39 00 80 80 44 f2 6c 05 6c 22 c0 f2
                ....9...D.l.l"..
15
16      => go 0x80800000
17      ## Starting application at 0x80800000 ...
18

```

- `tftp` 内存地址 服务器里的文件名。SDRAM的地址: 0x80800000
- `md.b` 内存地址: 按字节显示内存信息。类似 hexdump
- `go` 内存地址: 跳到内存地址执行指令 (无法跳回来, 即无法使用u-boot), 只能关机重启

■

3.3 Linux内核移植

内核源码获取方式:

- 1、芯片厂家提供的定制版本 (推荐)。芯片厂家在自家的公版上配置好, 只需要在基础上修改配置, 适配我们自己的硬件即可。本次由野火提供的linux_4.19.35_imx6ul内核
- 2、Linux主线版本。下载地址: www.kernel.org。(许多硬件厂商、云计算厂商和 Linux 发行版都会根据自身需求进行深度的“定制化 (Out-of-tree / Custom)”, 导致其源码无法在 内核官方上直接找到)

Linux内核的源码目录结构

先解压内核代码 `tar -zxvf linux_4.19.35_imx6ull.tar.gz -C ../kernel/`

1	Linux 内核源码根目录	
2	— arch/	--> 【体系结构相关】按 CPU 架构分类 (x86, arm, arm64)
3	— arm/	
4	— boot/	--> 镜像生成 (zImage)
5	— kernel/	--> 架构相关的中断、调度
6	— mm/	--> 架构相关的页表映射
7	— x86/	
8	— drivers/	--> 【外设驱动】硬件网卡(drivers/net)、USB、显卡等
9	— net/	--> 【网络协议】TCP/IP、Socket 抽象层
10	— kernel/	--> 【内核核心】进程调度、定时器、信号处理
11	— mm/	--> 【内存管理】通用虚拟内存、页分配器
12	— fs/	--> 【文件系统】EXT4、sysfs、proc 等
13	— init/	--> 【启动入口】start_kernel()
14	— include/	--> 【公共头文件】
15	— scripts/	--> 【宿主机工具】内核编译/配置辅助脚本


```

1  siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ ls
2  arch  COPYING  Documentation  fs      ipc      kernel  MAINTAINERS  mm
   samples  sound  virt
3  block  CREDITS  drivers      include  kbuild  lib      make_deb.sh  net
   scripts  tools
4  certs  crypto  firmware      init      Kconfig  LICENSES  Makefile      README
   security  usr
5
6  # 体系架构相关目录，分公司
7  siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch$ ls
8  alpha  arm      c6x      hexagon  Kconfig  microblaze  nds32  openrisc  powerpc  s390
   sparc  unicore32  xtensa
9  arc     arm64  h8300  ia64      m68k     mips      nios2  parisc    riscv    sh      um
   x86
10
11 # armCPU架构下的相关代码
12 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm$ ls
13 boot      mach-alpine  mach-ebsa110  mach-ixp4xx  mach-nomadik  mach-
   rpc      mach-u300    oprofile
14 common    mach-artpec  mach-efm32    mach-keystone  mach-npcm     mach-
   s3c24xx  mach-uniphier  plat-iop
15 configs   mach-asm9260  mach-ep93xx  mach-ks8695  mach-nspire   mach-
   s3c64xx  mach-ux500    plat-omap
16 crypto    mach-aspeed  mach-exynos   mach-lpc18xx  mach-omap1    mach-
   s5pv210  mach-versatile  plat-orion
17 firmware  mach-at91    mach-footbridge  mach-lpc32xx  mach-omap2    mach-
   sa1100  mach-vexpress  plat-pxa
18 include    mach-axxia   mach-gemini   mach-mediatek  mach-orion5x  mach-
   shmobile  mach-vt8500  plat-samsung
19 Kconfig    mach-bcm     mach-highbank  mach-meson    mach-oxnas    mach-
   socfpga  mach-w90x900  plat-versatile
20 Kconfig.debug  mach-berlin  mach-hisi     mach-mmp      mach-picoxcell  mach-
   spear    mach-zx      probes
21 Kconfig-nommu  mach-clps711x  mach-imx     mach-moxart    mach-prima2    mach-
   sti      mach-zynq    tools
22 kernel     mach-cns3xxx  mach-integrator  mach-mv78xx0  mach-pxa      mach-
   stm32    Makefile      vdso
23 kvm        mach-davinci  mach-iop13xx  mach-mvebu     mach-qcom      mach-
   sunxi    mm            vfp
24 lib        mach-digicolor  mach-iop32x  mach-mxs      mach-realview  mach-
   tango    net           xen
25 mach-actions  mach-dove     mach-iop33x  mach-netx     mach-rockchip  mach-
   tegra    nwfpe
26

```

- 体系结构相关

- `arch/`: 包含所有支持的 CPU 架构代码 (如 `x86`、`arm`、`arm64`、`riscv` 等)。如果不进行配置, 默认使用当前宿主机的CPU架构
 - `arch/架构/net`: 存放的是硬件的网卡驱动
 - `arch/架构/kernel`: 进程调度/中断处理
 - `arch/架构/mm`: 内存管理
 - `arch/架构/boot`: 引导构建程序
- 体系结构无关
 - 除开 `arch` 的其他目录
 - `drivers/`: **所有外设驱动的主阵地** (如 `drivers/net/` 硬件网卡驱动、`drivers/char/` 字符设备、`drivers/usb/` 等)。
 - `net/`: **网络协议栈实现** (如 TCP/IP、Bluetooth、Wi-Fi 协议等, 与具体硬件网卡无关。
 - `kernel/`: 内核核心逻辑 (进程调度、信号处理、定时器等)。
 - `mm/`: 通用内存管理代码 (页表管理、SLAB 分配器、虚拟内存等)。
 - `fs/`: 文件系统实现 (EXT4、FAT、NFS、sysfs、proc 等)。
 - `include/`: 公共头文件目录 (其中 `include/asm-*` 会通过符号链接指向当前 `arch/` 下对应的头文件)。
 - `init/`: 内核初始化代码 (著名的 `main.c` 和 `start_kernel()` 就存放在这里)。
 - `ipc/`: 进程间通信机制 (消息队列、共享内存、信号量等)。
 - `block/`: 块设备通用层及 IO 调度算法。
 - `sound/`: 音频子系统架构 (ALSA 等)。
 - `crypto/`: 加密/解密算法库 (AES、SHA 等)。
 - `security/`: 安全模块 (SELinux、AppArmor 等)。
- 内核中通过设置Makefile中的obj-来决定文件是否编译。
 - `obj-y`: 用于编译内核镜像;
 - `obj-m`: 用于编译可动态加载的内核模块 (不会打包进内核主镜像, 需要时才通过 `insmod` / `modprobe` 动态加载, 或用 `rmmod` 卸载);
 - `obj-` 不编译

Makefile中 `obj- $\{$ CONFIG_NET $\}$` 中的宏, 如何赋值?

有一个参考配置 (`arch/cpu架构/configs/`), 利用 `make menuconfig` 进行图形界面的配置和裁剪。

参考配置来源: 1. 芯片公司提供的公版参考配置; 2. 厂家提供的一个扩展配置 野火提供了 `npi_v7_defconfig`

```

1 CONFIG_CRYPTO_DEV_FSL_CAAM_SECVIO=y
2 CONFIG_CRYPTO_DEV_MXS_DCP=y
3 CONFIG_CRC_T10DIF=y
4 CONFIG_CRC7=m
5 CONFIG_FONTS=y
6 CONFIG_FONT_8x8=y

```

Makefile会读取根据上面参考配置生成的 `.config` 文件做为配置单，规划Makefile的生成目标情况

3.3.1 Linux移植步骤

3.3.1.1 编译前的准备工作

1. 检查内核解压文件顶层的Makefile，需要配置哪些宏（可以使用vim，按:，输入 / + 关键字arch + Enter，进行搜索，按n键跳转到下一个结果）

```
○ 1  # CROSS_COMPILE specify the prefix used for all executables used
2  # during compilation. Only gcc and related bin-utils executables
3  # are prefixed with $(CROSS_COMPILE).
4  # CROSS_COMPILE can be set on the command line
5  # make CROSS_COMPILE=ia64-linux-
6  # Alternatively CROSS_COMPILE can be set in the environment.
7  # Default value for CROSS_COMPILE is not to prefix executables
8  # Note: Some architectures assign CROSS_COMPILE in their arch/*/Makefile
9  ARCH      ?= $(SUBARCH)
10
11 # Architecture as present in compile.h
12 UTS_MACHINE := $(ARCH)
13 SRCARCH    := $(ARCH)
14
15 ...
16
17 # Make variables (CC, etc...)
18 AS        = $(CROSS_COMPILE)as
19 LD        = $(CROSS_COMPILE)ld
20 CC        = $(CROSS_COMPILE)gcc
21 CPP       = $(CC) -E
22 AR        = $(CROSS_COMPILE)ar
23 NM        = $(CROSS_COMPILE)nm
24 STRIP     = $(CROSS_COMPILE)strip
25 OBJCOPY   = $(CROSS_COMPILE)objcopy
26 OBJDUMP   = $(CROSS_COMPILE)objdump
27
28 ...
29
30 # The expansion should be delayed until arch/$(SRCARCH)/Makefile is
31 # included.
32 # Some architectures define CROSS_COMPILE in arch/$(SRCARCH)/Makefile.
33 # CC_VERSION_TEXT is referenced from Kconfig (so it needs export),
34 # and from include/config/auto.conf.cmd to detect the compiler upgrade.
35 CC_VERSION_TEXT = $(shell $(CC) --version | head -n 1)
36
37 ifeq ($(config-targets),1)
38 #
39 =====
40 =  # *config targets only - make sure prerequisites are updated, and
41   # descend
42   # in scripts/kconfig to make the *config target
```

```

39 # Read arch specific Makefile to set KBUILD_DEFCONFIG as needed.
40 # KBUILD_DEFCONFIG may point out an alternative default configuration
41 # used for 'make defconfig'
42 include arch/$(SRCARCH)/Makefile
43

```

- 根据上面的，需要配置 `ARCH` 引入体系架构（?= 如果为NULL，使用默认值SUBARCH【宿主机架构】进行配置。:= ,直接赋值，会管上面SUBARCH的展开）；需要配置 `CROSS_COMPILE`，确定交叉编译器

2. 编写脚本，配置相关参数，激活脚本（不做，会导致使用宿主机gcc编译而报错）

```

1 siguyuan@debian:~/EBF_6ULL/env$ cat arm_path.sh
2 #!/bin/bash
3
4 # 交叉编译工具链的环境变量配置
5 export PATH=$PATH:/opt/arm-linux-gnueabi/bin
6 # 告知交叉编译器的前缀
7 export CROSS_COMPILE=arm-none-linux-gnueabi-
8 # 配置内核需要的体系架构
9 export ARCH=arm
10
11 siguyuan@debian:~/EBF_6ULL/env$ source arm_path.sh

```

o

3.3.1.2 激活参考配置

- 根据厂家的参考配置进行配置，生成 `.config` 文件

```

1 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make npi_v7_defconfig
2 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ ls -a
3 .      block  COPYING  Documentation  fs      ipc      kernel  MAINTAINERS
4 mm     samples  sound    virt
5 ..     certs  CREDITS  drivers        include  kbuild  lib      make_deb.sh
6 net    scripts  tools
7 arch  .config  crypto  firmware      init     kconfig  LICENSES  Makefile
8 README security  usr

```

3.3.1.3 基于参考配置，进行修改和裁剪

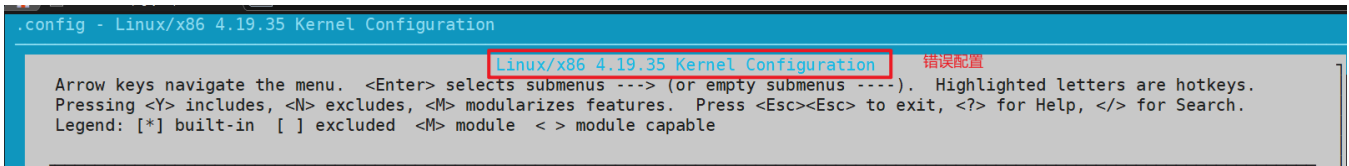
`make menuconfig`

一般工作中，第一个版本先不裁剪，生成一个可用的版本；在已经有可用版本的前提下进行裁剪。配置时，尽量赋值命令，写错不能删除修改，只能修改 `.config` 文件。

常见错误：

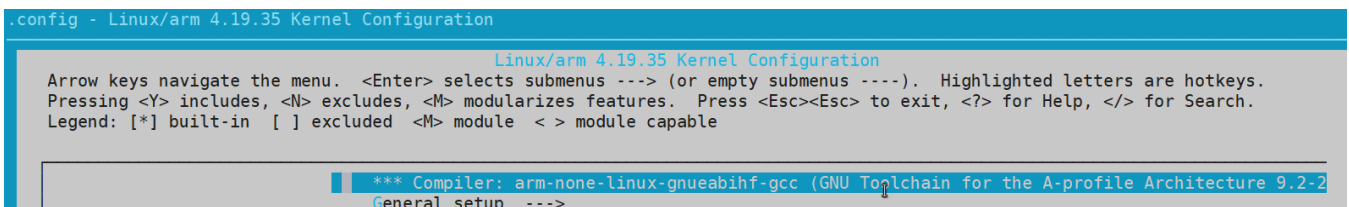
```
#
rocky@debian390:~/ebf/linux_4.19.35_imx6ull$ make menuconfig
*
* Unable to find the ncurses package.
* Install ncurses (ncurses-devel or libncurses-dev
* depending on your distribution).
*
make[1]: *** [scripts/kconfig/Makefile:228: scripts/kconfig/.mconf-cfg] 错误 1
make: *** [Makefile:539: menuconfig] 错误 2
rocky@debian390:~/ebf/linux_4.19.35_imx6ull$ sudo apt install libncurses
libncurses6          libncurses6_6.2.4 libncurses6-dev libncurses-dev libncursesw6
rocky@debian390:~/ebf/linux_4.19.35_imx6ull$ sudo apt install libncurses
```

安装对应的图形库



注意是不是和板子的cpu架构是否一致。比如上图是x86，我需要arm，就执行 `arm.sh` 脚本，重新进入这个界面。

正确结果：



修改内核名字：General setup → Local version - append to kernel release → 输入名字

3.3.1.4 编译内核

不能直接使用 `make`。使用 `make` 编译出来的是：内核主目标 = 内核核心镜像目标 + 内核模块目标 + 内核设备树目标。

使用 `make zImage`：只编译内核核心镜像目标。一般后面需要添加 `-j$(nproc)`，加快编译速度

`bear -- make zImage`：bear 检测make的过程，将make管理的目标 编译行为 汇总生成一个 `compile_commands.json` 文件，clangd的静态代码检测器识别。（方便再vscode中查看源码）

```
1 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make zImage
2 scripts/kconfig/conf -s --syncconfig kconfig
3 /bin/sh: 1: bc: not found
4 make[1]: *** [kbuild:42: include/generated/timeconst.h] 错误 127
5 make: *** [Makefile:1102: prepare0] 错误 2
6
7 # 找不bc工具，需要安装
8 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ sudo apt install bc
9
10 # 链接时出现multiple definition（多符号表）。原因：有些.c文件是程序生成，所以会出现多符号的情况。
11 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make zImage -j$(nproc)
12 /usr/bin/ld: scripts/dtc/dtc-parser.tab.o(.bss+0x10): multiple definition of
13 `__yyllloc'; scripts/dtc/dtc-lexer.lex.o(.bss+0x0): first defined here
collect2: error: ld returned 1 exit status
```

```

14 make[2]: *** [scripts/Makefile.host:99: scripts/dtc/dtc] 错误 1
15 make[1]: *** [scripts/Makefile.build:544: scripts/dtc] 错误 2
16 make[1]: *** 正在等待未完成的任务....
17 make: *** [Makefile:1067: scripts] 错误 2
18 make: *** 正在等待未完成的任务....
19
20 # 解决方式: 修改生成.c的脚本文件 (一劳永逸)
21 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ cd ./scripts/dtc/
22 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/scripts/dtc$ ls
23 checks.c dtc.h dtc-parser.tab.c dtx_diff flattree.o livetree.c
    srcpos.h util.c
24 checks.o dtc-lexer.l dtc-parser.tab.h fdt_dump.c fstree.c livetree.o
    srcpos.o util.h
25 data.c dtc-lexer.lex.c dtc-parser.tab.o fdtget.c fstree.o Makefile
    treesource.c util.o
26 data.o dtc-lexer.lex.o dtc-parser.y fdtput.c include-prefixes
Makefile.dtc treesource.o version_gen.h
27 dtc.c dtc.o dt_to_config flattree.c libfdt srcpos.c
    update-dtc-source.sh
28 # 修改其中一个脚本 (.l和.y都是脚本文件)
29 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/scripts/dtc$ nvim dtc-lexer.l
30
31 # 修改内容
32 extern YYLTYPE yylloc; # 在这个前面添加extern 告知其他地方声明了
33
34 # 出现没有openssl库
35 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make zImage -j$(nproc)
36 scripts/extract-cert.c:21:10: fatal error: openssl/bio.h: 没有那个文件或目录
37     21 | #include <openssl/bio.h>
38         |          ^~~~~~
39 compilation terminated.
40 make[1]: *** [scripts/Makefile.host:90: scripts/extract-cert] 错误 1
41 make[1]: *** 正在等待未完成的任务....
42 make: *** [Makefile:1067: scripts] 错误 2
43 make: *** 正在等待未完成的任务....
44
45 # 安装openssl库 注意: 直接写openssl是安装可执行文件
46 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ sudo apt install libssl-dev
47
48 # 错误原因: .config被污染, 再之前使用make menuconfig弄成x86之后, 没有清除.config就直接重新编译
49 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make zImage -j$(nproc)
50 ...
51 arm-none-linux-gnueabi-hf-ld: drivers/net/wireless/bcmdhd/dhd_gpio.o: in function
`dhd_wlan_set_carddetect':
52 dhd_gpio.c:(.text+0x44): undefined reference to `wifi_card_detect'
53 arm-none-linux-gnueabi-hf-ld: drivers/usb/chipidea/ci_hdrc_imx.o: in function
`imx_controller_suspend':
54 ci_hdrc_imx.c:(.text+0xb4): undefined reference to `release_bus_freq'
55 arm-none-linux-gnueabi-hf-ld: drivers/usb/chipidea/ci_hdrc_imx.o: in function
`imx_controller_resume':
56 ci_hdrc_imx.c:(.text+0x320): undefined reference to `request_bus_freq'
57 arm-none-linux-gnueabi-hf-ld: ci_hdrc_imx.c:(.text+0x3e4): undefined reference to
`release_bus_freq'

```

```

58 arm-none-linux-gnueabi-hf-ld: drivers/usb/chipidea/ci_hdrc_imx.o: in function
   `ci_hdrc_imx_remove':
59 ci_hdrc_imx.c:(.text+0x4b4): undefined reference to `release_bus_freq'
60 arm-none-linux-gnueabi-hf-ld: drivers/usb/chipidea/ci_hdrc_imx.o: in function
   `ci_hdrc_imx_probe':
61 ci_hdrc_imx.c:(.text+0xa78): undefined reference to `request_bus_freq'
62 arm-none-linux-gnueabi-hf-ld: ci_hdrc_imx.c:(.text+0xaa8): undefined reference to
   `release_bus_freq'
63 arm-none-linux-gnueabi-hf-ld: drivers/rpmsg/imx_rpmsg.o: in function
   `imx_rpmsg_notify':
64 imx_rpmsg.c:(.text+0xb0): undefined reference to `MU_SendMessage'
65 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0xe4): undefined reference to
   `MU_SendMessageTimeout'
66 arm-none-linux-gnueabi-hf-ld: drivers/rpmsg/imx_rpmsg.o: in function
   `imx_rpmsg_mu_init':
67 imx_rpmsg.c:(.text+0x544): undefined reference to `MU_Init'
68 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0x55c): undefined reference to
   `MU_EnableRxFullInt'
69 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0x570): undefined reference to
   `MU_EnableRxFullInt'
70 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0x580): undefined reference to
   `MU_SetFn'
71 arm-none-linux-gnueabi-hf-ld: drivers/rpmsg/imx_rpmsg.o: in function
   `imx_mu_rpmsg_isr':
72 imx_rpmsg.c:(.text+0xae0): undefined reference to `MU_ReadStatus'
73 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0xb2c): undefined reference to
   `MU_ReceiveMsg'
74 arm-none-linux-gnueabi-hf-ld: drivers/rpmsg/imx_rpmsg.o: in function
   `imx_rpmsg_restore':
75 imx_rpmsg.c:(.text+0xc1c): undefined reference to `MU_ReadStatus'
76 make: *** [Makefile:1032: vmlinux] 错误 1
77
78 # 清除.config 从第三步重新开始
79 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make distclean
80
81 # 重新生成.config
82 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make npi_v7_defconfig
83 # 重新配置
84 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make menuconfig
85 scripts/kconfig/mconf -s Kconfig
86 # 重新编译（一步到位）
87 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ bear -- make zImage -
j$(nproc)
88
89 # 缺少lzop工具，内核采取自解压的方式来实现加载到目标内存中 — 使用了lz压缩算法
90 drivers/usb/gadget/configfs.c:183:1: note: in expansion of macro
   'GI_DEVICE_DESC_SIMPLE_R_u16'
91     183 | GI_DEVICE_DESC_SIMPLE_R_u16(bcdDevice);
92         | ^~~~~~
93 /bin/sh: 1: lzop: not found
94 make[2]: *** [arch/arm/boot/compressed/Makefile:191:
   arch/arm/boot/compressed/piggy_data] 错误 1
95 make[1]: *** [arch/arm/boot/Makefile:64: arch/arm/boot/compressed/vmlinux] 错误 2

```

```
96 | make: *** [arch/arm/Makefile:336: zImage] 错误 2
97 |
98 | siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ sudo apt install lzop
99 |
```

建议在flash中存储压缩的内核镜像，遇到了谁来解压内核镜像 加载到目标机的内存里。

内核采取了自解压的方式 来实现加载到目标机内存中。

生成的文件在：arch/CPU架构/boot 下

Image：未被压缩的内核镜像文件，可以直接启动，但是太大，存储在flash上占用空间比较多

zImage：带自解压程序的携带压缩后的内核镜像文件，启动时，会自动解压内核镜像到内存上，再把PC指向解压后的内存位置

bzImage：内核镜像较大的文件，通常给x86体系用，而zImage通常给arm使用

uImage：zImage的基础上，增加了一段uboot能够识别的头格式，便于uboot里的boot命令启动。uImage = uboot能理解的头格式 + zImage

内核编译流程：

scripts/，宿主机的工具，make使用HOSTCC进行编译

体系结构无关,使用CC编译

每个目录 被obj-y包含的源文件 生成对应.o文件，以目录为单位，用AR命令将该目录下的所有.o打包生成built-in.a

```
1 | ...
2 | LD      vmlinux                                # ELF格式的文件
3 | OBJCOPY arch/arm/boot/Image
4 | Kernel: arch/arm/boot/Image is ready          # 镜像文件出来了
```

3.3.1.5 编译内核设备树

设备树的本质就是：用文本文件把硬件信息抽出来，让一套 Linux 内核镜像（zImage）可以通过搭配不同的设备树文件（.dtb），直接运行在各种开发板上。

make imx6ull-mmc-npi.dtb 编译im6ull板子的设备树

建议写脚本，后期需要频繁修改（记得给脚本添加可执行权限）


```

1  #!/bin/bash
2
3  # 将编译的内核镜像压缩文件，拷贝到共享目录中
4  sudo cp /home/siguyuan/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/boot/zImage
   /opt/out_dir/EBF_6ULL/
5  # 编辑设备树
6  make imx6ull-mmc-npi.dtb
7  # 将设备树的镜像文件拷贝到共享目录
8  sudo cp /home/siguyuan/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/boot/dts/imx6ull-
   mmc-npi.dtb /opt/out_dir/EBF_6ULL/
9

```

```

1  siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ ../../env/DT_update.sh

```

3.3.1.6 下载相关文件

启动Uboot，下载内核镜像文件和设备树镜像文件，配置相关的启动行为

内核解压目录/System.map：内核符号与内存地址的“映射表/对照表”文件（Kernel Symbol Map File）。知道下载内核镜像应该下到哪里。

板子上下载文件：

```
tftp 0x80800000 zImage
```

```
tftp 0x83000000 imx6ull-mmc-npi.dtb
```

```

1  # 拼写错误
2  => tftp 0x80800000 zImage
3  Using ethernet@20b4000 device
4  TFTP from server 192.168.50.51; our IP address is 192.168.50.50
5  Filename 'zImage'.
6
7  TFTP error: trying to overwrite reserved memory...
8
9  # 服务端（虚拟机）没有配置IP
10 => tftp 0x80800000 zImage
11 Using ethernet@20b4000 device
12 TFTP from server 192.168.50.51; our IP address is 192.168.50.50
13 Filename 'zImage'.
14 Load address: 0x80800000
15 Loading: *
16 ARP Retry count exceeded; starting again
17
18 # 成功
19 => tftp 0x80800000 zImage
20 Using ethernet@20b4000 device
21 TFTP from server 192.168.50.51; our IP address is 192.168.50.50
22 Filename 'zImage'.

```

```

23 Load address: 0x80800000
24 Loading: T #####
25 #####
26 #####
27 #####
28 #####
29 #####
30 #####
31 #####
32 #####
33 #####
34 #####
35 1.1 MiB/s
36 done
37 Bytes transferred = 10122704 (9a75d0 hex)
38

```

3.3.1.7 运行内核代码

不能直接使用go命令启动，Linux启动需要参数

```

1 => go 0x80800000 # 直接卡死
2 ## Starting application at 0x80800000 ...
3

```

使用boot命令启动

```

1 bootz 0x80800000 - 0x83000000 # 只使用了2个参数
2
3 => bootz 0x80800000 - 0x83000000
4 Kernel image @ 0x80800000 [ 0x000000 - 0x9a75d0 ]
5 ## Flattened Device Tree blob at 83000000
6 Booting using the fdt blob at 0x83000000
7 Using Device Tree in place at 83000000, end 8300c680
8
9 Starting kernel ...
10
11 [ 0.000000] Booting Linux on physical CPU 0x0
12 [ 0.000000] Linux version 4.19.35 (siguyuan@debian) (gcc version 9.2.1 20191025
(GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10))) #1 SMP PREEMPT
Thu Aug 13 17:16:18 CST 2026 # 出现版本和宿主机名字
13
14 ...
15 [ 2.602061] ALSA device list:
16 [ 2.605252] No soundcards found.
17 [ 2.609265] VFS: Cannot open root device "(null)" or unknown-block(0,0): error -6

```

```

18 [ 2.617056] Please append a correct "root=" boot option; here are the available
19 partitions:
20 [ 2.753302] kernel panic - not syncing: VFS: Unable to mount root fs on unknown-
21 block(0,0)
22 [ 2.761589] ---[ end kernel panic - not syncing: VFS: Unable to mount root fs on
23 unknown-block(0,0) ]---
24 [ 64.492564] cfg80211: failed to load regulatory.db
25
26 # 出现错误（内核惊恐 -- kernel panic），且错误提示VFS（虚拟文件系统）找不到根设备
27 # 解决方式：配置文件系统（boot启动时传入参数）
28 # 一般先使用厂家提供的文件系统，进行测试
29 tftp 0x80800000 zImage
30 tftp 0x83000000 imx6ull-mmc-npi.dtb
31 tftp 0x88000000 initrd.img # 下载文件系统
32 bootz 0x80800000 0x88000000:0x4ecebc 0x83000000 # 使用了第三个参数
33
34 # 出现命令提示即可
35 [ 3.590754] Freeing unused kernel memory: 1024K # 释放初始化时使用的内存
36
37 (initramfs) ls
38 bin      dev      init      proc      run      scripts  tmp      var
39 conf     etc      lib      root      sbin     sys      usr
40
41 # (initramfs) 可以通过PS1进行修改
42 export PS1='(my_board_env) \w # '

```

可能需要设置tftp的ip, `setenv serverip 192.168.50.51`

使用厂家提供文件系统时，会遇到的错误（并发文件本身的错）：

```

1 # 错误现象：系统文件正确，文件放入正确的内存地址，但是启动内核时出现惊恐
2 #
3
4 [ 0.449183] TCP bind hash table entries: 4096 (order: 3, 32768 bytes)
5 [ 0.449346] TCP: Hash tables configured (established 4096 bind 4096)
6 [ 0.449597] UDP hash table entries: 256 (order: 1, 8192 bytes)
7 [ 0.449665] UDP-Lite hash table entries: 256 (order: 1, 8192 bytes)
8 [ 0.450082] NET: Registered protocol family 1
9 [ 0.481755] RPC: Registered named UNIX socket transport module.
10 [ 0.481793] RPC: Registered udp transport module.
11 [ 0.481816] RPC: Registered tcp transport module.
12 [ 0.481835] RPC: Registered tcp NFSv4.1 backchannel transport module.
13 [ 0.493916] Trying to unpack rootfs image as initramfs...
14 [ 0.663966] rootfs image is not initramfs (read error); looks like an initrd
15 [ 0.674645] Freeing initrd memory: 1140K
16 [ 0.678971] Bus freq driver module loaded
17 [ 0.681635] Initialise system trusted keyrings
18
19
20 1.648333] fec 20b4000.ethernet: Linked as a consumer to regulator.0

```

```
21 [ 1.656222] pps pps0: new PPS source ptp0
22 [ 1.661753] libphy: fec_enet_mii_bus: probed
23 [ 1.680926] fec 20b4000.ethernet eth0: registered PHC device 0
24 [ 1.689038] fec 2188000.ethernet: 2188000.ethernet supply phy not found, using
dummy regulator
25 [ 1.698232] fec 2188000.ethernet: Linked as a consumer to regulator.0
26 [ 1.706222] pps pps1: new PPS source ptp1
27 [ 1.710868] fec 2188000.ethernet (unnamed net_device) (uninitialized): Invalid
MAC address: 00:00:00:00:00:00
28 [ 1.721101] fec 2188000.ethernet (unnamed net_device) (uninitialized): Using
random MAC address: 2a:b6:b2:49:da:0f
29 [ 1.733788] fec 2188000.ethernet eth1: registered PHC device 1
30 [ 1.741296] PPP generic driver version 2.4.2
31
32
33 2.802430] ALSA device list:
34 [ 2.805616] No soundcards found.
35 [ 2.809560] RAMDISK: gzip image found at block 0
36 [ 2.907039] RAMDISK: EOF while reading compressed data
37 [ 2.907059] uncompression error
38 [ 2.939332] List of all partitions:
39 [ 2.943124] 0100 65536 ram0
40 [ 2.943133] (driver?)
41 [ 2.949265] 0101 65536 ram1
42 [ 2.949271] (driver?)
43 [ 2.955701] 0102 65536 ram2
44 [ 2.955707] (driver?)
45 [ 2.961833] 0103 65536 ram3
46 [ 2.961839] (driver?)
47 [ 2.968067] 0104 65536 ram4
48 [ 2.968074] (driver?)
49 [ 2.974230] 0105 65536 ram5
50 [ 2.974236] (driver?)
51 [ 2.980359] 0106 65536 ram6
52 [ 2.980364] (driver?)
53 [ 2.986520] 0107 65536 ram7
54 [ 2.986528] (driver?)
55 [ 2.992718] 0108 65536 ram8
56 [ 2.992724] (driver?)
57 [ 2.998845] 0109 65536 ram9
58 [ 2.998851] (driver?)
59 [ 3.005005] 010a 65536 ram10
60 [ 3.005012] (driver?)
61 [ 3.011220] 010b 65536 ram11
62 [ 3.011225] (driver?)
63 [ 3.017496] 010c 65536 ram12
64 [ 3.017502] (driver?)
65 [ 3.023743] 010d 65536 ram13
66 [ 3.023748] (driver?)
67 [ 3.029954] 010e 65536 ram14
68 [ 3.029960] (driver?)
69 [ 3.036194] 010f 65536 ram15
70 [ 3.036200] (driver?)
```

```

71 [ 3.042413] b308 31178752 mmcblk0
72 [ 3.042420] driver: mmcblk
73 [ 3.049302] b300 7634944 mmcblk1
74 [ 3.049309] driver: mmcblk
75 [ 3.056156] No filesystem could mount root, tried:
76 [ 3.056162] ext3
77 [ 3.061046] ext4
78 [ 3.063004] ext2
79 [ 3.064934] vfat
80 [ 3.066867] fuseblk
81 [ 3.068797]
82 [ 3.072486] kernel panic - not syncing: VFS: Unable to mount root fs on unknown-
block(1,0)
83 [ 3.080771] ---[ end kernel panic - not syncing: VFS: Unable to mount root fs on
unknown-block(1,0) ]---
84 [ 64.492725] cfg80211: failed to load regulatory.db
85
86
87 # 错误原因: 不是网络、ALSA 或 regulatory.db, 而是 Linux 启动时使用的 RAMDISK/initrd 镜像损坏
或地址/长度传递不正确, 最终导致根文件系统无法挂载
88
89 # 整体检查流程
90 # 第一步: 检查文件
91 ls -lh /opt/out/
92 file /opt/out/zImage
93 file /opt/out/imx6ull-mmc-npi.dtb
94 file /opt/out/initrd.img
95 # 观察上面的现象
96 zImage → ARM Linux kernel 大小与bootz命令中的大小是否一致
97 .dtb → Device Tree Blob 设备树是否正常
98 initrd.img → gzip 压缩文件 是否是gzip
99
100 # 第二步: 检查 initrd 是否损坏
101 gzip -t /opt/out/initrd.img
102 zcat /opt/out/initrd.img | cpio -t | head
103 # 无报错 → initrd 本身基本正常。
104
105 # 第三步: 记录 initrd 实际大小
106 ls -l /opt/out/initrd.img
107 # 后面 bootz 的 initrd 长度必须与实际文件大小一致 (注意换算成16进制比较)
108
109 # 第四步: 检查TFTP
110 tftp 0x80800000 zImage
111 tftp 0x83000000 imx6ull-mmc-npi.dtb
112 tftp 0x88000000 initrd.img
113 # 确认 TFTP 实际传输的文件大小。与宿主机中文件一致 → TFTP 传输正常。
114
115 # 第五步: 检查 initrd 是否完整进入 RAM
116 md.b 0x88000000 0x40 # 输出 1f 8b 08 表示gzip文件头, 正确
117 md.b 0x884ecea0 0x1c # 其中0x88000000 + 0x4ECEBC = 0x884ECEBC
118 # 确认 RAM 中的 initrd 与 PC 文件一致
119
120 # 第六步: 检查 bootargs环境变量【比较重要】

```

```

121 printenv bootargs
122 # 实际 initrd 大小 与 bootargs 中 initrd 大小。错误的大小会导致: RAMDISK: EOF while
    reading compressed data
123 # 解决方式: setenv bootargs 'console=ttyMxc0,115200' 115200为initrd的实际大小
124

```

3.3.1.8 配置VScode查看内核代码

vscode的管理方式:

- 以目录为项目，实现多文件管理
- 以多个项目（目录）为工作空间进行管理

配置步骤

1. 先使用VScode打开解压的内核目录，让clangd自动配置（读取之前bear创建的json文件）
2. 点击设置 —— 工作区 —— 打开设置 —— 关闭窗口（看到生成 `.vscode/settings.json` 即为成功）
3. 配置settings.json

```

1  {
2      // 屏蔽展示文件
3      "files.exclude": {
4          "**/.git": true,
5          "**/.svn": true,
6          "**/.hg": true,
7          "**/.DS_Store": true,
8          "**/Thumbs.db": true,
9
10         // --- 内核编译中间过程文件（极大清爽侧边栏） ---
11         // "**/*.o": true,           // 目标文件,暂时保留,看哪些文件编译了
12
13         // 的
14         "**/*.o.cmd": true,         // 编译命令记录文件
15         "**/*.cmd": true,          // kbuild 命令文件
16         "**/*.a": true,            // built-in.a 归档文件
17         "**/*.s": true,            // 汇编中间文件
18         "**/*.ko": true,           // 编译出的模块文件
19         "**/*.mod.c": true,        // 模块描述文件
20         "**/*.symvers": true,      // 模块符号文件
21         "**/*.order": true,        // 编译顺序文件
22
23         // --- 架构过滤（精炼：屏蔽除了 32位 arm 以外的所有架构） ---
24         // "**/arch/[!a]*": true,   // 简单写法,屏蔽除了a开头的目录
25         "arch/alpha": true,
26         "arch/arm64": true,        // 屏蔽 64位 ARM (arm64/AArch64)
27         "arch/avr32": true,

```

```

27     "arch/blackfin": true,
28     "arch/c6x": true,
29     "arch/cris": true,
30     "arch/frv": true,
31     "arch/h8300": true,
32     "arch/hexagon": true,
33     "arch/ia64": true,
34     "arch/m32r": true,
35     "arch/m68k": true,
36     "arch/metag": true,
37     "arch/microblaze": true,
38     "arch/mips": true,
39     "arch/nds32": true,
40     "arch/nios2": true,
41     "arch/openrisc": true,
42     "arch/parisc": true,
43     "arch/powerpc": true,
44     "arch/riscv": true,
45     "arch/s390": true,
46     "arch/score": true,
47     "arch/sh": true,
48     "arch/sparc": true,
49     "arch/tile": true,
50     "arch/unicore32": true,
51     "arch/x86": true,           // 屏蔽 x86/x86_64 架构
52     "arch/xtensa": true,
53     "**/arch/arm/mach-[!i]*": true,
54
55     // --- 驱动目录精炼（屏蔽 x86 专用驱动，按需开启） ---
56     "drivers/acpi": true,       // ACPI 高级电源管理（多用于 x86 PC）
57     "drivers/xen": true,        // xen 虚拟化驱动
58
59     // --- 内核顶层大链接及临时文件（图片里的这些） ---
60     "System.map": true,         // 符号映射表
61     "vmlinux": true,            // 原始 ELF 镜像
62     "vmlinux.o": true,          // 顶层大链接文件
63     ".tmp_*": true,             // 彻底匹配 .tmp_kallsyms*,
64     ".tmp_vmlinux*", ".tmp_System.map"
65     "missing-syscalls.d": true, // 系统调用检查临时文件
66     ".version": true,           // 版本记录文件
67     ".config*": true            // 配置文件及旧配置备份（注意：如果你想在
68     },                          侧边栏看 .config，这行可以不加）
69
70     // 屏蔽搜索时的范围
71     "search.exclude": {
72         "**/node_modules": true,
73         "**/bower_components": true,
74         "**/*.code-search": true,
75
76         // --- 屏蔽内核编译最终产物与构建目录 ---
77         "vmlinux*": true,        // 原始 ELF 镜像文件
78         "System.map": true,      // 符号映射表文件

```

```

78     "**/include/generated": true, // 自动生成的头文件目录
79     "**/scripts": true,           // 宿主机工具脚本目录
80     "**/tools": true              // 辅助工具集
81
82
83 },
84
85 // 明确告诉 C/C++ 插件不要去扫描 64位 ARM 和其他 CPU 的代码
86 "C_cpp.files.exclude": {
87     "arch/arm64": true,
88     "arch/x86": true,
89     "arch/mips": true,
90     "arch/powerpc": true,
91     "arch/riscv": true
92 }
93 }

```

o

4. 关闭文件夹。在虚拟机中创建一个工作空间目录（用于关联内核目录、自己的驱动目录、Uboot目录）和一个自己驱动的目录。点击文件 —— 将工作区另存为，选中刚刚创建的工作空间目录 —— 添加
 - o `/home/siguyuan/EBF_6ULL/vscode/linux_4.19.35_imx6ull.code-workspace` 选中时不用修改后面的配置脚本，改前面的路径即可
5. 打开Uboot目录进行配置（与上面内核相同，只是setting.json有些不同，没有添加到工作空间的步骤，直接关闭文件夹）

o

```

1  {
2      // ===== 1. 侧边栏屏蔽（不展示的文件与文件夹） =====
3      "files.exclude": {
4          // --- 系统与版本控制 ---
5          "**/.git": true,
6          "**/.svn": true,
7          "**/.DS_Store": true,
8
9          // --- U-Boot 编译产生的中间文件 ---
10         "**/*.o": true,           // 目标文件
11         "**/*.o.cmd": true,       // kbuild 编译指令文件
12         "**/*.cmd": true,        // 编译过程脚本
13         "**/*.a": true,          // built-in.a 归档文件
14         "**/*.s": true,          // 汇编中间文件
15         "**/*.su": true,         // 栈使用情况分析文件
16
17         // --- 架构过滤（只保留 ARM，屏蔽其他 CPU 架构） ---
18         "arch/arm": true,
19         "arch/arm64": true,      // 屏蔽 64 位 ARM
20         "arch/m68k": true,
21         "arch/microblaze": true,
22         "arch/mips": true,
23         "arch/nds32": true,
24         "arch/nios2": true,
25         "arch/powerpc": true,
26         "arch/riscv": true,

```



```

27     "arch/sandbox": true,           // 沙盒测试架构
28     "arch/sh": true,
29     "arch/x86": true,              // 屏蔽 x86 架构
30     "arch/xtensa": true,
31
32     // --- U-Boot 根目录编译生成的镜像与日志文件 ---
33     "System.map": true,            // 地址映射表
34     "u-boot": true,                // ELF 原始文件
35     "u-boot*": true,               // 通配匹配 u-boot-dtb.bin, u-boot.imx
    等
36     "u-boot.*": true,              // 补全匹配带后缀的: u-boot.bin, u-
    boot.cfg, u-boot.map, u-boot.lds 等
37 },
38
39     // ===== 2. 全局搜索屏蔽 (Ctrl+Shift+F 流畅搜索)
    =====
40     "search.exclude": {
41         // --- 显式过滤编译中间产物 (防止误触导致的搜索污染) ---
42         "**/*.o": true,
43         "**/*.o.cmd": true,
44         "**/*.cmd": true,
45         "**/*.a": true,
46         "**/*.su": true,
47
48         // --- 屏蔽 U-Boot 编译生成的最终镜像与工具目录 ---
49         "u-boot*": true,            // u-boot, u-boot.bin, u-boot.imx 等
    产物
50         "System.map": true,         // 地址符号映射表
51         "**/include/generated": true, // 自动生成的头文件
52         "**/include/autoconf.mk*": true, // 编译自动生成的配置
53         "**/scripts": true,         // 编译脚本目录
54         "**/tools": true             // 宿主机构建工具
55     },
56
57     // ===== 3. C/C++ 语法索引优化 (提升跳转速度)
    =====
58     "C_cpp.files.exclude": {
59         "arch/arm64": true,
60         "arch/x86": true,
61         "arch/mips": true,
62         "arch/riscv": true,
63         "arch/powerpc": true
64     }
65 }

```

o

6. 自己创建的工作区目录有以下内容

```

o 1  siguyuan@debian:~/EBF_6ULL/vscode$ ls
   2  linux_4.19.35_imx6ull.code-workspace  uboot-ebf_v2020_10_imx.code-workspace
   3

```

- 打开工作区目录，可以看到内核源码即成功
- 右键将文件夹添加到工作区，选中uboot和自己的驱动目录

3.4 根文件系统

文件系统本质是用于管理持久化存储设备、实现“逻辑文件/路径”到“物理存储块/内存页面”映射关系的高效**数据结构与软件抽象层**。内核找到根设备，在根设备里部署一个文件系统，利用文件系统间接访问存储资源

根设备：让内核启动后，找到一个设备让其工作，这个设备就是根设备，需要驱动程序的支持。

- 类型：
 - Flash类型（本地持久化挂载）：闪存、SD卡、磁盘
 - RAM 类（内存根文件系统）：内存。
 - 启动速度快，但掉电丢失。有一种处理方式：**修改不写回**：因为整个根目录都在 RAM 里，所有读写都在内存（Page Cache）中进行，**掉电后所有的修改全丢失**，完全不会影响外部介质。（这样做用户的修改不会影响核心程序）
 - 网络类：网络文件系统（使用宿主机的磁盘，通过网络连接）—— NFS协议。在调式阶段完全不需要重新烧录。
 - NFS：network FS 网络文件系统服务。使用的是C/S架构（客户端 —— 服务端）。

如何制作根文件系统，分为两个步骤：1、制作文件系统内容；2、找到一个管理员（文件系统格式）格式化，把这些做好的内容，有序的放入一个镜像之中。

3.4.1 制作基本的根文件系统内容

文件系统准备工作：准备好NFS环境

- 宿主机（NFS服务器）：
 - 安装：`sudo apt install nfs-kernel-server`
 - 创建文件系统根目录（共享目录）：`mkdir /home/siguyuan/EBF_6ULL/rootfs`
 - 配置NFS：
 - 配置文件：`/etc/exports`
 - 配置内容（配置共享目录）

```
1  # /etc/exports: the access control list for filesystems which may be
   exported
2  #      to NFS clients.  See exports(5).
3  #
4  # Example for NFSv2 and NFSv3:
5  # /srv/homes      hostname1(rw,sync,no_subtree_check)
   hostname2(ro,sync,no_subtree_check)
6  #
7  # Example for NFSv4:
8  # /srv/nfs4
   gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
9  # /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
10
11 # 共享目录路径    允许任何IP的客户端访问(可读可写,同步写入,禁用子树检查,不压制
   客户端权限)
12 /home/siguyuan/EBF_6ULL/rootfs  *
   (rw,sync,no_subtree_check,no_root_squash)
13
```

- `/home/siguyuan/EBF_6ULL/rootfs` (共享目录路径)
- `*` (允许访问的客户端 IP 匹配规则)
 - **允许任何 IP 地址的客户端** (即你的 i.MX6ULL 开发板) 连接和挂载这个目录
 - 如果限定特定 IP,如: `/home/siguyuan/EBF_6ULL/rootfs 192.168.1.0/24(rw,sync,no_subtree_check,no_root_squash)` 限制 192.168.1.x 这个网段

参数	含义与作用
<code>rw</code>	Read-Write (可读可写) 。开发板挂载后不仅可以读取文件,还可以在上面创建、修改和删除文件。如果设为 <code>ro</code> (Read-Only), 开发板启动时会因为无法创建临时文件而报错。
<code>sync</code>	Synchronous (同步写入) 。开发板写入数据时, 宿主机必须将数据真正落盘(写回内存/磁盘)后才返回成功响应。 保证数据实时一致 , 防止开发板刚写入的文件在宿主机上还没更新。
<code>no_subtree_check</code>	禁用子树检查 。强制 NFS 检查客户端访问的文件是否属于整个文件系统树的某个子目录。 禁用它可以提高性能和稳定性 , 在共享整个根文件系统目录 (<code>rootfs</code>) 时几乎是必选参数。
<code>no_root_squash</code>	不压制 (squash) 客户端的 root 用户权限 。NFS 默认启用 <code>root_squash</code> , 也就是说客户端(开发板)以 root 身份访问时, 会被映射成匿名用户 (nobody), 这样一来开发板挂载 <code>rootfs</code> 后, 很多需要 root 权限的操作(比如创建设备节点、写 <code>/etc</code> 下的文件、修改属主等)可能会失败, 报 “Permission denied”。

■  **Caution**

路径和后面的属性，需要一个tab的空格，属性和()不需要空格，()里面是英文逗号分隔

◦ 启动NFS服务：

- 重启NFS服务 `sudo service nfs-kernel-server restart`。
- 启动NFS服务： `sudo service nfs-kernel-server start`。
- 查看NFS服务启动状态： `sudo systemctl status nfs-kernel-server`。有 SUCCESS 即为成功。
 - `sudo exportfs`，出现路径即为成功。

◦ 在该目录下创建一个临时的测试文件，并使用交叉编译器编译。

• 目标机（板子，客户端）验证：

◦ 先使用芯片厂商提供的文件系统，进行验证文件是否共享出去。（即NFS服务是否正常）

```
1  # 设置tftp服务端IP
2  setenv serverip 192.168.50.51
3  # 验证是否正确
4  printenv serverip
5
6  # 使用tftp下载
7  tftp 0x80800000 zImage;tftp 0x83000000 imx6ull-mmc-npi.dtb;tftp
   0x88000000 initrd.img
8  # 使用boot命令启动内核
9  bootz 0x80800000 0x88000000:0x4ECEBC 0x83000000
10
11 # 检查IP
12 ifconfig -a
13 # 重新目标机配置IP
14 ifconfig eth0 192.168.50.50
15 # 验证是否成功
16 ping 192.168.50.51
17
18
19 # 将nfs的目录挂载到/mnt目录下（没有mnt目录，就创建）
20 mount -t nfs -o nolock 192.168.50.51:/home/siguyuan/EBF_6ULL/rootfs
   /mnt
21
22 # 查看文件
23 cd /mnt && ls
24
25 # 运行刚刚的测试文件
26 ./build
```

```
■ mount -t nfs -o nolock 192.168.50.51:/home/siguyuan/EBF_6ULL/rootfs
   /mnt
```

- `-t (Type)`：用于指定要挂载的**文件系统类型**。
- `nfs`：告知内核这是一个**网络文件系统**（Network File System），内核会调用 NFS 协议栈驱动（而不是 Ext4、FAT 等本地块设备驱动）去建立网络连接和解析 RPC 协议。

- **-o (Options)**: 后面跟具体的挂载附加选项 (如果有多个选项, 用逗号分隔, 如 `-o nolock,nfsvers=3`) 。
- **nolock (关键选项)** : 禁用 NLM (Network Lock Manager) 文件锁机制。
 - **为什么嵌入式开发必须加它?**
 - 标准 NFS 为了防止多台机器同时修改同一个文件, 需要运行 `rpc.statd` 和 `rpc.lockd` 这两个后台服务来进行文件锁定。然而, 在嵌入式设备启动阶段 (如 `initramfs` 或精简版系统) 中, **根本没有运行这些后台锁定服务**。如果**不加** `nolock`, 开发板在向宿主机申请文件锁时会因为等不到响应而**一直卡死 (Timeout / Hang)**。加上 `nolock` 可以让挂载操作直接跳过加锁申请。
 - `192.168.50.51:/home/siguyuan/EBF_6ULL/rootfs`: 远端 NFS 服务器上的资源路径
 - `/mnt`: 开发板本地的**挂载点目录**。
- 后续将该共享目录, 作为根设备进行文件系统的挂载。

制作文件系统内容:

1. shell可执行程序 (命令), 如`ls`、`cd`..

- 为了避免逐个实现和编译几百个 Linux 命令, 我们使用开源软件 `Busybox` (应用层软件) 来一站式生成并填充嵌入式根文件系统 (Rootfs) 所需的**系统命令、Shell 环境与网络服务工具**。

⚠ Caution

busybox编译时, 应该注意编译器版本 (即编译后命令需要在什么环境下执行)

如果出现问题, 有可能busybox语法不能被编译器失败, 可以修改busybox源码 (删除编译器不认识的代码) 或者 替换busybox版本

- 下载对应的源码文件 `busybox-1.36.1.tar.bz2`
- 解压源码文件: `tar -jxvf busybox-1.36.1.tar.bz2 -C ../busybox/`
- 配置busybox: `make menuconfig`

■ 错误

```
1 siguyuan@debian:~/EBF_6ULL/busybox/busybox-1.36.1$ make
2 menuconfig
3 ...
4 HOSTCC scripts/kconfig/lxdialog/yesno.o
5 HOSTLD scripts/kconfig/lxdialog/lxdialog
6 *** Unable to find the ncurses libraries or the
7 *** required header files.
8 *** 'make menuconfig' requires the ncurses libraries.
9 ***
10 *** Install ncurses (ncurses-devel) and try again.
11 ***
make[2]: *** [/home/siguyuan/EBF_6ULL/busybox/busybox-1.36.1/scripts/kconfig/lxdialog/Makefile:15: scripts/kconfig/lxdialog/dochecklxdialog] 错误 1
```

```

12 make[1]: *** [/home/siguyuan/EBF_6ULL/busybox/busybox-
13 1.36.1/scripts/kconfig/Makefile:14: menuconfig] 错误 2
14
15 # 发现库存在，且之前能够使用make menuconfig
16 siguyuan@debian:~/EBF_6ULL/busybox/busybox-1.36.1$ sudo apt
install libncurses-dev
17 libncurses-dev 已经是最新版 (6.5+20250216-2)。
18 libncurses-dev 已设置为手动安装。
19 摘要：
20 升级：0，安装：0，卸载：0，不升级：85
21
22 # 解决方式：该busybox的脚本出现了问题，修改脚本
23 nvim scripts/kconfig/lxdialog/Makefile +15
24 # 发现该Makefile与check-lxdialog.sh脚本有关，且之前报错有
scripts/kconfig/lxdialog/dochecklxdialog
25 # 检查该脚本
26 nvim scripts/kconfig/lxdialog/check-lxdialog.sh
27 # 发现有语法错误 缺少返回值 补齐 保存
28 main() {} => int main() {}
29
30 # 重新执行
31 make distclean
32 make menuconfig

```

- 解决思路：先确定库是存在，确定打印信息的脚本，读脚本
- 进行配置（裁剪）



- settings 中的 Build static binary (no shared libs) (NEW) 这里选择动态，为了遇到更多的错误，实际开发应由需求决定

对比维度	勾选（静态链接 / Static）	不勾选（动态链接 / Dynamic）
底层原理	编译时将 BusyBox 用到的所有 C 语言标准库函数代码， 直接打包打进 BusyBox 单个二进制文件中 。	编译时不打包库代码， 仅保存符号引用 。运行时依赖目标板 <code>/lib</code> 或 <code>/usr/lib</code> 目录下的动态库（如 <code>libc.so.6</code> 、 <code>libm.so</code> 等）。
对根文件系统（rootfs）库依赖	零依赖！ 即使 <code>/lib</code> 目录下没有任何 <code>.so</code> 动态库，BusyBox 也能完美独立运行。	强依赖！ 如果开发板 <code>/lib</code> 下缺少对应的交叉编译链 <code>.so</code> 库文件，启动时会直接抛出著名的崩溃报错： <code>kernel panic - not syncing: Attempted to kill init!</code> 或 <code>Failed to execute /init</code>
BusyBox 单文件体积	较大 （通常 2MB ~ 5MB 左右）。	极小 （通常几百 KB 左右）。
系统整体空间占用	如果系统里只有 BusyBox，总空间较小。但如果系统里有很多其他独立程序，静态链接会导致每个程序都重复打包一份 C 库代码，浪费 Flash 空间。	系统中所有程序 共享同一套 .so 动态库 ，大幅节省闪存（Flash）和内存（RAM）空间。
库状态	其他开发者无法使用	方便其他开发者使用

■ Applets :裁剪清单

菜单模块 (Menu)	建议操作	裁剪理由 / 作用说明
<code>Print Utilities</code> --->	全部取消	打印机服务（如 <code>lpd</code> ），嵌入式开发完全用不到。
<code>Mail Utilities</code> --->	全部取消	邮件收发工具（如 <code>sendmail</code> ），嵌入式根文件系统不需要。

菜单模块 (Menu)	建议操作	裁剪理由 / 作用说明
<code>Runit Utilities</code> ---->	全部取消	另一种替代 <code>init</code> 的进程管理服务，保留默认的 <code>Init Utilities</code> 即可。
<code>klibc-utils</code> --->	全部取消	为 <code>klibc</code> 提供的极小工具集， <code>BusyBox</code> 本身功能已完全覆盖。
<code>Debian Utilities</code> ---->	全部取消	特定于 <code>Debian</code> 发行版的辅助工具（如 <code>run-parts</code> ），极简系统用不上。
<code>Linux Ext2 FS Progs</code> ---->	全部取消	<code>Ext2/3/4</code> 文件系统修复与格式化工具（如 <code>e2fsck</code> ），挂载 <code>NFS</code> 或只读系统时无需格式化磁盘。
<code>Archival Utilities</code> ---->	按需精简	压缩/归档工具（如 <code>zip</code> , <code>dpkg</code> , <code>rpm</code> , <code>unxz</code> ）。若无解压需求可全关，仅保留 <code>tar</code> 即可。
<code>Login/Password Management</code> ---->	调试期取消	用户登录与密码管理（如 <code>login</code> , <code>passwd</code> , <code>su</code> ）。取消后系统启动将直接进入 <code>root Shell</code> ，免去输入密码步骤。
<code>Networking Utilities</code> ----> 取消指定项	取消指定项	已取消项： <code>IPv6 Support</code> （精简协议栈）、 <code>dnssd</code> （ <code>DNS</code> 服务）、 <code>ftpd</code> / <code>ftpput</code> / <code>ftpget</code> （ <code>FTP</code> 服务及客户端上传/下载工具）、 <code>httpd</code> （ <code>Web</code> 服务）、 <code>inetd</code> （超级服务）、 <code>ntpd</code> （时间同步）、 <code>tcpsvd</code> / <code>udpsvd</code> （网络服务监听）、 <code>telnet</code> / <code>telnetd</code> （远程登录客户端与服务端）、 <code>tftp</code> / <code>tftpd</code> （ <code>TFTP</code> 传输工具与服务）、 <code>udhcpd</code> （ <code>DHCP</code> 服务端，保留客户端 <code>udhcpc</code> 即可）、 <code>tc</code> （流量控制）。

菜单模块 (Menu)	建议操作	裁剪理由 / 作用说明
Shells →	取消指定项	已取消项: <code>hush</code> (极简 Shell, 功能较弱)。

- 退出保存后, 在当前解压目录下生成一个 `.config` 文件。【如果: 建议备份一些这个配置文件, 避免后面出现问题, 再次进行配置 `cp .config ../config.bak`】
- 编译, `bear -- make -j$(nproc)` 【这一步经常出错很正常】
- 安装 (将各个文件汇总): `make install` 【这一步经常出错很正常】
- 检查 `file _install/bin/busybox` 是否为arm体系
- 将该文件拷贝到共享目录rootfs中: `cp -a _install/* ../../rootfs/`
 - `-a`: 全属性保真拷贝 (把文件和目录“原封不动”地复制过去), 会保留软链接
- 为了遵循 **FHS (文件系统层次结构标准)**, 需要在共享目录 `rootfs` 下创建一些必须的目录
 - `mkdir -p root dev etc boot tmp var sys proc lib mnt home opt`

```
1 # busybox目录
2 siguyuan@debian:~/EBF_6ULL/busybox/busybox-1.36.1$ ls
3 applets                coreutils             INSTALL               Makefile.flags
4   procps                testsuite
5 applets_sh             debianutils           klibc-utils          Makefile.help
6   qemu_multiarch_testing TODO
7 arch                   docs                  libbb
8 make_single_applets.sh README                  TODO_unicode
9 archival               e2fsprogs             libpwdgrp             miscutils
10   runit                 util-linux
11 AUTHORS               editors               LICENSE               modutils
12   scripts
13 busybox_ldscript.README.txt examples             loginutils            networking
14   selinux
15 Config.in              findutils             mailutils
16 NOFORK_NOEXEC.lst      shell
17 configs                include               Makefile
18 NOFORK_NOEXEC.sh       size_single_applets.sh
19 console-tools          init                  Makefile.custom      printutils
20   sysklogd
21
22 # 这个架构目录, 并发busybox自带的, 是厂家加的补丁或者扩展。busybox具体在哪个平台执行由编译器决定
23 siguyuan@debian:~/EBF_6ULL/busybox/busybox-1.36.1$ ls arch
24 i386  sparc  sparc64  x86_64
```

- 错误

- 使用宿主机编译器编译的错误

- **Caution**

- 使用这个编译器编译和安装不会报错，去安装看的 `_install/bin/busybox` 的ELF格式头

- 1 # 编译和安装都没有报错，且生成了 `_install` 目录
 - 2 # 进入 `_install/bin` 目录，发现只有一个可执行程序，其他都是软连接
 - 3 siguyuan@debian:~/EBF_6ULL/busybox/busybox-1.36.1/_install/bin\$
 - 4 `ls -l`
 - 5 总计 756
 - 6 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 arch ->`
 - 7 `busybox`
 - 8 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 ash ->`
 - 9 `busybox`
 - 10 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 base32 ->`
 - 11 `busybox`
 - 12 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 base64 ->`
 - 13 `busybox`
 - 14 `-rwxr-xr-x 1 siguyuan siguyuan 771624 8月14日 17:11 busybox`
 - 15 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 cat ->`
 - 16 `busybox`
 - 17 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 chgrp ->`
 - 18 `busybox`
 - 19 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 chmod ->`
 - 20 `busybox`
 - 21 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 chown ->`
 - 22 `busybox`
 - 23 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 conspy ->`
 - 24 `busybox`
 - 25 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 cp ->`
 - 26 `busybox`
 - 27 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 ctttyhack ->`
 - 28 `busybox`
 - 29 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 date ->`
 - 30 `busybox`
 - 31 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 dd ->`
 - 32 `busybox`
 - 33 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 df ->`
 - 34 `busybox`
 - 35 `lrwxrwxrwx 1 siguyuan siguyuan 7 8月14日 17:11 dmesg ->`
 - 36 `busybox`
 - 37
 - 38 # 查看唯一可执行程序的ELF头
 - 39 siguyuan@debian:~/EBF_6ULL/busybox/busybox-1.36.1/_install\$
 - 40 `file bin/busybox`
 - 41 `bin/busybox: ELF 64-bit LSB pie executable, x86-64, version 1`
 - 42 `(SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-`
 - 43 `64.so.2,`
 - 44 `BuildID[sha1]=ac31424cbe808a7ba6519e790d008e7cdae22fd0, for`
 - 45 `GNU/Linux 3.2.0, stripped`

```

25 # 发现 该文件是64位，运行在x86-64机器上的文件，且是动态链接，并发arm环境
    下的32位的文件
26
27 # 解决方式：先切换成交叉编译器，备份.config文件，在make distclean
    将.config复制过来，重新编译（make -j4 ），安装（make install）
28 source ../../env/arm_path.sh
29 make distclean
30 cp ../config.bak .config
31 bear -- make -j$(nproc)
32 make install
33
34 # 检查_install/bin目录下的可执行文件，以及当前目录下有没有json文件
35 siguyuan@debian:~/EBF_6ULL/busybox/busybox-1.36.1$ file
    _install/bin/busybox
36 _install/bin/busybox: ELF 32-bit LSB executable, ARM, EABI5
    version 1 (SYSV), dynamically linked, interpreter /lib/ld-
    linux-armhf.so.3, for GNU/Linux 3.2.0, stripped
37
38 siguyuan@debian:~/EBF_6ULL/busybox/busybox-1.36.1$ ls
    compile_commands.json
39 compile_commands.json
40

```

- 为什么一个可执行文件 busybox 能被这么多功能链接

- 根据名字分支执行（argv[0] 机制）

当你在终端敲下 `cat` 时，系统顺着软链接调用的依然是 `busybox` 可执行文件。BusyBox 启动后读取自己的调用名称 `argv[0]` 为 `"cat"`，便自动跳转去执行内部的 `cat_main()` 函数。 `int main(int argc, char **argv){}`

- 共享底层函数库（代码重用）

几百个命令不再各自编写文件处理、参数解析等基础代码，而是统一复用 BusyBox 内部公共库（`libbb`），极大缩减了冗余代码。

- 极小化体积与内存开销

避免了生成几百个小文件带来的磁盘空间浪费（Flash 碎片化）；同时在运行时，内存中只需要加载一份 `busybox` 代码段（RAM 占用极低）。

○

2. 将刚刚弄好的文件系统，通过NFS挂载成为板子的根文件系统

- 1 |

```

1  # 上电
2  # 设置tftp服务端IP
3  setenv serverip 192.168.50.51
4  # 验证是否正确
5  printenv serverip
6
7  # 使用tftp下载
8  tftp 0x80800000 zImage;tftp 0x83000000 imx6ull-mmc-npi.dtb;
9  ...
10

```

- 配置boots启动时，以NFS共享目录为跟设备

```

1  # 配置变量
2  setenv bootargs console=ttymxc0,115200 root=/dev/nfs rw
   nfsroot=192.168.50.51:/home/siguyuan/EBF_6ULL/rootfs,proto=tcp,nfsvers=3
   ip=192.168.50.50:192.168.50.51::255.255.255.0::eth0:off init=/linuxrc
3
4  # 使用boot启动内核，不要写文件系统位置
5  bootz 0x80800000 - 0x83000000

```

- `setenv bootargs`：设置名为 `bootargs` 的 U-Boot 环境变量。U-Boot 引导内核时，会自动把这一长串字符串传递给内核。
- `console=ttymxc0,115200`
 - `console=ttymxc0`：指定串口 `ttymxc0`（NXP i.MX6ULL 芯片对应的 UART1 串口）作为系统的控制台输出/输入终端。
 - `115200`：设置串口通信的波特率为 115200bps。
 - `root=/dev/nfs`：告诉内核，根文件系统不是存放在本地的 EMMC、SD 卡或 Flash 上，而是要**通过 NFS（网络文件系统）从网络挂载**。
 - `rw`：以可读可写（Read-Write）模式挂载根文件系统。如果写成 `ro` 就是只读。
 - `nfsroot=192.168.50.51:/home/siguyuan/EBF_6ULL/rootfs,proto=tcp,nfsvers=3`
 - `192.168.50.51`：NFS 服务器（你的宿主机 Linux PC/虚拟机）的 IP 地址。
 - `/home/siguyuan/EBF_6ULL/rootfs`：主机上共享出来的根文件系统绝对路径。
 - `proto=tcp`：指定使用 TCP 协议传输 NFS 数据包（比默认的 UDP 更稳定，不易丢包）。
 - `nfsvers=3`：显式指定使用 NFSv3（版本 3）协议。这是嵌入式 Linux NFS 挂载最兼容、最稳定的版本。

- `ip=192.168.50.50:192.168.50.51::255.255.255.0::eth0:off`

告诉内核在挂载 NFS 之前，先为网络接口配置好静态 IP。它的格式为：

```

1  ip=<client-ip>:<server-ip>:<gw-ip>:<netmask>:<hostname>:<device>:<autoconf>

```

- `192.168.50.50`：开发板自身的 IP 地址（Client IP）。
- `192.168.50.51`：NFS 服务器/网关 IP（Server IP）。
- `::`：跳过了网关 IP（如果不需要跨网段路由可留空）。

- `255.255.255.0`: 子网掩码 (Netmask) 。
- `::`: 跳过了主机名 (Hostname) 。
- `eth0`: 指定使用的物理网卡接口名 (NXP 平台上通常是 `eth0` 或 `eth1`) 。
- `off`: 关闭自动配置 (不使用 DHCP 或 BOOTP 自动获取, 直接使用前面指定的静态 IP) 。

- `init=/linuxrc`

- 指定内核挂载完根文件系统后, 执行的**第一个用户空间程序 (PID = 1)** 。
- 在这里执行的是 BusyBox 生成的 `/linuxrc` (通常是一个软链接, 指向 `bin/busybox`) , 由它来继续初始化系统并唤醒控制台 Shell。
- 错误1:

```

1  # 使用boot启动内核时, 出现内核惊恐, 且错误码为 -2 , 表示缺少了交叉编译器的 C 动态链接库
   (动态解析器 / libc)
2
3  [ 5.948643] ALSA device list:
4  [ 5.951640]   No soundcards found.
5  [ 6.036677] VFS: Mounted root (nfs filesystem) on device 0:15.
6  [ 6.044779] devtmpfs: mounted
7  [ 6.050745] Freeing unused kernel memory: 1024K
8  [ 6.055907] Run /linuxrc as init process
9  [ 6.080460] Kernel panic - not syncing: Requested init /linuxrc failed
   (error -2).
10 [ 6.088088] ---[ end kernel panic - not syncing: Requested init
   /linuxrc failed (error -2). ]---
11
12 # 解决方式, 在虚拟机 (宿主机) 上, 查看rootfs/bin目录下的可执行文件需要哪些库
13 # 需要/lib/ld-linux-armhf.so.3库
14 siguyuan@debian:~/EBF_6ULL/rootfs$ file bin/busybox
15 bin/busybox: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV),
   dynamically linked, interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux
   3.2.0, stripped
16
17 # 查看符号表 , 发现缺少libresolv.so.2、libm.so.6
18 siguyuan@debian:~/EBF_6ULL/rootfs$ arm-none-linux-gnueabi-hf-readelf -d
   bin/busybox
19
20 Dynamic section at offset 0x8df00 contains 27 entries:
21   Tag          Type                               Name/Value
22   0x00000001 (NEEDED)                Shared library: [libm.so.6]
23   0x00000001 (NEEDED)                Shared library: [libresolv.so.2]
24   0x00000001 (NEEDED)                Shared library: [libc.so.6]
25   0x0000000c (INIT)                  0x14ed8
26   0x0000000d (FINI)                   0x7db7c
27   0x00000019 (INIT_ARRAY)            0xadef8
28   0x0000001b (INIT_ARRAYSZ)          4 (bytes)
29   0x0000001a (FINI_ARRAY)            0xadefc
30   0x0000001c (FINI_ARRAYSZ)          4 (bytes)
31   0x00000004 (HASH)                  0x10190
32   0x6ffffff5 (GNU_HASH)              0x10be0
33   0x00000005 (STRTAB)                0x130d8
34   0x00000006 (SYMTAB)                0x11828

```

```

35 0x0000000a (STRSZ) 3529 (bytes)
36 0x0000000b (SYMENT) 16 (bytes)
37 0x00000015 (DEBUG) 0x0
38 0x00000003 (PLTGOT) 0xae000
39 0x00000002 (PLTRELSZ) 3072 (bytes)
40 0x00000014 (PLTREL) REL
41 0x00000017 (JMPREL) 0x142d8
42 0x00000011 (REL) 0x14298
43 0x00000012 (RELSZ) 64 (bytes)
44 0x00000013 (RELENT) 8 (bytes)
45 0x6fffffff (VERNEED) 0x141b8
46 0x6fffffff (VERNEEDNUM) 3
47 0x6fffffff0 (VERSYM) 0x13ea2
48 0x00000000 (NULL) 0x0
49
50 # 去交叉编译器中拷贝这些库（注意如果是软链接，需要将背后的那个实际文件也拷贝过来）
51 siguyuan@debian:~/EBF_6ULL/rootfs$ ls /opt/arm-linux-gnueabi/
52 9.2-2019.12-x86_64-arm-none-linux-gnueabi/manifest.txt arm-none-linux-
gnueabi/ bin include lib lib64 libexec share
53
54 # 外面的bin、lib、lib64是给宿主主机用的
55 # arm-none-linux-gnueabi/下面的bin、lib、lib64才是给目标机用的
56 # lib是给C++用的、libc是C使用的
57 siguyuan@debian:~/EBF_6ULL/rootfs$ ls /opt/arm-linux-gnueabi/arm-none-
linux-gnueabi/
58 bin include lib libc
59 # 选择对应的库
60 siguyuan@debian:~/EBF_6ULL/rootfs$ ls -l /opt/arm-linux-gnueabi/arm-
none-linux-gnueabi/libc/lib/{ld-linux-
armhf.so.3,libm.so.6,libresolv.so.2}
61 lrwxrwxrwx 1 siguyuan siguyuan 10 2019年12月 9日 /opt/arm-linux-
gnueabi/arm-none-linux-gnueabi/libc/lib/ld-linux-armhf.so.3 -> ld-
2.30.so
62 lrwxrwxrwx 1 siguyuan siguyuan 12 2019年12月 9日 /opt/arm-linux-
gnueabi/arm-none-linux-gnueabi/libc/lib/libm.so.6 -> libm-2.30.so
63 lrwxrwxrwx 1 siguyuan siguyuan 17 2019年12月 9日 /opt/arm-linux-
gnueabi/arm-none-linux-gnueabi/libc/lib/libresolv.so.2 -> libresolv-
2.30.so
64
65 # 发现是软连接需要，使用cp -a拷贝
66 siguyuan@debian:~/EBF_6ULL/rootfs$ cp -a /opt/arm-linux-gnueabi/arm-
none-linux-gnueabi/libc/lib/{ld*,libc*,libm*,libresolv*} lib/
67
68 siguyuan@debian:~/EBF_6ULL/rootfs$ ls lib/
69 ld-2.30.so ld-linux-armhf.so.3 libm-2.30.so libm.so.6 libresolv-
2.30.so libresolv.so.2

```

板子上重新上电，下载，启动

- 错误2:

- | | |
|---|---|
| 1 | 找不到文件 |
| 2 | # 需要系统起来之前，需要第一个脚本（调度员inittab【配置文件】，找不执行者rcs【脚本】） |

```

3      # 该脚本的功能是，挂载文件，如：mount -t proc proc /proc
4      [ 5.892907] IP-Config: Complete:
5      [ 5.896248]         device=eth0, hwaddr=7e:38:d0:f0:74:60,
ipaddr=192.168.50.50, mask=255.255.255.0, gw=255.255.255.255
6      [ 5.907306]         host=192.168.50.50, domain=, nis-domain=(none)
7      [ 5.913699]         bootserver=192.168.50.51, rootserver=192.168.50.51,
rootpath=
8      [ 5.921275] cfg80211: Loading compiled-in x.509 certificates for
regulatory database
9      [ 5.933290] cfg80211: Loaded x.509 cert 'sforshee: 00b28ddf47aef9cea7'
10     [ 5.940095] platform regulatory.0: Falling back to sysfs fallback for:
regulatory.db
11     [ 5.948735] ALSA device list:
12     [ 5.951780]   No soundcards found.
13     [ 6.003336] VFS: Mounted root (nfs filesystem) on device 0:15.
14     [ 6.011557] devtmpfs: mounted
15     [ 6.018599] Freeing unused kernel memory: 1024K
16     [ 6.024072] Run /linuxrc as init process
17     can't run '/etc/init.d/rcS': No such file or directory
18
19     # 制作配置文件 和 脚本

```

- 错误3（如果busybox拷贝到/opt目录【即需要root权限】）

- ```

1 # 解决方式：让sudo的环境变量env使用当前终端的环境变量
2 # 注意：使用sudo，会导致后面生成的命令都是需要sudo执行（root用户）【不推荐】
3 # 所以命令之前添加sudo PATH=$PATH -E
4 sudo PATH=$PATH -E make menuconfig

```

### 3. 系统的启动配置文件和脚本制作

#### 1. init配置文件：共享目录下的 `etc/inittab`

##### ⚠ Caution

注意路径，`rootfs/etc/inittab`，不要修改到了`/etc/inittab`

```

1 # 进入etc目录
2 cd etc/
3 # 创建inittab文件
4 nvim inittab

```

#### inittab内容

```

1 # /etc/inittab
2 # 格式为：<id>:<runlevels>:<action>:<process>
3
4 # -----
5 # 1. 系统启动初始化 (sysinit)
6 # -----
7 # 内核启动后执行的第一件事，指定运行 /etc/rc.d/rcS 脚本
8 # 在这个脚本中通常会处理 mount -a 挂载 proc/sys 等文件系统以及 mdev 自动创建设备节点
9 ::sysinit:/etc/rc.d/rcS

```

```

10
11 # -----
12 # 2. 交互控制台与 shell 配置
13 # -----
14 # 在控制台 (console) 上打开一个 shell。
15 # askfirst 表示在开启 shell 前提示 "Please press Enter to activate this
 console."
16 # -/bin/sh 前面的短横线 '-' 表示以 login shell 方式启动，会加载环境变量
17 console::askfirst:~/bin/sh
18
19 # -----
20 # 3. 关机与卸载操作 (shutdown)【可选】
21 # -----
22 # 当执行 reboot 或 poweroff 关机/重启时，先安全卸载所有的文件系统
23 ::shutdown:/bin/umount -a -r

```

在 BusyBox 中，`/etc/inittab` 的语法格式非常简单固定，每一行的基本格式如下：`id : runlevels : action : process`

**id**：表示命令行终端 (TTY)。比如 `console` (系统默认控制台)、`tty1`，或者**留空** (适用于无需绑定特定终端的系统初始化任务)。

**runlevels**：运行级别。在嵌入式 BusyBox 中**通常忽略并留空** (BusyBox 不区分传统的 Linux 运行级别)。

**action**：动作类型。**最核心的字段**，告诉 `init` 何时以及如何运行 `process`：

- `sysinit`：系统**初始化时**执行 (仅执行一次，且必须等它执行完毕才继续后续任务)。
- `respawn`：**死而复生**。程序一旦退出，`init` 就会自动重新启动它。
- `askfirst`：类似 `respawn`，但在启动前会先打印提示语 (如 "Please press Enter to activate this console.")，敲回车后才启动 Shell。
- `shutdown`：关机或重启时执行。
- `ctrlaltdel`：按下 `Ctrl + Alt + Del` 组合键时执行。

**process**：要执行的程序或脚本路径 (可以带参数)。

## 2. 创建执行者 `rcs` 脚本

```

1 # 创建目录
2 mkdir etc/rc.d
3 # 创建脚本
4 nvim etc/rc.d/rcS
5 # 修改脚本执行权限
6 chmod +x etc/rc.d/rcS
7

```

`rcS` 的内容

```

1 #!/bin/sh
2
3 # 挂载目录

```



```

4 /bin/mount -a
5
6 # 自动创建设备节点，配置“热插拔监听”
7 # 以后只要有任何新设备插入或拔出（比如插拔 U 盘、网卡等），请立刻调用 /sbin/mdev 这个程序来处理
8 echo /sbin/mdev > /proc/sys/kernel/hotplug
9
10 # 开始创建
11 # 在系统启动时，把板子上“已经存在/板载”的所有硬件设备，一次性全部在 /dev 目录下生成对应的设备节点
12 mdev -s
13
14 # 开机终端打印信息
15 echo "aaa"

```

`mount -a`：会自动读取 `/etc/fstab` 文件进行脚本挂载

### 3. 创建，并编写 `fstab` 文件。 `nvim /etc/fstab`

```

1 # /etc/fstab: 静态文件系统挂载配置文件
2 #
3 # <file system> <mount point> <type> <options> <dump>
4 # [设备/虚拟源] [挂载点目录] [文件系统类型] [挂载选项] [备份标志] [自
5 # -----
6 # -----
7 # 1. 挂载内核进程信息虚拟文件系统（必需，很多系统命令如 ps/top 依赖此目录）
8 proc /proc proc defaults 0 0
9 # 2. 挂载系统设备树与驱动信息虚拟文件系统（必需，mdev 动态创建设备节点依赖此目录）
10 tmpfs /tmp tmpfs defaults 0 0
11 # 3. 挂载内存临时文件系统（将 /tmp 放在内存中，读写更快且减少 Flash/SD卡写磨损）
12 sysfs /sys sysfs defaults 0 0
13 # 4. 挂载设备节点虚拟文件系统（配合 mdev 管理 /dev 目录下的设备文件）
14 tmpfs /dev tmpfs defaults 0 0
15
16 # 5. 挂载伪终端文件系统（用于支持 SSH / Telnet / 虚拟终端等多用户登录，可选）
17 # devpts /dev/pts devpts mode=0620,gid=5 0 0

```

### 4. shell 的配置文件 `/etc/profile`。 `nvim /etc/profile`

```

1 #!/bin/sh
2 # /etc/profile: 全局环境变量配置文件
3 # BusyBox Shell 在登录或启动控制台时会自动加载此文件
4
5 # 1. 设置主机名环境变量（可在提示符中引用或被应用程序读取）
6 export HOSTNAME=localhost
7
8 # 2. 设置当前登录用户名（嵌入式系统默认通常为 root 超级用户）
9 export USER=root
10 # 3. 设置终端命令行提示符格式（PS1）

```

```

11 # \w : 显示当前工作目录的完整路径
12 # \# : 显示当前命令的执行序号
13 # 终端最终显示效果示例: [root@localhost /etc/init.d]#
14 export PS1='[${USER}@${HOSTNAME}] \w]\# '
15
16 # 4. 设置可执行命令的全局搜索路径 (PATH)
17 # 冒号分隔。当你在命令行敲命令时, shell 会按顺序到这些目录下查找对应的可执行程序
18 export PATH=/bin:/sbin:/usr/bin:/usr/sbin
19
20 # 5. 设置动态链接库搜索路径 (LD_LIBRARY_PATH)
21 # 告诉动态链接器 (ld-linux) 在运行程序时优先到 /lib 和 /usr/lib 寻找 .so 动态库
22 export LD_LIBRARY_PATH=/lib:/usr/lib:$LD_LIBRARY_PATH

```

加权限: `chmod +x etc/profile`

#### 4. 重新烧写到板子中

```

1 # 启动成功的现象
2 [5.933286] cfg80211: Loaded x.509 cert 'sforshee: 00b28ddf47aef9cea7'
3 [5.940093] platform regulatory.0: Falling back to syfs fallback for:
 regulatory.db
4 [5.948604] ALSA device list:
5 [5.951602] No soundcards found.
6 [6.004033] VFS: Mounted root (nfs filesystem) on device 0:15.
7 [6.012121] devtmpfs: mounted
8 [6.019250] Freeing unused kernel memory: 1024K
9 [6.024735] Run /linuxrc as init process
10 "EBF-6ULL"
11
12 Please press Enter to activate this console.
13
14 [root@localhost ~]# ls proc/
15 1 18 85 devices mounts
16 10 19 86 diskstats mtd
17 103 2 87 driver net
18 104 20 88 execdomains pagetypeinfo
19 105 21 89 fb partitions
20 106 22 9 filesystems self
21 107 23 90 fs softirqs
22 108 24 91 interrupts stat
23 109 25 92 iomem swaps
24 11 26 93 ioports sys
25 114 27 94 irq sysrq-trigger
26 115 28 95 kallsyms sysvipc
27 116 29 asound key-users thread-self
28 117 3 buddyinfo keys timer_list
29 118 30 bus kmsg tty
30 12 31 cgroups kpagecgroup uptime
31 122 4 cmdline kpagecount version
32 123 5 config.gz kpageflags vmallocinfo
33 13 6 consoles loadavg vmstat
34 14 7 cpu locks zoneinfo
35 15 8 cpuinfo meminfo

```

```

36 | 16 81 crypto misc
37 | 17 82 device-tree modules
38 | [root@localhost ~]#
39 |
40 |

```

### ⚠ Caution

服务脚本：运行时需要添加 & `./build &`

**阻塞性命令/死循环程序**（比如自定义应用 `./build`），**必须放在后台运行（追加 &）**

#### 3.4.1.1 一步到位脚本（需要修改）：

```

1 | # mkfs
2 | #!/bin/sh
3 | cd ~/EBF_6ULL/rootfs || exit # rootfs目录
4 |
5 | echo "-----Create root,dev..."
6 | mkdir -p root dev etc boot tmp var sys proc lib mnt home opt
7 | mkdir -p etc/rc.d
8 |
9 | echo "create inittab"
10 | echo "::sysinit:/etc/rc.d/rcS" > etc/inittab
11 | echo "::askfirst:-/bin/sh" >> etc/inittab
12 |
13 | echo "create fstab"
14 | echo "proc /proc proc defaults 0 0" > etc/fstab
15 | echo "tmpfs /tmp tmpfs defaults 0 0" >> etc/fstab
16 | echo "sysfs /sys sysfs defaults 0 0" >> etc/fstab
17 | echo "tmpfs /dev tmpfs defaults 0 0" >> etc/fstab
18 |
19 | echo "create rcS file"
20 | echo '#!/bin/sh' > etc/rc.d/rcS
21 | echo "/bin/mount -a" >> etc/rc.d/rcS
22 | echo "echo /sbin/mdev > /proc/sys/kernel/hotplug" >> etc/rc.d/rcS
23 | echo "mdev -s" >> etc/rc.d/rcS
24 | echo 'mkdir -p /lib/modules/`uname -r`' >> etc/rc.d/rcS
25 | chmod +x etc/rc.d/rcS
26 |
27 | echo "create profile"
28 | echo "export HOSTNAME=localhost" > etc/profile
29 | echo "export USER=root" >> etc/profile
30 | echo "PS1='[\${USER}@${HOSTNAME} \w]\#'" >> etc/profile
31 | echo "PATH=/bin:/sbin:/usr/bin:/usr/sbin" >> etc/profile
32 | echo "LD_LIBRARY_PATH=/lib:/usr/lib:${LD_LIBRARY_PATH}" >> etc/profile
33 | echo "export PATH LD_LIBRARY_PATH" >> etc/profile
34 |
35 | echo "----make direction done-----\n"

```

### 3.4.2 找到一个管理员（文件系统格式）格式化 —— 将NFS创建好的系统打包到ram类或者flash类

在计算机中，无论是物理磁盘（HDD/SSD/SD卡）还是内存中的虚拟盘（RAMDisk/tmpfs），“裸存储空间”都只是一块连续的字节数组。**文件系统就是给这片空间建立的“账本与目录树”**，它决定了数据在介质上如何组织、存储、寻址和恢复。

常见文件系统全景整理与补充：

| 平台/场景      | 文件系统              | 开发/维护者    | 核心特点与适用场景                                                                                                                                                                   |
|------------|-------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Windows 体系 | FAT32             | 微软        | <b>兼容性极强</b> （所有系统都认识）。但非常老旧， <b>单文件最大不能超过 4GB</b> ，且缺乏日志机制，易损坏。常用于 U 盘、嵌入式 SD 卡。                                                                                           |
|            | exFAT             | 微软        | 专门为闪存（U 盘/SD 卡）设计的 <b>FAT32 升级版</b> 。突破了 4GB 文件限制，兼容 Windows/macOS/Linux，是目前大容量 U 盘/内存卡的主流格式。                                                                               |
|            | NTFS              | 微软        | Windows 现代主系统的标准格式。支持高权限控制（ACL）、 <b>日志恢复</b> 、大文件存储。但在 macOS 上默认 <b>只读</b> 。                                                                                                |
| Linux 体系   | EXT 系列 (EXT2/3/4) | Linux 社区  | Linux 系统的经典文件系统。 <ul style="list-style-type: none"><li>• <b>EXT2</b>：无日志，适合老旧闪存；</li><li>• <b>EXT3</b>：引入日志机制；</li><li>• <b>EXT4</b>：当前最普及的 Linux 标准格式，性能强劲、稳定性高。</li></ul> |
|            | Btrfs             | 社区/Oracle | 下一代 Linux 文件系统。支持 <b>写时复制 (CoW)</b> 、快照 (Snapshot)、RAID 和动态扩容。                                                                                                              |
|            | F2FS              | 三星/Linux  | 专为 <b>Flash 闪存/NVMe SSD</b> 命名的文件系统。优化了读写寿命，广泛用于 Android 手机和嵌入式 Linux 存储。                                                                                                   |
| Apple 体系   | APFS              | 苹果        | 替代老旧 HFS+ 的现代文件系统。专为 SSD 优化，支持极速文件克隆、空间共享、全盘加密与快照。                                                                                                                          |
| 嵌入式专门      | UBIFS / JFFS2     | Linux 社区  | 专门用于 <b>Raw MTD Flash</b> （如裸 SPI Flash / Parallel NAND）的文件系统，自带 <b>磨损平衡 (Wear Leveling)</b> 和坏块管理。                                                                         |
| 内存/虚拟      | tmpfs / proc      | Linux 内核  | <b>纯内存文件系统</b> 。数据在内存中，掉电即失。 <code>tmpfs</code> 常用于 <code>/tmp</code> 提高读写性能； <code>proc</code> / <code>sysfs</code> 用于暴露内核数据。                                              |

“格式化”，本质上就是在存储介质的头部和特定位置**写入文件系统的“元数据 (Metadata)”**。

如：在 Windows 里格式化时，系统会在磁盘上画出以下区域：



- 1. **超级块 (Superblock)**：记录整个文件系统的全局信息（比如总容量、块大小、已用/剩余空间、文件系统类型标志）。内核挂载（`mount`）时首先读取的就是它。
- 2. **inode 索引表 (Linux 核心)**：保存每个文件的元数据（文件大小、权限、所有者、修改时间、以及**指向数据实际物理存放位置的指针**）。
- 3. **数据块区 (Data Blocks)**：真正用来存放文件内容的地方。

3.4.2.1 ram类

1、先创建目录：

```
1 mkdir tools
2
3 cd tools/
```

2、在当前目录下创建一个大小为 8MB（8192 KB）、内容全为二进制 0 的空白镜像文件（`ebf_rd.img`）：

常用于**制作 RAMDisk（内存盘）、极简根文件系统（Rootfs）镜像，或者给 Flash 烧录镜像预留固定大小的空白空间。**

```
1 dd if=/dev/zero of=ebf_rd.img bs=1k count=8192
```

| 参数                         | 全称与含义                           | 在此命令中的作用                                                                                                  |
|----------------------------|---------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>dd</code>            | <b>Data Duplicator</b> (数据复制工具) | Linux 下用于低级数据拷贝、镜像制作的底层物理复制命令。                                                                            |
| <code>if=/dev/zero</code>  | <b>Input File</b> (输入源)         | 从 <code>/dev/zero</code> 读取数据。 <code>/dev/zero</code> 是 Linux 的特殊设备，会无限源源不断地提供空字节（全为 <code>0x00</code> ）。 |
| <code>of=ebf_rd.img</code> | <b>Output File</b> (输出目标)       | 把读取到的数据写入当前目录下名为 <code>ebf_rd.img</code> 的文件。                                                             |
| <code>bs=1k</code>         | <b>Block Size</b> (块大小)         | 指定单次读写的块大小为 <b>1 KB</b> （即 1024 字节）。                                                                      |
| <code>count=8192</code>    | <b>Count</b> (复制块的总数量)          | 告诉 <code>dd</code> 总共拷贝 <b>8192 个块</b> 。                                                                  |

3、强制将刚才创建的 `ebf_rd.img` 镜像文件格式化为 `ext2` 文件系统：

此时 `ebf_rd.img` 已经从一个“纯 0 的空白块”变成了一个包含完整 `ext2` 文件系统结构（超级块、inode 表、根目录 `/`）的虚拟磁盘镜像。

```
1 | sudo mkfs.ext2 -F ebf_rd.img
```

| 参数                      | 含义与作用                                                                                                                                                                           |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sudo</code>       | 以管理员/超级用户权限执行格式化操作。                                                                                                                                                             |
| <code>mkfs.ext2</code>  | <b>M</b> ake <b>F</b> ile <b>S</b> ystem <code>ext2</code> 。专门用于创建/格式化 <code>ext2</code> 文件系统的工具（底层调用的其实是 <code>mke2fs</code> ）。                                                |
| <code>-F</code>         | <b>F</b> orce（强制执行）。默认情况下， <code>mkfs</code> 发现 <code>ebf_rd.img</code> 是一个普通文件而不是物理硬盘设备（如 <code>/dev/sdb</code> ）时，会弹出警告并询问确认。加上 <code>-F</code> 参数可以 <b>跳过交互式确认，直接强制格式化</b> 。 |
| <code>ebf_rd.img</code> | 目标镜像文件。格式化后，这个文件内部就会建立起 <code>ext2</code> 的“账本与目录树”。                                                                                                                            |

4、把之前构建好的根文件系统（包含 `/etc/inittab`、`/etc/init.d/rcs`、`bin`、`sbin` 等所有文件）安全完整地“灌入”到刚刚格式化好的 `ebf_rd.img` 镜像文件中：

挂载镜像到本地目录：

```
1 | # 创建一个挂载目录
2 | mkdir -p aaa
3 | # 挂载
4 | sudo mount -o loop ebf_rd.img aaa
```

- `mount`：挂载命令。
- `-o loop`：**Loopback 设备挂载模式**（核心参数）。`ebf_rd.img` 只是一个普通的虚拟磁盘文件，并不是物理硬盘。`-o loop` 告诉 Linux 内核：“把这个镜像文件模拟/挂载成一个虚拟的块设备（类似 Windows 下加载 ISO 虚拟光驱）”。
- `ebf_rd.img`：上一步用 `mkfs.ext2` 格式化好的 8MB 镜像文件。
- `initrd`：宿主机上的本地挂载点文件夹。
- **作用**：挂载成功后，进入 `initrd/` 目录就相当于直接进入了这个 8MB 虚拟磁盘的根目录。

5、拷贝 Rootfs 文件（保持全量属性）：把准备好的 BusyBox、配置文件（`inittab`、`rcs`、`fstab`、`profile`）以及依赖库（`/lib`）**无损地复制**到虚拟磁盘镜像中。

```
1 | sudo cp -a ~/EBF_6ULL/rootfs/* aaa/
```

- `cp`：文件复制命令。
- `-a`：**Archive（归档/完全保留属性）**（核心参数）。它等价于 `-dpr`：
  - `-d`：保留软链接（Symbolic Links）。这对于 **BusyBox 极其关键！** 之前提到的 `/sbin/mdev ->`

`../bin/busybox` 等大量软链接必须原封不动地拷贝，不能被解析展开成普通文件。

- `-p`：保留原文件的权限（如 `rcs` 的可执行权限 `+x`）、所有者及时间戳。
- `-r`：递归复制整个目录树。
- `~/ebf/rootfs/\*`：你前面配置好的根文件系统源目录。
- `initrd/`：刚刚挂载好的镜像目标目录。

```
1 # 错误：设备空间不够
2 siguyuan@debian:~/EBF_6ULL/tools$ sudo cp -a ~/EBF_6ULL/rootfs/* aaa/
3 cp: 写入 'aaa/lib/libc-2.30.so' 时出错：设备上没有空间
4 cp: 无法创建目录 'aaa/mnt'：设备上没有空间
5 cp: 无法创建目录 'aaa/opt'：设备上没有空间
6 cp: 无法创建目录 'aaa/proc'：设备上没有空间
7 cp: 无法创建目录 'aaa/root'：设备上没有空间
8 cp: 无法创建目录 'aaa/sbin'：设备上没有空间
9 cp: 无法创建目录 'aaa/sys'：设备上没有空间
10 cp: 无法创建目录 'aaa/tmp'：设备上没有空间
11 cp: 无法创建目录 'aaa/usr'：设备上没有空间
12 cp: 无法创建目录 'aaa/var'：设备上没有空间
13
14 # 查看目录大小
15 siguyuan@debian:~/EBF_6ULL/tools$ du -sh ../rootfs/
16 20M ../rootfs/
17
18 # 查看各个目录的大小，发现lib目录大小过大
19 siguyuan@debian:~/EBF_6ULL/tools$ du -h ../rootfs/
20 8.0K ../rootfs/etc/rc.d
21 24K ../rootfs/etc
22 4.0K ../rootfs/tmp
23 19M ../rootfs/lib
24 4.0K ../rootfs/boot
25 4.0K ../rootfs/mnt
26 4.0K ../rootfs/usr/bin
27 4.0K ../rootfs/usr/sbin
28 12K ../rootfs/usr
29 4.0K ../rootfs/sys
30 4.0K ../rootfs/var
31 576K ../rootfs/bin
32 4.0K ../rootfs/home
33 4.0K ../rootfs/proc
34 4.0K ../rootfs/opt
35 4.0K ../rootfs/dev
36 4.0K ../rootfs/root
37 4.0K ../rootfs/sbin
38 20M ../rootfs/
39
40 # 发现libc-2.30.so 和 libm-2.30.so 过大
41 siguyuan@debian:~/EBF_6ULL/tools$ ls -lh ../rootfs/lib/
42 总计 19M
43 -rwxr-xr-x 1 siguyuan siguyuan 1.3M 2019年12月 9日 ld-2.30.so
44 lrwxrwxrwx 1 siguyuan siguyuan 10 2019年12月 9日 ld-linux-armhf.so.3 -> ld-2.30.so
45 -rwxr-xr-x 1 siguyuan siguyuan 15M 2019年12月 9日 libc-2.30.so
```

```

46 1rwxrwxrwx 1 siguyuan siguyuan 12 2019年12月 9日 libc.so.6 -> libc-2.30.so
47 -rwxr-xr-x 1 siguyuan siguyuan 2.5M 2019年12月 9日 libm-2.30.so
48 1rwxrwxrwx 1 siguyuan siguyuan 12 2019年12月 9日 libm.so.6 -> libm-2.30.so
49 -rwxr-xr-x 1 siguyuan siguyuan 407k 2019年12月 9日 libresolv-2.30.so
50 1rwxrwxrwx 1 siguyuan siguyuan 17 2019年12月 9日 libresolv.so.2 -> libresolv-2.30.so
51
52 # 发现二者有符号表和debug_info
53 siguyuan@debian:~/EBF_6ULL/tools$ file ../rootfs/lib/libc-2.30.so
54 ../rootfs/lib/libc-2.30.so: ELF 32-bit LSB shared object, ARM, EABI5 version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, with
debug_info, not stripped
55 siguyuan@debian:~/EBF_6ULL/tools$ file ../rootfs/lib/libm-2.30.so
56 ../rootfs/lib/libm-2.30.so: ELF 32-bit LSB shared object, ARM, EABI5 version 1 (SYSV),
dynamically linked, for GNU/Linux 3.2.0, with debug_info, not stripped
57
58 # 解决方式: 使用剔除指令
59 siguyuan@debian:~/EBF_6ULL/tools$ arm-none-linux-gnueabi-hf-strip ../rootfs/lib/*
60 siguyuan@debian:~/EBF_6ULL/tools$ ls -lh ../rootfs/lib/
61 总计 1.4M
62 -rwxr-xr-x 1 siguyuan siguyuan 104K 8月14日 21:05 ld-2.30.so
63 1rwxrwxrwx 1 siguyuan siguyuan 10 2019年12月 9日 ld-linux-armhf.so.3 -> ld-2.30.so
64 -rwxr-xr-x 1 siguyuan siguyuan 905K 8月14日 21:05 libc-2.30.so
65 1rwxrwxrwx 1 siguyuan siguyuan 12 2019年12月 9日 libc.so.6 -> libc-2.30.so
66 -rwxr-xr-x 1 siguyuan siguyuan 346K 8月14日 21:05 libm-2.30.so
67 1rwxrwxrwx 1 siguyuan siguyuan 12 2019年12月 9日 libm.so.6 -> libm-2.30.so
68 -rwxr-xr-x 1 siguyuan siguyuan 50K 8月14日 21:05 libresolv-2.30.so
69 1rwxrwxrwx 1 siguyuan siguyuan 17 2019年12月 9日 libresolv.so.2 -> libresolv-2.30.so
70
71 # 删除重新来卸载挂载点
72 sudo umount aaa
73 rm -fr ebf_rd.img

```

## 6、卸载挂载点:

```

1 sudo umount aaa
2

```

## 7、为了满足嵌入式设备的存储要求, 可以使用压缩格式

```

1 # 使用这种方式是为了防止原文件被更改
2 gzip --best -c ebf_rd.img> ebf_rd.img.gz
3 rm -fr aaa ebf_rd.img
4 # 放大tftp共享目录中
5 sudo cp ebf_rd.img.gz /opt/out_dir/EBF_6ULL/
6

```

## 8、记录文件大小



```

1 siguyuan@debian:~/EBF_6ULL/tools$ ls -l /opt/out_dir/EBF_6ULL/
2 总计 16700
3 -rw-r--r-- 1 root root 1361949 8月14日 21:16 ebf_rd.img.gz
4 -rw-r--r-- 1 root root 38529 8月13日 18:29 imx6ull-mmc-npi.dtb
5 -rw-rw-r-- 1 siguyuan siguyuan 5164732 8月13日 18:52 initrd.img
6 -rwxr-xr-x 1 root root 196 8月12日 19:56 led.bin
7 -rw-rw-r-- 1 tftp tftp 400384 8月12日 16:33 u-boot-dtb.img
8 -rwxr-xr-x 1 root root 10122704 8月13日 18:29 zImage
9
10 # ebf_rd.img.gz文件大小为1361949,转化成16进制: 0x14C81D
11

```

## 9、在板子上下载，并且启动

```

1 # 改IP
2 setenv serverip 192.168.50.51
3
4 # initrd=地址,ebf的文件大小
5 setenv bootargs console=ttymxc0,115200 root=/dev/ram rw initrd=0x88000000,0x14C81D
 init=/linuxrc
6 tftp 0x80800000 zImage;tftp 0x83000000 imx6ull-mmc-npi.dtb;tftp 0x88000000
 ebf_rd.img.gz;bootz 0x80800000 - 0x83000000

```

成功现象：

```

1 [2.592847] imx_thermal 2000000.aps-bus:tempmon: Industrial CPU temperature grade
 - max:105C critical:100C passive:95C
2 [2.605602] snvs_rtc 20cc000.snvs:snvs-rtc-lp: setting system clock to 1970-01-01
 00:01:38 UTC (98)
3 [2.615190] cfg80211: Loading compiled-in x.509 certificates for regulatory
 database
4 [2.626888] cfg80211: Loaded x.509 cert 'sforshee: 00b28ddf47aef9cea7'
5 [2.633975] platform regulatory.0: Falling back to syfs fallback for: regulatory.db
6 [2.642263] ALSA device list:
7 [2.645454] No soundcards found.
8 [2.649391] RAMDISK: gzip image found at block 0
9 [2.921238] EXT4-fs (ram0): mounted filesystem without journal. Opts: (null)
10 [2.928655] VFS: Mounted root (ext4 filesystem) on device 1:0.
11 [2.935331] devtmpfs: mounted
12 [2.941245] Freeing unused kernel memory: 1024K
13 [2.946409] Run /linuxrc as init process
14 "EBF-6ULL"
15
16 Please press Enter to activate this console.
17 [root@localhost ~]# ls proc/
18 1 20 87 diskstats mounts
19 10 21 88 driver mtd

```

```

20 103 22 89 execdomains net
21 104 23 9 fb pagetypeinfo
22 105 24 90 filesystems partitions
23 106 25 91 fs self
24 107 26 92 interrupts softirqs
25 108 27 93 iomem stat
26 109 28 94 ioports swaps
27 11 29 95 irq sys
28 114 3 asound kallsyms sysrq-trigger
29 118 30 buddyinfo key-users sysvipc
30 119 31 bus keys thread-self
31 12 4 cgroups kmsg timer_list
32 13 5 cmdline kpagecgrou tty
33 14 6 config.gz kpagecount uptime
34 15 7 consoles kpageflags version
35 16 8 cpu loadavg vmallocinfo
36 17 81 cpuinfo locks vmstat
37 18 82 crypto meminfo zoneinfo
38 19 85 device-tree misc
39 2 86 devices modules
40 [root@localhost ~]#
41

```

将busybox的源码添加到VScode中

### 3.4.2.1.1 一步到位脚本（需要修改）

```

1 # mkRamDisk.sh
2 #!/bin/sh
3 # 创建一个挂载点
4 mkdir -p initrd
5 # 分配一个镜像文件（多少个block，每块多大）查看制作好的文件系统的大小，计划分配8M
6 # dd if=/dev/zero of=initrd.img bs=1k count=8192
7 dd if=/dev/zero of=ebf_rd.img bs=1k count=8192
8 # 分配一个镜像，不需要分区（ramdisk），但需要格式化
9 sudo mkfs.ext2 -F ebf_rd.img
10 # 挂载格式化好的镜像到宿主机里的目录，方便写入
11 sudo mount -o loop ebf_rd.img initrd
12 sudo cp -a ~/EBF_6ULL/rootfs/* initrd/
13 sudo umount initrd
14 # 为了满足嵌入式设备的存储要求，可以使用压缩格式
15 gzip --best -c ebf_rd.img> ebf_rd.img.gz
16 rm -fr initrd ebf_rd.img
17 cp ebf_rd.img.gz /opt/out/
18

```

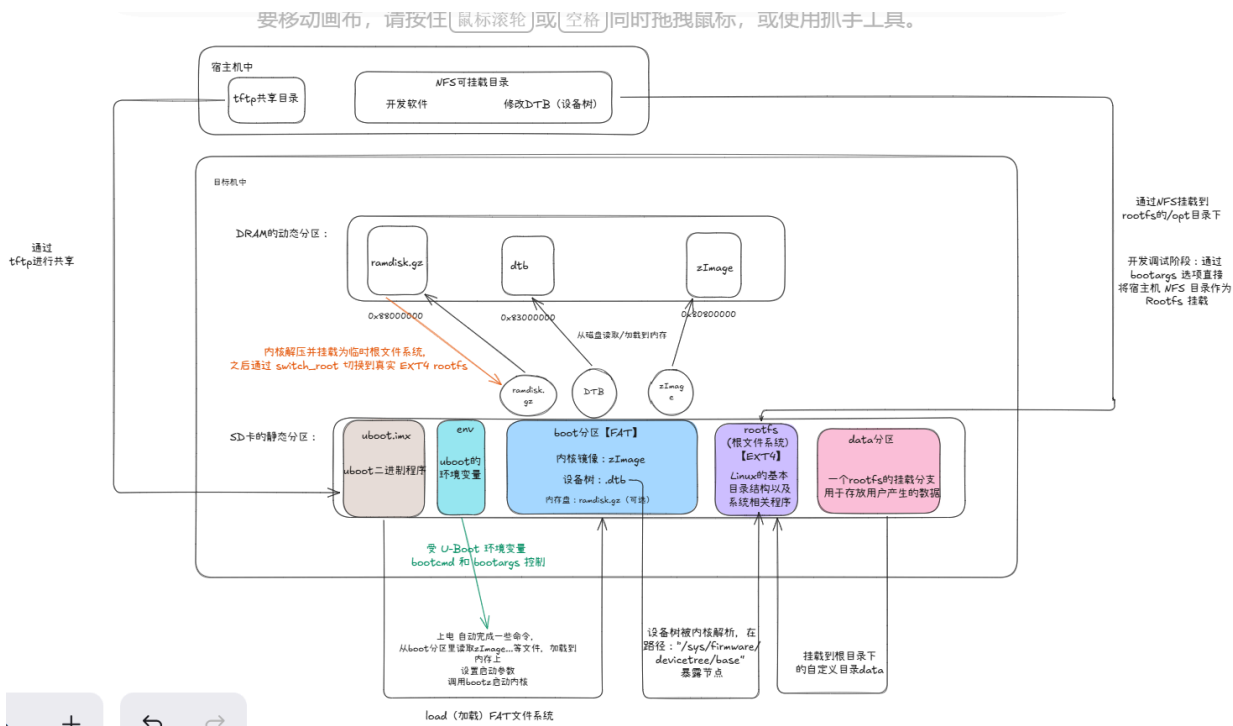
3.4.2.2 flash类

系统移植所需要的所有的镜像文件 打包生成一个可以直接交给工厂批量烧写的一个文件（镜像文件 `.img` / `.raw`）。

生成该文件之前，需要对存储器（SD卡 / flash）的进行布局（哪一块内存放什么程序）设计。

该文件需要实现程序上电自动进行 `bootloader`、`内核加载`、`文件系统挂载` 步骤

1. 对存储器进行分局与布局：



将32GB的SD卡，预留200MB的默认系统镜像。

对于200MB的分区：

- a. 分区表：用于记录SD卡的分区情况
- b. Raw Area （裸区域）：需要读取boot分区，将其加载到内存中，不设计文件名，所以不需要文件系统
  - i. `uboot.imx` —— 纯二进制文件，bootloader
  - ii. `uboot`使用的相关环境变量 `env` 【`saveenv` 之后，变量保存的磁盘空间】
    - `bootdelay`：Uboot启动倒计时。
    - `bootcmd`：U-Boot 倒计时结束后自动执行的命令组合。
    - `bootargs`：传递给 Linux 内核的启动参数（内核命令行 Parameters）。
- c. boot区域：调用各个文件引导并启动Linux内核，需要引入文件系统。不涉及软链接等Linux相关的文件，使用简单的FAT文件系统
  - i. `zImage`：内核镜像。
  - ii. `.dtb`：设备树
  - iii. `ramdisk.gz`：内存盘（可选）
- d. rootfs区域：挂载根文件系统，引入Linux的EXT4文件系统，读取下面配置文件，完成启动Linux完整系统。

- i. `/sbin/init`:第一个用户态程序
  - ii. `/etc/inittab`:系统初始化配置脚本, BusyBox 体系常见
  - iii. `/etc/init.d/rcS`:启动服务脚本
  - iv. `/etc/fstab`:文件系统挂载表
  - v. `/etc/profile`:用户环境变量与 Shell 配置
  - e. 其他区域, 根据实际需要决定。
2. 需要完成系统上电后, 自动加载内核和设备树, 启动内核, 内核自动挂载第二个分区作为根文件系统, 所以对uboot进行修改。
- a.uboot启动之后(启动倒计时结束), 自动执行 `bootcmd` 环境变量中的命令。 `run distro_bootcmd`

#### b.uboot相关的命令

`go`: 只是把PC指向后面的地址执行, 同时uboot放弃主动权(即一旦进入不能回退到uboot) `go 0x20008000` 直接将PC指向0x20008000, 放弃了主动权。

`bootz`: 启动内核的命令 `bootz 0x20008000 - -` 读取 `bootargs` 的环境变量, 准备内核启动前的环境设置, 将PC指向0x20008000。【`bootargs`有默认值】

`run`: 执行后面变量里的值, 变量里存储需要执行的脚本的内容 `run abc`; 执行abc里面存储的命令。

`load`: 加载文件的命令【从磁盘加载到内存中】 `load` 设备信息 (Uboot的表示方式: `mmc 0:1` =》 存储设备类型 设备号:分区号 `mmc`:多媒体卡, SD卡等存储介质)

#### c. uboot启动后的操作

- i. 设置环境变量`bootargs`, 指定root所在位置SD卡的第二个分区

```

1 # 三种启动方式
2 # 设置根设备(SD卡)、文件系统类型、启动初始化程序 — SD卡本地启动
3 setenv bootargs console=ttyS0,115200 root=/dev/mmcblk0p2 rw rootfstype=ext4
 init=/linuxrc
4
5 # 设置根设备(RAM)、文件系统类型、文件系统位置与大小、启动初始化程序 — RAM启动
6 setenv bootargs console=ttyS0,115200 root=/dev/ram rw rootfstype=ext2
 initrd=0x88000000,0x123F34 init=/linuxrc
7
8 # 设置根设备(宿主机上的NFS可挂载目录)、具体的NFS根挂载目录、网络连接协议、服务器版本、目标机
 ip:宿主机ip:网关:子网掩码:设备名: autoconf、启动初始化程序 — 网络挂载启动
9 setenv bootargs console=ttyS0,115200 root=/dev/nfs rw
 nfsroot=192.168.50.51:/home/siguyuan/EBF_6ULL/rootfs,proto=tcp,nfsvers=3
 ip=192.168.50.50:192.168.50.51::255.255.255.0::eth0:off init=/linuxrc

```

- 1 ii. 加载zImage dtb文件 从SD卡的第一个分区里读取

```

1 # 加载内核镜像
2 load mmc 0:1 0x80800000 zImage
3 # 加载设备树
4 load mmc 0:1 0x83000000 imx6ull-mmc-npi.dtb
5 # 加载文件系统
6 load mmc 0:1 0x88000000 initrd.img

```

### iii. 整合

```

1 # 使用SD卡启动, 定义设置 bootargs 的宏变量
2 setenv args_mmc_old 'setenv bootargs console=ttymxc0,115200 root=/dev/mmcblk0p2
 rw rootfstype=ext4 init=/linuxrc'
3
4 # 保存环境变量到 Flash / SD 卡(可选)
5 saveenv
6
7 # 定义从 SD 卡加载内核、设备树并启动的自定义脚本 bootcmd_sd
8 # load 将mmc 0:1 中的zImage文件加载到0x80800000的内存中
9 load mmc 0:1 0x80800000 zImage;load mmc 0:1 0x83000000 imx6ull-mmc-npi.dtb;bootz
 0x80800000 - 0x83000000;
10

```

将上面两句整合一下:

```

1 # 将 bootcmd_sd 赋值给系统默认自动启动脚本 bootcmd
2 bootcmd_sd run args_mmc_old;load mmc 0:1 0x80800000 zImage;load mmc 0:1
 0x83000000 imx6ull-mmc-npi.dtb;bootz 0x80800000 - 0x83000000;
3
4 # 将 bootcmd_sd 赋值给系统默认自动启动脚本 bootcmd
5 bootcmd run bootcmd_sd
6

```

### 3. 将上面第2小点, 写入uboot的配置文件中, 重新编译uboot

野火板子的配置文件位置: `include/configs/mx6ullfire.h`

找到对应变量【修改时, 先备份】

```

1 // 使用\0的目的是这堆字符串拼接到一起时, 方便解析。 \ 续行符 "abc" \ "123" => "abc123"
2 #define CONFIG_EXTRA_ENV_SETTINGS \
3 "bootdelay=2\0" \
4 "args_sd_boot=setenv bootargs console=ttymxc0,115200 root=/dev/mmcblk0p2 rw
 rootfstype=ext4 init=/linuxrc\0" \
5 "bootcmd_mmcx=load mmc 0:1 0x80800000 zImage; load mmc 0:1 0x83000000 imx6ull-mmc-
 npi.dtb; run args_sd_boot; bootz 0x80800000 - 0x83000000;\0" \
6 "bootcmd=run bootcmd_mmcx\0" \

```

该头文件中还有 `include/config_distro_bootcmd.h`

```

1 // 最后的位置进行修改
2 #ifndef CONFIG_BOOTCOMMAND
3 #define CONFIG_BOOTCOMMAND "run bootcmd_mmcx" // 修改成自己的启动方式
4 // #define CONFIG_BOOTCOMMAND "run distro_bootcmd"
5 #endif

```

修改完成之后，编译 `make -j$(nproc)`，拷贝到tftp共享目录下：`sudo cp u-boot-dtb.imx /opt/out_dir/EBF_6ULL/`

### ❗ Important

找该变量的流程：

1. 板上查看所有环境变量 `printenv`
2. 板上精准查找某个特定变量（如查找 `bootcmd` 或 `args`）`printenv bootcmd; printenv args_sd_boot`
3. 根据上面的结果，去宿主主机上的uboot源代码中 `include/configs/对应板子.h`，打开文件后搜索 `CONFIG_EXTRA_ENV_SETTINGS` 宏，所有默认的环境变量（如 `bootdelay`、`bootcmd` 等）都在这个宏定义里面集中声明

## 4. 开始制作整个系统镜像

### i. 分配空镜像文件

```

1 # 先创建分区目录
2 mkdir -p sd/boot sd/root
3
4 # 生成200M大小的空镜像文件，等待填充分区
5 # 生成200M的img文件，50M为boot目录，150M为rootfs，每个block大小为512B
6 dd if=/dev/zero of=./imx6ull.img bs=512 count=409600

```

### ii. 创建分区表，并进行设置

```

1 # 先对分区表分配格式：msdos格式（MX6ULL能支持），使用parted命令，找不到，就装一下命令
2 sudo parted imx6ull.img --script -- mklabel msdos

```

### iii. 进行文件系统分区

```

1 # u-boot-dtb.img的文件大小为：404480B， 一个块512B，需要790块
2 # uboot前面还需要留：
3 # 扇区 0（Sector 0，前 512 字节）：留给 MBR 分区表
4 # 扇区 2（Sector 2，物理偏移 1KB / 2 个块处）：i.MX6ULL 等芯片的 Boot ROM 规定，固件镜像（u-boot.img）必须烧写在这个固定的起始物理地址。
5 # 所以总共需要保留790 + 2 块 => 近似800块
6 # 又因为需要满足现代文件系统 1MB （2048块）物理对齐，必须让一次读取较多的数据，所以进行对齐，第一个分区（boot）开始位置为2048块，结束位置：50M => 102400个block
7 # boot分区,文件系统为fat32,起始为2048,终止为102400 + 2048 - 1;（闭区间，s表示块）
8 sudo parted imx6ull.img --script -- mkpart primary fat32 2048s 104447s
9 # rootfs分区，文件系统为ext4,起始为104448,终止为- 1(表示后续区域都是rootfs分区)
10 sudo parted imx6ull.img --script -- mkpart primary ext4 104448s -1
11
12 # 对于裸区域（uboot.img），不应该进行文件系统分区（纯二进制程序）
13 # 一般的分区只分这两个，其他分区是在操作系统完全启动后，在linux上进行分区管理

```

#### iv. 对镜像文件进行格式化

```

1 # 使用回环设备，建立虚拟磁盘映射，每个机器上的回环设备不一样，现在的回环设备为:/dev/loop0
2 sudo losetup -f --show imx6ull.img

```

`losetup` 是 Linux 中的 **Loop 环回设备（Loop Device）管理工具**。

**原理：**在 Linux 中，普通文件（如 `.img`、`.iso`）不能直接像磁盘驱动器一样按扇区读写或格式化。Loop 设备（如 `/dev/loop0`、`/dev/loop1`）充当了“转换桥梁”，它将一个普通文件模拟/映射为一个虚拟的硬件磁盘设备。

```

1 # 使用kpartx，读取镜像中的分区表并映射分区节点
2 sudo kpartx -va /dev/loop0

```

`kpartx`：专门用于解析 Loop 设备内部的分区表，并在 `/dev/mapper/` 目录下为每个分区创建独立的块设备节点。

**参数：**

`-a` (add)：添加分区映射。

`-v` (verbose)：显示详细信息。

**作用：**前面用 `parted` 在 `imx6ull.img` 里划分了 `p1`（FAT32）和 `p2`（EXT4）。`kpartx` 会读取 `/dev/loop0` 的 MBR 分区表，生成：【不执行这个命令，mapper下只有control】

`/dev/mapper/loop0p1`（对应第一分区，即 Boot 分区）

`/dev/mapper/loop0p2`（对应第二分区，即 Rootfs 分区）

```

1 # 格式化两个分区
2 # 格式成fat32形式
3 sudo mkfs.vfat /dev/mapper/loop0p1
4 # 格式化成ext4形式
5 sudo mkfs.ext4 /dev/mapper/loop0p2
6
7 # 如果找不到命令, 安装sudo apt install e2fsprogs 和 sudo apt install dosfstools

```

#### v. 对分区进行挂载

```

1 # 将目录挂载
2 sudo mount /dev/mapper/loop0p1 ./sd/boot/
3 sudo mount /dev/mapper/loop0p2 ./sd/root/
4
5 # 将uboot烧写进去
6 sudo dd if=/opt/out_dir/EBF_6ULL/u-boot-dtb.img of=/dev/loop0 bs=512 seek=2 conv=fsync
7
8 # 将boot、ramdisk.gz、DTB拷贝进去
9 sudo cp /opt/out_dir/EBF_6ULL/ebf_rd.img.gz sd/boot/
10 sudo cp /opt/out_dir/EBF_6ULL/zImage sd/boot/
11 sudo cp /opt/out_dir/EBF_6ULL/imx6ull-mmc-npi.dtb sd/boot/
12
13 # 将rootfs拷贝进去
14 sudo cp -a ~/EBF_6ULL/rootfs/* sd/root/

```

#### vi. 卸载

```

1 sudo umount ./sd/boot/ ./sd/root/
2
3 sudo kpartx -d /dev/loop0
4 sudo losetup -d /dev/loop0

```

#### vii. 压缩

```

1 xz -zf ./imx6ull.img

```

5. 将linux中的压缩包 `imx6ull.img.xz` 下载下来, 在windows下使用 `BalenaEtcher` 进行下载 (速度快一些)。上电之后, 不需要按任何键, 等倒计时结束

#### ⚠ Caution

使用 **BalenaEtcher** 烧写镜像时, 必须格外注意设备选择, 以防误删重要数据:

- **慎选目标设备:** 如果电脑插有其他外接硬盘、移动固态或 U 盘, 烧录前**强烈建议将所有不相关的外接存储设备直接拔掉**。
- **物理级覆写风险:** BalenaEtcher 采用的是**逐位 (Bit-by-Bit) 底层镜像覆写**机制。一旦选错盘符并启动烧录, 目标设备上的所有分区与数据将被瞬间彻底覆写, **数据几乎无法恢复**。

为避免“误杀”其他硬盘, 建议按以下标准化顺序操作:

1. **清理环境:** 拔掉除目标 SD 卡/读卡器之外的所有无关外接存储。



2. **确认盘符/容量**: 在 balenaEtcher 的 `Select target` 界面中, **仔细核对设备容量** (例如: 16GB SD 卡在软件中显示约 `14.8 GB` 或 `15.x GB`) 。
3. **开始烧写**: 点击 `Flash!` 并等待校验 (Verify) 完成。
4. **安全移除**: 烧录完成后, 先在操作系统中“安全弹出”设备, 再拔出 SD 卡。

6. 在宿主机上, 创建一个 `shared` 目录, 利用 NFS 进行共享, 方便之后传输文件

```
1 mkdir shared
2
3 sudo nvim /etc/exports
4 # 添加
5 /home/siguyuan/EBF_6ULL/shared *(rw,sync,no_subtree_check,no_root_squash)
6 # 重启服务
7 sudo service nfs-kernel-server restart
```

7. 将 NFS 共享目录和 boot 分区挂载在 /opt 下面, 方便之后更新

```
1 # 板子上
2 ifconfig eth0 192.168.50.50 netmask 255.255.255.0
3
4 # 挂载 NFS 目录
5 mount -t nfs -o nolock,nfsvers=3 192.168.50.51:/home/siguyuan/EBF_6ULL/shared /opt
6
7 # 挂载 boot 分区, 方便之后更新设备树
8 mount -t vfat /dev/mmcblk0p1 /mnt
9
10 # 重启, 同时将数据保存到 SD 卡
11 reboot
```

8. 配置一个启动脚本

```
1 # net.sh
2 #!/bin/sh
3
4 # 1. 配置网卡 IP
5 ifconfig eth0 192.168.50.50 netmask 255.255.255.0
6
7 # 2. 等待网络 Link Up
8 echo "Waiting for eth0 link up..."
9 cnt=0
10 while [$cnt -lt 10]; do
11 if ping -c 1 -W 1 192.168.50.51 >/dev/null 2>&1; then
12 echo "Network is up and ready!"
13 break
14 fi
15 sleep 1
16 cnt=$((cnt+1))
17 done
18
```

```

19 # 3. 挂载 Boot 分区（如果未挂载才进行修复与挂载）
20 if ! mountpoint -q /mnt; then
21 dosfsck -a /dev/mmcb1k0p1 >/dev/null 2>&1
22 mount -t vfat /dev/mmcb1k0p1 /mnt
23 else
24 echo "/mnt is already mounted."
25 fi
26
27 # 4. 挂载 NFS 目录（如果未挂载才挂载）
28 if ! mountpoint -q /opt; then
29 mount -t nfs -o nolock,nfsvers=3 192.168.50.51:/home/siguyuan/EBF_6ULL/shared /opt
30 else
31 echo "/opt is already mounted."
32 fi

```

一键脚本：

```

1 #!/bin/bash
2 # 安装格式化工具包，当前目录包含u-boot-dtb.img、zImage、imx6ull-mmc-npi.dtb
3 # 生成测试好的文件系统，打包命名为rootfs.tar
4 # sudo apt-get install dosfstools dump parted kpartx
5 BASE_DIR=img
6 BOOT_DIR=img/boot
7 ROOT_DIR=img/root
8 # 镜像文件名
9 IMG_NAME=E390.img
10 # 要烧写的uboot镜像文件名
11 UBOOT_NAME=/opt/out/u-boot-dtb.imgx
12 # 要烧写的boot的路径
13 IMG_BOOT_DIR=/opt/out
14 OSL_IMG_NAME=zImage
15 DTB_IMG_NAME=imx6ull-mmc-npi.dtb
16 ROOT_FS_NAME=ebf_rd.img.gz
17 # 要烧写rootfs的路径
18 IMG_ROOT_DIR=~/.ebf/rootfs
19
20 mkdir -p ${BOOT_DIR} ${ROOT_DIR}
21 echo "正在创建文件,需要一定时间,请勿中断"
22 # 生成200M的img文件，50M为boot目录，150M为rootfs
23 dd if=/dev/zero of=${IMG_NAME} bs=512 count=409600
24 # 在imx6ull.img文件上创建一个msdos分区表
25 sudo parted ${IMG_NAME} --script -- mklabel msdos
26 # 其中uboot.img文件大小是400384，可以使用前(782+2)个block，那么偏移800个block就可以
27 # 考虑到文件系统对齐，选择偏移2048个block，第一个分区50M = 102400个block
28 sudo parted ${IMG_NAME} --script -- mkpart primary fat32 2048s 104447s
29 # 第二个分区是存放根文件系统，可以读写相关内容，一直到最后一个分区
30 sudo parted ${IMG_NAME} --script -- mkpart primary ext4 104448s -1
31 # 在Linux系统中找到一个可用的回环设备，并将其与img文件进行关联
32 LOOPX=$(sudo losetup -f --show ${IMG_NAME})

```

```

33 # 在回环设备中创建并激活分区映射
34 sudo kpartx -va $LOOPX
35 # 从字符串变量 ${LOOPX} 中提取最后一个字符, 并将提取的字符赋值给变量 NUMB
36 NUMB=$(echo "$LOOPX" | grep -o '[0-9]\+$')
37 [-z $NUMB] && exit
38 echo "NUMB=${NUMB}"
39
40 # 在分区1进行格式化, 格式化为vfat格式
41 sudo mkfs.vfat /dev/mapper/loop${NUMB}p1
42 # 在分区2进行格式化, 格式化为ext4格式
43 sudo mkfs.ext4 /dev/mapper/loop${NUMB}p2
44 # 将已经格式化好的分区, 挂载在系统上, 便于写入数据
45 sudo mount /dev/mapper/loop${NUMB}p1 ${BOOT_DIR}
46 sudo mount /dev/mapper/loop${NUMB}p2 ${ROOT_DIR}
47
48 # 将uboot镜像烧写到SD卡未分区的块内, 根据要求, 偏移量为2
49 sudo dd if=${UBOOT_NAME} of=${LOOPX} bs=512 seek=2 conv=fsync
50
51 sudo cp ${IMG_BOOT_DIR}/${OSL_IMG_NAME} ${BOOT_DIR} # 将生成的内核镜像到第一个BOOT分区
52 sudo cp ${IMG_BOOT_DIR}/${DTB_IMG_NAME} ${BOOT_DIR} # 将生成的内核设备树拷贝到第一个
BOOT分区
53 sudo cp ${IMG_BOOT_DIR}/${ROOT_FS_NAME} ${BOOT_DIR} # 将Ramdisk镜像拷贝到第一个BOOT分
区
54 sudo cp -a ${IMG_ROOT_DIR}/* ${ROOT_DIR} # 拷贝根文件系统到第二个ROOT分区
55
56 # 卸载挂载目录
57 sudo umount ${BOOT_DIR} ${ROOT_DIR}
58
59 sudo kpartx -d ${LOOPX}
60 sudo losetup -d ${LOOPX}
61
62 echo "正在打包,需要一定时间,请勿中断"
63 xz -zf ${IMG_NAME}
64 rm -rf ${BOOT_DIR} ${ROOT_DIR} ${BASE_DIR}
65

```

## 多线程库如何在板子上运行

虚拟机中创建一个测试文件:

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 void thread01(){

```

```

5 int i = 100;
6 while (i) {
7 printf("thread01: %d\n", i);
8 --i;
9 }
10 }
11
12
13 void thread02(){
14 int i = 100;
15 while(i){
16 printf("thread02:%d\n", i);
17 --i;
18 }
19 }
20
21 int main(){
22 // 创建线程
23 pthread_t thread1, thread2;
24 int ret = pthread_create(&thread1, NULL, (void (*)(void *))thread01, NULL);
25 if(ret){
26 perror("thread1");
27 return -1;
28 }
29
30 ret = pthread_create(&thread2, NULL, (void (*)(void *))thread02, NULL);
31 if(ret){
32 perror("thread2");
33 pthread_cancel(thread1);
34 pthread_join(thread1, NULL);
35 return -1;
36 }
37
38 pthread_join(thread1, NULL);
39 pthread_join(thread2, NULL);
40 return 0;
41 }
42

```

使用交叉编译器编译: `arm-none-linux-gnueabi-gcc -o build thread01.c -lpthread`

检查可执行文件:

```

1 # 发现确实是arm32位下能执行
2 siguyuan@debian:~/EBF_6ULL/test/thread_test$ file build
3 build: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked,
 interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, with debug_info, not
 stripped
4
5 # 检查动态链接符号表 发现需要libpthread.so.0、libc.so.6
6 siguyuan@debian:~/EBF_6ULL/test/thread_test$ arm-none-linux-gnueabi-readelf -d build
7
8 Dynamic section at offset 0xf08 contains 26 entries:

```

|    | Tag                                                                                                              | Type           | Name/Value                        |
|----|------------------------------------------------------------------------------------------------------------------|----------------|-----------------------------------|
| 9  |                                                                                                                  |                |                                   |
| 10 | 0x00000001                                                                                                       | (NEEDED)       | Shared library: [libpthread.so.0] |
| 11 | 0x00000001                                                                                                       | (NEEDED)       | Shared library: [libc.so.6]       |
| 12 | 0x0000000c                                                                                                       | (INIT)         | 0x1041c                           |
| 13 | 0x0000000d                                                                                                       | (FINI)         | 0x10690                           |
| 14 | 0x00000019                                                                                                       | (INIT_ARRAY)   | 0x20f00                           |
| 15 | 0x0000001b                                                                                                       | (INIT_ARRAYSZ) | 4 (bytes)                         |
| 16 | 0x0000001a                                                                                                       | (FINI_ARRAY)   | 0x20f04                           |
| 17 | 0x0000001c                                                                                                       | (FINI_ARRAYSZ) | 4 (bytes)                         |
| 18 | 0x00000004                                                                                                       | (HASH)         | 0x10190                           |
| 19 | 0x6ffffef5                                                                                                       | (GNU_HASH)     | 0x101d0                           |
| 20 | 0x00000005                                                                                                       | (STRTAB)       | 0x102c0                           |
| 21 | 0x00000006                                                                                                       | (SYMTAB)       | 0x10210                           |
| 22 | 0x0000000a                                                                                                       | (STRSZ)        | 187 (bytes)                       |
| 23 | 0x0000000b                                                                                                       | (SYMENT)       | 16 (bytes)                        |
| 24 | 0x00000015                                                                                                       | (DEBUG)        | 0x0                               |
| 25 | 0x00000003                                                                                                       | (PLTGOT)       | 0x21000                           |
| 26 | 0x00000002                                                                                                       | (PLTRELSZ)     | 64 (bytes)                        |
| 27 | 0x00000014                                                                                                       | (PLTREL)       | REL                               |
| 28 | 0x00000017                                                                                                       | (JMPREL)       | 0x103dc                           |
| 29 | 0x00000011                                                                                                       | (REL)          | 0x103d4                           |
| 30 | 0x00000012                                                                                                       | (RELSZ)        | 8 (bytes)                         |
| 31 | 0x00000013                                                                                                       | (RELENT)       | 8 (bytes)                         |
| 32 | 0x6fffffff                                                                                                       | (VERNEED)      | 0x10394                           |
| 33 | 0x6fffffff                                                                                                       | (VERNEEDNUM)   | 2                                 |
| 34 | 0x6fffffff0                                                                                                      | (VERSYM)       | 0x1037c                           |
| 35 | 0x00000000                                                                                                       | (NULL)         | 0x0                               |
| 36 |                                                                                                                  |                |                                   |
| 37 | # 检查rootfs下的lib有没有对应的库，发现确实线程库                                                                                   |                |                                   |
| 38 | siguyuan@debian:~/EBF_6ULL/rootfs\$ ls lib/                                                                      |                |                                   |
| 39 | ld-2.30.so ld-linux-armhf.so.3 libc-2.30.so libc.so.6 libm-2.30.so libm.so.6<br>libresolv-2.30.so libresolv.so.2 |                |                                   |
| 40 |                                                                                                                  |                |                                   |
| 41 | # 将可执行文件拷贝到rootfs/home下                                                                                          |                |                                   |
| 42 | siguyuan@debian:~/EBF_6ULL/rootfs\$ cp ../test/thread_test/build home/                                           |                |                                   |
| 43 | siguyuan@debian:~/EBF_6ULL/rootfs\$ ls /home/                                                                    |                |                                   |
| 44 | siguyuan                                                                                                         |                |                                   |
| 45 |                                                                                                                  |                |                                   |

```

1 # 为了方便，使用网络方式作为根文件系统
2 # 板子上电
3 # 设置tftp服务端IP
4 setenv serverip 192.168.50.51
5
6 # 配置变量,下载，启动
7 setenv bootargs console=ttymx0,115200 root=/dev/nfs rw
 nfsroot=192.168.50.51:/home/siguyuan/EBF_6ULL/rootfs,proto=tcp,nfsvers=3
 ip=192.168.50.50:192.168.50.51::255.255.255.0::eth0:off init=/linuxrc;tftp 0x80800000
 zImage;tftp 0x83000000 imx6ull-mmc-npi.dtb;bootz 0x80800000 - 0x83000000
8

```

## ⚠ Caution

如果使用arm或者flash，因为文件系统提取打包好了，所以宿主新添加的文件，需要重新打包，才能在板子上找到

在板子上运行如果不添加线程库的现象：

```
1 # 板子运行错误现象
2 # 找不线程库
3 [root@localhost ~]# root/build
4 root/build: error while loading shared libraries: libpthread.so.0: cannot open shared
 object file: No such file or directory
5
6 # 虚拟机中
7 # 解决方式，添加线程动态库
8 siguyuan@debian:/opt/arm-linux-gnueabi/hf/arm-none-linux-gnueabi/hf/libc/lib$ ls
9 ld-2.30.so libcrypt-2.30.so libm.so.6 libnss_dns-2.30.so
 libpthread-2.30.so libthread_db-1.0.so
10 ld-linux-armhf.so.3 libc.so.1 libnsl-2.30.so libnss_dns.so.2
 libpthread.so.0 libthread_db.so.1
11 libanl-2.30.so libc.so.6 libnsl.so.1 libnss_files-2.30.so
 libresolv-2.30.so libutil-2.30.so
12 libanl.so.1 libdl-2.30.so libnss_compat-2.30.so libnss_files.so.2
 libresolv.so.2 libutil.so.1
13 libBrokenLocale-2.30.so libdl.so.2 libnss_compat.so.2 libnss_hesiod-
 2.30.so librt-2.30.so
14 libBrokenLocale.so.1 libm-2.30.so libnss_db-2.30.so libnss_hesiod.so.2
 librt.so.1
15 libc-2.30.so libmemusage.so libnss_db.so.2 libpcprofile.so
 libSegFault.so
16 # 拷贝库
17 siguyuan@debian:/opt/arm-linux-gnueabi/hf/arm-none-linux-gnueabi/hf/libc/lib$ cp -a
 libpth* /home/siguyuan/EBF_6ULL/rootfs/lib/
18 siguyuan@debian:/opt/arm-linux-gnueabi/hf/arm-none-linux-gnueabi/hf/libc/lib$ cd
 /home/siguyuan/EBF_6ULL/rootfs/lib/
19 # 检查是否拷贝完成
20 siguyuan@debian:~/EBF_6ULL/rootfs/lib$ ls
21 ld-2.30.so libc-2.30.so libm-2.30.so libpthread-2.30.so libresolv-2.30.so
22 ld-linux-armhf.so.3 libc.so.6 libm.so.6 libpthread.so.0 libresolv.so.2
23
24 # 板子上检查、运行
25 [root@localhost ~]# ls lib/
26 ld-2.30.so libm-2.30.so libresolv-2.30.so
27 ld-linux-armhf.so.3 libm.so.6 libresolv.so.2
28 libc-2.30.so libpthread-2.30.so
29 libc.so.6 libpthread.so.0
30
31 [root@localhost ~]# root/build
```

如果是厂家没有提供的库，使用交叉编译器编译对应的源码之后，将动态库拷贝到rootfs/lib下

### 动态库【面试题】：

1. 编译链接时，只要宿主机含有这个库，只需要在gcc过程中，通过-L指定库的查找路径，通过-l库名告诉链接器链接的库名
2. 运行时，目标机也应该有这个库，否则是运行不了的

解决方案：

1. 运行时，默认查找/lib目录
2. 通过设置环境变量LD\_LIBRARY\_PATH指定目标机在哪个目录下含有这个库文件

场景一：只读根文件系统 (RAMDisk) + 可写 Flash 路径 (/opt)

根文件系统为 RAMDisk 且只读（不可修改），仅 /opt/ 挂载了外部可读写 Flash。在此环境下部署第三方库。

方案 A：静态链接或编译期头文件指定（针对编译期）

- a. 头文件引入：编译时使用 `-I/opt/mylib/include` 指定头文件路径。
- b. 库文件链接：编译时使用 `-L/opt/mylib/lib -lxxx` 指定库路径与库名。
- c. 注：此方案仅解决编译通过问题，运行期仍需配好库路径（`LD_LIBRARY_PATH=库路径`）或使用 RPATH。

方案 B：运行期环境变量配置（修改启动脚本）

a. 由于 `/etc/profile` 或 `~/.bashrc` 位于只读的 RAMDisk 中无法修改，可以在 `/opt/` 下自定义初始化脚本（如 `/opt/env.sh`），并在开机自动运行的某些可写入口脚本（如开机挂载脚本）中用 `source` 加载：`export LD_LIBRARY_PATH=/opt/mylib/lib:$LD_LIBRARY_PATH`

方案 C：编译期写入硬编码运行路径（RPATH - 最推荐）

a. 编译链接时直接将运行期的库路径写入二进制文件的 ELF Header 中，彻底脱离环境变量依赖：`arm-linux-gnueabi-gcc main.c -L/opt/mylib/lib -lxxx -wl,-rpath=/opt/mylib/lib -o app`

场景二：开发板通过 NFS 挂载宿主机目录调试

在开发阶段，开发板通过 NFS 将宿主机的共享目录挂载到 `/opt`，用于频繁编译和调试程序

- a. 将交叉编译好的动态库 `.so` 文件直接放在宿主机的 NFS 共享目录中（例如 `/home/siguyuan/EBF_6ULL/shared/lib`）
- b. 开发板挂载到 `/opt` 后，库文件直接映射在 `/opt/lib`。
- c. 配合 RPATH (`-wl,-rpath=/opt/lib`) 或临时指定环境变量运行：`LD_LIBRARY_PATH=/opt/lib ./app`

场景四：程序显式动态加载（运行时代码加载 .so）

程序不希望在编译期直接链接库，而是在代码运行过程中根据需要**显式/动态加载**动态库（如实现插件化架构）。

a. **标准 C 库 API** (`<dldfcn.h>`)：

- 1、`dlopen()`：打开并加载指定的 `.so` 动态库文件。
- 2、`dlsym()`：从已加载的动态库中获取函数指针或全局变量符号。
- 3、`dLError()`：捕获并返回加载/查找过程中的错误信息。
- 4、`dldclose()`：卸载动态库并释放资源。

b. **编译注意**：使用 `dlopen` 系列函数时，编译需要链接 `dld` 库 (`-ldld`)：`arm-linux-gnueabi-hf-gcc`

`main.c -ldld -o app`

```
1 // 例子：加法函数
2 #include <stdio.h>
3 #include <dldfcn.h> // 1. 必须包含此头文件
4
5 int main() {
6 // 定义一个函数指针类型，用来接收一会儿找到的 add 函数
7 int (*my_add_func)(int, int);
8
9 // 2. 【dlopen】运行时去 /opt 目录下打开库文件
10 void *handle = dlopen("/opt/libcalc.so", RTLD_LAZY);
11 if (!handle) {
12 printf("打开库失败: %s\n", dLError());
13 return -1;
14 }
15
16 // 3. 【dlsym】在库里寻找名字叫 "add" 的函数，并拿到它的地址
17 my_add_func = dlsym(handle, "add");
18 if (!my_add_func) {
19 printf("找不到 add 函数: %s\n", dLError());
20 dldclose(handle);
21 return -1;
22 }
23
24 // 4. 通过拿到的函数指针直接调用函数
25 int result = my_add_func(10, 20);
26 printf("10 + 20 = %d\n", result);
27
28 // 5. 【dldclose】用完卸载动态库
29 dldclose(handle);
30 return 0;
31 }
```



## 3.5 给内核添加驱动

### 3.5.1 基本概念

主要熟悉内核的Makefile以及Kconfig配置文件（使用的是Kbuild语法 —— 在内核 `linux_4.19.35_imx6ull/Documentation/kbuild` 目录下有说明文档）

架构：

```
1 | . (内核根目录)
2 | ├── kconfig <-- 顶层配置入口（定义所有子菜单）
3 | ├── Makefile <-- 顶层 Makefile（定义主编译规则与 ARCH/CROSS_COMPILE）
4 | |
5 | ├── arch/ <-- 架构相关代码
6 | | └── arm/
7 | | ├── kconfig <-- ARM 体系架构配置项
8 | | ├── Makefile <-- ARM 平台专有编译规则
9 | | ├── mach-imx/ <-- i.MX 系列板级代码
10 | | | ├── kconfig <-- 定义 CONFIG_SOC_IMX6ULL 等选项
11 | | | └── Makefile
12 | | └── boot/
13 | | └── dts/
14 | | └── Makefile <-- 控制设备树 (.dtb) 编译的目标列表
15 | |
16 | ├── drivers/ <-- 设备驱动程序目录
17 | | ├── kconfig <-- 引入各子驱动目录的 kconfig
18 | | ├── Makefile <-- 递归进入各驱动子目录 (obj-y += gpio/)
19 | | ├── gpio/
20 | | | ├── kconfig
21 | | | └── Makefile
22 | | └── net/
23 | | ├── kconfig
24 | | └── Makefile
25 | |
26 | ├── fs/ <-- 文件系统 (ext4, fat 等)
27 | | ├── kconfig
28 | | └── Makefile
29 | |
30 | ├── init/ <-- 内核初始化入口 (main.c)
31 | | ├── kconfig
32 | | └── Makefile
33 | |
34 | └── kernel/ <-- 进程调度、中断等核心代码
35 | ├── kconfig
36 | └── Makefile
```

主目录中：

**Makefile**：编译规则的指定，汇总其他各个目录的Makefile下的obj的变量值。编译、链接、`objcopy` 获得内核镜像等规则。

```

2 ARCH ?= $(SUBARCH)
3 ...
4
5 # 读取make menuconfig生成的.config配置清单
6 KCONFIG_CONFIG ?= .config
7 export KCONFIG_CONFIG
8 ...
9
10 # 各种工具的定义
11 AS = $(CROSS_COMPILE)as
12 LD = $(CROSS_COMPILE)ld
13 CC = $(CROSS_COMPILE)gcc
14 CPP = $(CC) -E
15 AR = $(CROSS_COMPILE)ar
16 NM = $(CROSS_COMPILE)nm
17 STRIP = $(CROSS_COMPILE)strip
18 OBJCOPY = $(CROSS_COMPILE)objcopy
19 OBJDUMP = $(CROSS_COMPILE)objdump
20 ...
21
22
23

```

**kconfig**：汇总所有子目录下的配置菜单，最终在本文件编译给 **make menuconfig** 显示。

```

1 mainmenu "Linux/$(ARCH) $(KERNELVERSION) kernel Configuration"
2
3 comment "Compiler: $(CC_VERSION_TEXT)"
4
5 # 类似头文件，编译时展开
6 source "scripts/kconfig.include"
7
8 source "init/kconfig"
9
10 source "kernel/kconfig.freezer"
11
12 source "fs/kconfig.binfmt"
13
14 source "mm/kconfig"
15
16 source "net/kconfig"
17
18 source "drivers/kconfig"
19
20 source "fs/kconfig"
21
22 source "security/kconfig"
23
24 source "crypto/kconfig"
25
26 source "lib/kconfig"
27

```

子目录中:

**Makefile**: 确定哪些文件是否编译, 怎么编译。

```

1 obj-y += mem.o random.o # 使用追加形式, 不能改变前面的obj-y /
 obj-m的配置值
2 obj-$(CONFIG_TTY_PRINTK) += ttyprintk.o
3 obj-y += misc.o
4 obj-$(CONFIG_ATARI_DSP56K) += dsp56k.o
5 obj-$(CONFIG_VIRTIO_CONSOLE) += virtio_console.o
6 obj-$(CONFIG_RAW_DRIVER) += raw.o
7 obj-$(CONFIG_SGI_SNSC) += snsc.o snsc_event.o
8 obj-$(CONFIG_MSPEC) += mspec.o
9 obj-$(CONFIG_UV_MMTIMER) += uv_mmtimer.o
10 obj-$(CONFIG_IBM_BSR) += bsr.o
11

```

- `obj-y`: 编译进内核
- `obj-m`: 编译成内核模块 (按需调用), 但不写进内核
- `obj-`: 不编译

-  **Caution**  
makefile中 `$()` 表示取变量值, 这个变量可以是makefile自己定义的, 也可以是shell定义的  
shell脚本中 `${}` 表示取变量值, `$()` 表示执行一个子shell

**kconfig**: 具体的菜单的选项。

```

1 # 下面的内容并非来自同一个文件
2 menu "Character devices"
3 ...
4 endmenu
5
6 source "drivers/tty/kconfig"
7
8 config DEVMEM
9 bool "/dev/mem virtual device support"
10 default y
11 help
12 Say Y here if you want to support the /dev/mem device.
13 The /dev/mem device is used to access areas of physical
14 memory.
15 When in doubt, say "Y".
16
17 config SGI_MBCS
18 tristate "SGI FPGA Core Services driver support"
19 depends on SGI_TIOCX
20 help
21 If you have an SGI Altix with an attached SABrick
22 say Y or M here, otherwise say N.

```

```

23
24 menuconfig GPIOLIB
25 bool "GPIO Support"
26 select ANON_INODES
27 help
28 This enables GPIO support through the generic GPIO library.
29 You only need to enable this, if you also want to enable
30 one or more of the GPIO drivers below.
31
32 If unsure, say N.
33

```

- `menu` 与 `endmenu`：菜单内容。

- ```

1 | menu "菜单显示的标题"
2 |     菜单的选项
3 | endmenu

```

-

- `choice` 与 `endchoice`：单选菜单

- ```

1 | choice
2 | prompt "这里写单选菜单的提示标题"
3 | default MY_OPTION_A # 默认选中的项
4 | help
5 | 这里可以写对这个单选菜单的总体说明。
6 |
7 | config MY_OPTION_A
8 | bool "选项 A 的描述"
9 |
10 | config MY_OPTION_B
11 | bool "选项 B 的描述"
12 |
13 | config MY_OPTION_C
14 | bool "选项 C 的描述"
15 |
16 | endchoice

```

- `source "文件路径"`：文件包含，类似C语言的include，会将该文件全部插入到引入处。
  - 该文件路径，必须基于主Kconfig位置来规定。因为最后都是汇总到主Kconfig编译。
- 所有带 `config` 的关键字：都是定义一个新的**配置变量节点**的开头（变量生成器）。在这个关键字下面，定义 `bool`、`tristate`、`depends on` 等（这些都是需要这个变量的）。如果选中，会在最后的 `.config` 配置清单中生成 `CONFIG_变量名 = y`，提供给对于的 Makefile 文件编译
- `tristate "显示在make menuconfig 让用户看信息"`：三态选择。
  - `y` (Yes / Built-in)：编译进内核（界面显示为 `<*>`）。
  - `m` (Module)：编译为独立的内核模块（界面显示为 `<M>`）。
  - `n` (No / Off)：不编译（界面显示为 `<>`）。
- `bool "显示在make menuconfig 让用户看信息"`：两态选择。

- **y** (Yes / Built-in): 编译进内核 (界面显示为 `<*>`) 。
- **n** (No / Off): 不编译 (界面显示为 `<>`) 。
- **depends on 变量值**: 如果该变量值没有被选中, 则当前配置项在 `make menuconfig` 界面中会**直接隐藏 (或变灰不可选)**。类似C语言的 `#if xx`
- **select 变量名**: **强制 (自动) 选中目标变量**

`.config` 文件的产生:

- 通过参考配置, 激活。 `make npiv7_defconfig` 通过厂家的参考配置清单生成 `.config`
- 通过 `make menuconfig` 图形化界面进行配置 (交互式修改), 生成 `.config`

### 3.5.2 添加驱动流程

1. 在 `drivers` 根据下面两种情况创建驱动文件或者驱动目录:

- **只写一个简单文件** → **不建目录**, 直接放进已有的同类驱动文件夹下 (修改原文件夹里的 `Kconfig` 和 `Makefile`)
- **功能独立 / 多个源文件 / 板级特定模块** → **建议新建目录** (拥有独立的 `Kconfig` 和 `Makefile`) 【本次选择 `drivers/char/my_free`】

2. 一般先使用官方驱动程序将整个顺序调通, 然后在自己编写。使用野火提供的LED驱动 `led_cdev.c` 和测试程序 `led_test.c`, 将驱动程序拷贝到 `drivers/my_fire/` 目录下。

- ```
1 | siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers$ ls
2 | my_fire/
3 | led_cdev.c
```

◦

3. 打通 `Makefile`, 形成编译链。【如果偷懒, 可以直接使用 `obj-y`, 不定义变量】

- 对于 `my_fire/Makefile`
- ```
1 | # 自己定义变量名 链接生成的.o文件
2 | # 变量名, 必须为CONFIG_xxx, 因为这是Kconfig的生成规则
3 | obj-$(CONFIG_MY_LED) += led_cdev.o
```

- 对于 `drivers/Makefile`
- ```
1 | # 随便找一处添加
2 | # 将自己的驱动放入drivers中
3 | obj-$(CONFIG_MY_FIRE) += my_fire/
```

4. 打通 `Kconfig`, 形成配置选项。

- 对于 `my_fire/kconfig`

```

1  # 这里的名字必须和CONFIG_后面一样
2  menuconfig MY_FIRE
3      bool "EmbedFire Drivers Configs"
4      help
5          EmbedFire board custom driver configuration.
6
7  # 这里的名字必须和CONFIG_后面一样
8  config MY_LED
9      tristate "LED Drivers Config"
10     depends on MY_FIRE
11     help
12         my led driver.

```

○ 对于 `drivers/kconfig`

```

1  # 随便找一个位置添加，但必须在menu 和 endmenu之间
2  # 告知自己的驱动目录
3  source "drivers/my_fire/kconfig"

```

5. 使用 `make menuconfig`，进行配置。在“Device Drivers”下能找到“EmbedFire Drivers Configs”，并且能成功选中，即为写入成功。【记得，切换成交叉编译器】
6. 检查主目录下的 `.config` 文件中是否有自己定义的变量
7. 进行 `make zImage -j$(nproc)`，进行编译，编译之后，生成设备树 `make imx6ull-mmc-npi.dtb`，之后将 `zImage`、`dtb` 文件拷贝到 NFS 共享目录中
8. (可选)，添加脚本，方便后面修改

```

1  #!/bin/bash
2
3  # 将编译的内核镜像压缩文件，拷贝到共享目录中
4  sudo cp
5      /home/siguyuan/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/boot/zImage
6      /opt/out_dir/EBF_6ULL/
7  sudo cp
8      /home/siguyuan/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/boot/zImage
9      ~/EBF_6ULL/shared/img/
10
11 # 编辑设备树
12 make imx6ull-mmc-npi.dtb
13
14 # 将设备树的镜像文件拷贝到共享目录
15 sudo cp
16     /home/siguyuan/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/boot/dts/imx6ull-
17     mmc-npi.dtb /opt/out_dir/EBF_6ULL/
18 sudo cp
19     /home/siguyuan/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/boot/dts/imx6ull-
20     mmc-npi.dtb ~/EBF_6ULL/shared/img/

```

9. 将内核镜像文件拷贝到 boot 区（之前将 boot 区挂载到了 `/mnt` 目录下）`cp /opt/img/zImage /mnt/`，重启 `reboot`

10. 验证

1. `cat /proc/devices`：查看里面有没有 `led_chrdev` 驱动的存在。（根据驱动源代码，得来的验证方式）
2. `ls /dev`：有没有驱动创建。（根据驱动源代码，得来的验证方式）
3. 使用官方测试函数，进行测试，看现象

11. 官方驱动文件（优化差） + 测试文件

```
1  #include <linux/init.h>
2  #include <linux/module.h>
3  #include <linux/cdev.h>
4  #include <linux/fs.h>
5  #include <linux/uaccess.h>
6  #include <linux/io.h>
7
8  #define DEV_NAME          "led_chrdev"
9  #define DEV_CNT            (3)
10
11  static dev_t devno;
12  struct class *led_chrdev_class;
13
14  struct led_chrdev {
15      struct cdev dev;
16      unsigned int __iomem *va_dr;
17      unsigned int __iomem *va_gdir;
18      unsigned int __iomem *va_iomuxc_mux;
19      unsigned int __iomem *va_ccm_ccgrx;
20      unsigned int __iomem *va_iomux_pad;
21
22      unsigned long pa_dr;
23      unsigned long pa_gdir;
24      unsigned long pa_iomuxc_mux;
25      unsigned long pa_ccm_ccgrx;
26      unsigned long pa_iomux_pad;
27
28      unsigned int led_pin;
29      unsigned int clock_offset;
30  };
31
32  static int led_chrdev_open(struct inode *inode, struct file *filp)
33  {
34      unsigned int val = 0;
35      struct led_chrdev *led_cdev =
36          (struct led_chrdev *)container_of(inode->i_cdev, struct led_chrdev,
37          dev);
38      filp->private_data =
39          container_of(inode->i_cdev, struct led_chrdev, dev);
40
41      printk("open\n");
42
43      led_cdev->va_dr = ioremap(led_cdev->pa_dr, 4);
44      led_cdev->va_gdir = ioremap(led_cdev->pa_gdir, 4);
```

```

45     led_cdev->va_iomuxc_mux = ioremap(led_cdev->pa_iomuxc_mux, 4);
46     led_cdev->va_ccm_ccgrx = ioremap(led_cdev->pa_ccm_ccgrx, 4);
47     led_cdev->va_iomux_pad = ioremap(led_cdev->pa_iomux_pad, 4);
48
49     val = ioread32(led_cdev->va_ccm_ccgrx);
50     val &= ~(3 << led_cdev->clock_offset);
51     val |= (3 << led_cdev->clock_offset);
52     iowrite32(val, led_cdev->va_ccm_ccgrx);
53
54     iowrite32(5, led_cdev->va_iomuxc_mux);
55
56     iowrite32(0x1F838, led_cdev->va_iomux_pad);
57
58     val = ioread32(led_cdev->va_gdir);
59     val &= ~(1 << led_cdev->led_pin);
60     val |= (1 << led_cdev->led_pin);
61     iowrite32(val, led_cdev->va_gdir);
62
63     val = ioread32(led_cdev->va_dr);
64     val |= (0x01 << led_cdev->led_pin);
65     iowrite32(val, led_cdev->va_dr);
66
67     return 0;
68 }
69
70 static int led_chrdev_release(struct inode *inode, struct file *filp)
71 {
72     struct led_chrdev *led_cdev =
73         (struct led_chrdev *)container_of(inode->i_cdev, struct led_chrdev,
74         dev);
75     iounmap(led_cdev->va_dr);
76     iounmap(led_cdev->va_gdir);
77     iounmap(led_cdev->va_iomuxc_mux);
78     iounmap(led_cdev->va_ccm_ccgrx);
79     iounmap(led_cdev->va_iomux_pad);
80     return 0;
81 }
82
83 static ssize_t led_chrdev_write(struct file *filp, const char __user * buf,
84     size_t count, loff_t * ppos)
85 {
86     unsigned long val = 0;
87     unsigned long ret = 0;
88
89     int tmp = count;
90
91     kstrtoul_from_user(buf, tmp, 10, &ret);
92     struct led_chrdev *led_cdev = (struct led_chrdev *)filp->private_data;
93
94     val = ioread32(led_cdev->va_dr);
95     if (ret == 0)
96         val &= ~(0x01 << led_cdev->led_pin);
97     else

```



```

98         val |= (0x01 << led_cdev->led_pin);
99
100     iowrite32(val, led_cdev->va_dr);
101     *ppos += tmp;
102     return tmp;
103 }
104
105 static struct file_operations led_chrdev_fops = {
106     .owner = THIS_MODULE,
107     .open = led_chrdev_open,
108     .release = led_chrdev_release,
109     .write = led_chrdev_write,
110 };
111
112 static struct led_chrdev led_cdev[DEV_CNT] = {
113     {.pa_dr = 0x0209C000, .pa_gdir = 0x0209C004, .pa_iomuxc_mux =
114     0x20E006C, .pa_ccm_ccgrx = 0x20C406C, .pa_iomux_pad =
115     0x20E02F8, .led_pin = 4, .clock_offset = 26},
116     {.pa_dr = 0x20A8000, .pa_gdir = 0x20A8004, .pa_iomuxc_mux =
117     0x20E01E0, .pa_ccm_ccgrx = 0x20C4074, .pa_iomux_pad =
118     0x20E046C, .led_pin = 20, .clock_offset = 12},
119     {.pa_dr = 0x20A8000, .pa_gdir = 0x20A8004, .pa_iomuxc_mux =
120     0x20E01DC, .pa_ccm_ccgrx = 0x20C4074, .pa_iomux_pad =
121     0x20E0468, .led_pin = 19, .clock_offset = 12},
122 };
123
124 static __init int led_chrdev_init(void)
125 {
126     int i = 0;
127     dev_t cur_dev;
128     printk("led chrdev init\n");
129
130     alloc_chrdev_region(&devno, 0, DEV_CNT, DEV_NAME);
131
132     led_chrdev_class = class_create(THIS_MODULE, "led_chrdev");
133
134     for (; i < DEV_CNT; i++) {
135         cdev_init(&led_cdev[i].dev, &led_chrdev_fops);
136         led_cdev[i].dev.owner = THIS_MODULE;
137
138         cur_dev = MKDEV(MAJOR(devno), MINOR(devno) + i);
139
140         cdev_add(&led_cdev[i].dev, cur_dev, 1);
141
142         device_create(led_chrdev_class, NULL, cur_dev, NULL,
143             DEV_NAME "%d", i);
144     }
145
146     return 0;
147 }
148
149 module_init(led_chrdev_init);
150

```

```

151 static __exit void led_chrdev_exit(void)
152 {
153     int i;
154     dev_t cur_dev;
155     printk("led chrdev exit\n");
156
157     for (i = 0; i < DEV_CNT; i++) {
158         cur_dev = MKDEV(MAJOR(devno), MINOR(devno) + i);
159
160         device_destroy(led_chrdev_class, cur_dev);
161
162         cdev_del(&led_cdev[i].dev);
163
164     }
165     unregister_chrdev_region(devno, DEV_CNT);
166     class_destroy(led_chrdev_class);
167
168 }
169
170 module_exit(led_chrdev_exit);
171
172 MODULE_AUTHOR("embedfire");
173 MODULE_LICENSE("GPL");
174

```

```

1 // 测试文件
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <string.h>
6
7 char *led_on = "0\n";
8 char *led_off= "1\n";
9
10 int main(void)
11 {
12     printf("led_cdev test\n");
13     //打开设备
14     int fd = open("/dev/led_chrdev0", O_RDWR);
15     if(fd>0)
16         printf("led_chrdev0 open success\n");
17     else
18         printf("led_chrdev0 open fail\n");
19     //写入数据
20     write(fd, led_on, strlen(led_on));
21     //写入完毕，关闭文件
22     close(fd);
23     sleep(1);
24     //打开设备
25     fd = open("/dev/led_chrdev1", O_RDWR);
26     if(fd>0)
27         printf("led_chrdev1 open success\n");

```

```

28     else
29         printf("led_chrdev1 open fail\n");
30
31     //写入数据
32     write(fd, led_on, strlen(led_on));
33     //写入完毕，关闭文件
34     close(fd);
35     sleep(1);
36     //打开设备
37     fd = open("/dev/led_chrdev2", O_RDWR);
38     if(fd>0)
39         printf("led_chrdev2 open success\n");
40     else
41         printf("led_chrdev2 open fail\n");
42     //写入数据
43     write(fd, led_on, strlen(led_on));
44     //写入完毕，关闭文件
45     close(fd);
46     sleep(1);
47
48     //关闭设备
49     fd = open("/dev/led_chrdev0", O_RDWR);
50     write(fd, led_off, strlen(led_off));
51     close(fd);
52     fd = open("/dev/led_chrdev1", O_RDWR);
53     write(fd, led_off, strlen(led_off));
54     close(fd);
55     fd = open("/dev/led_chrdev2", O_RDWR);
56     write(fd, led_off, strlen(led_off));
57     close(fd);
58
59     return 0;
60 }

```

3.5.3 内核裁剪

裁剪清单（野火E6ULL板子，在官方配置上，在进行一次裁剪）：

i.MX6ULL 内核精简裁剪清单

菜单路径	配置项目 (Config Option)	保留/禁用标志	说明
General setup	Support initial ramdisk/ramfs compressed using bzip2 / LZMA / XZ / LZO / LZ4	☐ 禁用	仅保留需要的解压格式，其余全关以缩减内核体积
System Type → Freescale i.MX family	i.MX6 UltraLite support	☒ 保留	目标平台芯片支持

菜单路径	配置项目 (Config Option)	保留/禁用标志	说明
	i.MX6 ULL support	<*> 保留	目标平台芯片支持
	其他 CPU 系列 (如 i.MX6Q/DL/Solo、i.MX7 等)	[] 禁用	裁剪无关 SOC 架构平台
Networking support	Networking options → The IPv6 protocol	<*> 保留	保留 IPv6 协议栈
	Networking options → 802.1Q/802.1ad VLAN Support	<*> 保留	保留 VLAN 支持
	CAN bus subsystem support	<*> 保留	保留 CAN 总线支持
	Bluetooth subsystem support	[] 禁用 / 视需	蓝牙子系统
	Wireless	[] 禁用 / 视需	无线网络子系统
Device Drivers → Network device support	Ethernet driver support → Freescale devices	<*> 保留	保留飞思卡尔网卡驱动核心
	Ethernet driver support → FEC ethernet controller	<*> 保留	关键: i.MX6ULL 内部 FEC 百兆网卡驱动
	其他无关厂商网卡驱动 (如 Intel、Realtek 等)	[] 禁用	批量清理冗余网卡驱动
File systems	CD-ROM/DVD Filesystems → ISO 9660 / UDF	[] 禁用	嵌入式开发板无光驱设备
	Network File Systems → Root file system on NFS	<*> 必须开启	核心配置: 开启 NFS 挂载根文件系统 (依赖 NFS_FS 和 IP_PNP)

保存之后, `cp .config ../config.bak -> make distclean -> cp ../config.bak .config -> make zImage -j$(nproc) -> make imx6ull-mmc-npi.dtb ->` 将镜像文件和设备树拷贝到NFS共享目录中

错误:

```
1 | drivers/usb/gadget/configfs.c:183:1: note: in expansion of macro
  | 'GI_DEVICE_DESC_SIMPLE_R_u16'
2 |   183 | GI_DEVICE_DESC_SIMPLE_R_u16(bcdDevice);
  |       | ^~~~~~
4 | # 找不到这些变量
```



```

7  grep: busfreq_optee.o: 匹配到二进制文件
8  smp_wfe_imx6.S:143:.globl imx_smp_wfe_optee_end
9  smp_wfe_imx6.S:145:ENTRY(imx_smp_wfe_optee)
10 smp_wfe_imx6.S:183:imx_smp_wfe_optee_end:
11 smp_wfe_imx6.S:184:ENDPROC(imx_smp_wfe_optee)
12
13 # 分析:
14 # 1. 前面的busfreq_optee.c都是在使用这个变量(因为`extern`), 所以不是问题所在的地方
15 # 2. smp_wfe_imx6.S 中使用.globl imx_smp_wfe_optee_end 定义了该变量
16 # 如果这里是.c, 就看谁是定义, 让去看定义这个的.c文件是否被编译, 逐级往上推导
17
18 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/mach-imx$ nvim
smp_wfe_imx6.S +143
19 # 发现确实存在
20 # 3.检查是否有.o文件
21 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/mach-imx$ ls
smp_wfe_imx6.*
smp_wfe_imx6.S
22
23 # 4.发现没有.o文件, 说明当前的Makefile没有汇编阶段, 即开关没有被打开, 通过Makefile查找开关
24 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/arch/arm/mach-imx$ nvim
Makefile
25 # 5.通过/smp_wfe_imx6 搜索文件(按n下一个, 按N上一个), 发现CONFIG_SOC_IMX6Q开关
26 obj-$(CONFIG_SOC_IMX6Q) += mach-imx6q.o ddr3_freq_imx6.o smp_wfe_imx6.o \
27     lpddr2_freq_imx6q.o
28
29 # 6.在make menuconfig中按? 进行搜索CONFIG_SOC_IMX6Q
30 symbol: SOC_IMX6Q [=n]          # 所属符号
31 Type : bool                    # 两态选择类型
32 Prompt: i.MX6 Quad/DualLite support # 所属的选项名字
33 Location:                      # 配置路径
34 -> System Type
35 (1) -> Freescale i.MX family (ARCH_MXC [=y])
36 Defined at arch/arm/mach-imx/kconfig:505
37 Depends on: ARCH_MXC [=y] && ARCH_MULTI_V7 [=y] # 前置依赖
38 Selects: ARM_ERRATA_764369 [=y] && HAVE_ARM_SCU [=y] && HAVE_ARM_TWD [=y] &&
PCI_DOMAINS [=n] && PINCTRL_IMX6Q [=n] && SOC_IMX # 相关联的选中选项
39 # 如果有select by (反向依赖), 需要重点关注, 其依赖决定当前的值。
40
41 # 7.解决
42 # a. 在该配置界面, 找到这个配置i.MX6 Quad/DualLite, 并选上
43 # b. 如果这个配置确实不使用, 查程序 —— 上网搜索需要怎么解决, 发现是厂家没有对a7和a9的内核做区
分
44

```

o 解决b的修改 linux_4.19.35_imx6ull/arch/arm/mach-imx/busfreq_optee.c

```

o
1  #ifdef CONFIG_SMP
2  /*
3   * External declaration
4   */
5  extern void imx_smp_wfe_optee(u32 cpuid, u32 status_addr);
6  extern unsigned long imx_smp_wfe_start asm("imx_smp_wfe_optee");
7  extern unsigned long imx_smp_wfe_end asm("imx_smp_wfe_optee_end");

```

```

8
9 // 改成
10 #if defined(CONFIG_SMP) && (defined(CONFIG_SOC_IMX6Q) ||
    defined(CONFIG_SOC_IMX6SL) || defined(CONFIG_SOC_IMX7D))
11 /*
12  * External declaration
13  */
14 extern void imx_smp_wfe_optee(u32 cpuid, u32 status_addr);
15 extern unsigned long imx_smp_wfe_start asm("imx_smp_wfe_optee");
16 extern unsigned long imx_smp_wfe_end asm("imx_smp_wfe_optee_end");
17
18
19 // -----
20 // 所有的 #ifdef CONFIG_SMP
21
22 // 改成
23
24 #if defined(CONFIG_SMP) && (defined(CONFIG_SOC_IMX6Q) ||
    defined(CONFIG_SOC_IMX6SL) || defined(CONFIG_SOC_IMX7D))
25
26
27
28

```

o

3. 重新 `make zImage -j$(nproc)`，有错误，继续按照上面思路解决。

```

1 # 还有一个错误没有解决
2 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull$ make zImage -j$(nproc)
3 arm-none-linux-gnueabi-hf-ld: drivers/rpmsg/imx_rpmsg.o: in function
    `imx_rpmsg_notify':
4 imx_rpmsg.c:(.text+0xb4): undefined reference to `MU_SendMessage'
5 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0xe8): undefined reference to
    `MU_SendMessageTimeout'
6 arm-none-linux-gnueabi-hf-ld: drivers/rpmsg/imx_rpmsg.o: in function
    `imx_rpmsg_mu_init':
7 imx_rpmsg.c:(.text+0x45c): undefined reference to `MU_Init'
8 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0x474): undefined reference to
    `MU_EnableRxFullInt'
9 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0x488): undefined reference to
    `MU_EnableRxFullInt'
10 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0x498): undefined reference to
    `MU_SetFn'
11 arm-none-linux-gnueabi-hf-ld: drivers/rpmsg/imx_rpmsg.o: in function
    `imx_mu_rpmsg_isr':
12 imx_rpmsg.c:(.text+0x9a8): undefined reference to `MU_ReadStatus'
13 arm-none-linux-gnueabi-hf-ld: imx_rpmsg.c:(.text+0x9f4): undefined reference to
    `MU_ReceiveMsg'
14 arm-none-linux-gnueabi-hf-ld: drivers/rpmsg/imx_rpmsg.o: in function
    `imx_rpmsg_restore':
15 imx_rpmsg.c:(.text+0xae8): undefined reference to `MU_ReadStatus'
16 make: *** [Makefile:1032: vmlinux] 错误 1

```

```

17
18 # 发现没有，跳到上一级
19 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers/rpmsg$ grep -nR
    "MU_SendMessage" *
20 imx_rpmsg.c:158:          MU_SendMessageTimeout(rpvq->rpdev->mu_base, 1,
    mu_rpmsg, 2000);
21 imx_rpmsg.c:160:          MU_SendMessage(rpvq->rpdev->mu_base, 1,
    mu_rpmsg);
22 imx_rpmsg.c:457:          * When wait remote ready timeout, use the
    MU_SendMessageTimeout
23 grep: imx_rpmsg.o: 匹配到二进制文件
24
25 # 发现是mx8_mu.c定义
26 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers/rpmsg$ cd ../
27 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers$ grep -nR
    "MU_SendMessage" *
28 mxc/vpu_malone/vpu_b0.c:462:    MU_SendMessage(base, 1, value);
29 mxc/vpu_malone/vpu_b0.c:463:    MU_SendMessage(base, 0, type);
30 mxc/vpu_malone/vpu_b0.c:2695:    MU_SendMessage(ctx->dev->mu_base_virtaddr, 0,
    COMMAND);
31 mxc/vpu_windsor/vpu_encoder_b0.c:286:    MU_SendMessage(base, 1, value);
32 mxc/vpu_windsor/vpu_encoder_b0.c:287:    MU_SendMessage(base, 0, type);
33 mxc/vpu_windsor/vpu_encoder_b0.c:1624:    MU_SendMessage(core->mu_base_virtaddr,
    0, COMMAND);
34 rpmsg/imx_rpmsg.c:158:          MU_SendMessageTimeout(rpvq->rpdev->mu_base, 1,
    mu_rpmsg, 2000);
35 rpmsg/imx_rpmsg.c:160:          MU_SendMessage(rpvq->rpdev->mu_base, 1,
    mu_rpmsg);
36 rpmsg/imx_rpmsg.c:457:          * When wait remote ready timeout, use the
    MU_SendMessageTimeout
37 grep: rpmsg/imx_rpmsg.o: 匹配到二进制文件
38 soc/imx/mu/mx8_mu.c:89:void MU_SendMessage(void __iomem *base, uint32_t
    regIndex, uint32_t msg)
39 soc/imx/mu/mx8_mu.c:110:void MU_SendMessageTimeout(void __iomem *base, uint32_t
    regIndex, uint32_t msg,
40 soc/imx/sc/main/ipc.c:247:          MU_SendMessage(base, 0, *((uint32_t *) msg));
41 soc/imx/sc/main/ipc.c:252:          MU_SendMessage(base, count %
    MU_TR_COUNT, msg->DATA.u32[count - 1]);
42
43 # 发现没有编译.o
44 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers$ cd soc/imx/mu/
45 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers/soc/imx/mu$ ls
    Makefile  mx8_mu.c
46
47 # 宏也是obj-y
48 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers/soc/imx/mu$ cat
    Makefile
49 obj-y += mx8_mu.o
50
51 # 跳到上一级，发现CONFIG_HAVE_IMX_MU定义了该目录
52 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers/soc/imx$ cat
    Makefile
53 obj-$(CONFIG_HAVE_IMX_GPC) += gpc.o
54 obj-$(CONFIG_IMX7_PM_DOMAINS) += gpcv2.o

```



```

55 obj-$(CONFIG_HAVE_IMX_MU) += mu/
56 obj-$(CONFIG_HAVE_IMX_SC) += sc/
57 obj-$(CONFIG_HAVE_IMX_SC_PD) += pm-domains.o
58 obj-$(CONFIG_HAVE_IMX_BUSFREQ) += busfreq-imx8mq.o
59 obj-$(CONFIG_HAVE_IMX8_SOC) += soc-imx8.o
60 obj-$(CONFIG_HAVE_IMX_GPC_PSCI) += gpc-psci.o
61 obj-$(CONFIG_HAVE_IMX8_SECO) += seco_mu/
62
63 # 进入make menuconfig 搜索该变量
64 # 发现没有 Prompt, 不能被修改
65 # 又发现Selected by, 该值取决于下面两组 (满足其一即可), 打开SOC_IMX6SX
66 Symbol: HAVE_IMX_MU [=n]
67 Type : bool
68   Defined at arch/arm/mach-imx/Kconfig:58
69   Depends on: ARCH_MXC [=y]
70   selected by [n]:
71     - SOC_IMX6SX [=n] && ARCH_MXC [=y] && ARCH_MULTI_V7 [=y]
72     - SOC_IMX7 [=n] && ARCH_MXC [=y] && ARCH_MULTI_V7 [=y]
73
74 # 搜索SOC_IMX6SX
75 Symbol: SOC_IMX6SX [=n]
76 Type : bool
77 Prompt: i.MX6 SoloX support
78   Location:
79     -> System Type
80     (1) -> Freescale i.MX family (ARCH_MXC [=y])
81   Defined at arch/arm/mach-imx/Kconfig:536
82   Depends on: ARCH_MXC [=y] && ARCH_MULTI_V7 [=y]
83   Selects: PINCTRL_IMX6SX [=n] && HAVE_IMX_AMP [=n] && SOC_IMX6 [=y] &&
HAVE_IMX_MU [=n] && HAVE_IMX_RPMSG [=y] && RPMSG [=y] && IMX_SEMA4 [=n] &&
MXC_MLB150 [=y] && KEYBOARD_SNVS_PWRKEY [=y]
84
85 # 发现i.MX6 SoloX support 对该板子没有任何作用,
86 # 解决
87 # a. 打开该开关
88 # b. 消除该配置

```

```

o 1 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers/rpmsg$
grep -n "imx_rpmsg" Makefile
2 9:obj-$(CONFIG_HAVE_IMX_RPMSG) += imx_rpmsg.o
3 10:obj-$(CONFIG_IMX_RPMSG_PINGPONG) += imx_rpmsg_pingpong.o
4 11:obj-$(CONFIG_IMX_RPMSG_TTY) += imx_rpmsg_tty.o
5
6 siguyuan@debian:~/EBF_6ULL/kernel/linux_4.19.35_imx6ull/drivers/rpmsg$
grep -n "IMX_RPMSG" Kconfig
7 58:config HAVE_IMX_RPMSG
8 63:config IMX_RPMSG_PINGPONG
9 68:config IMX_RPMSG_TTY
10
11 # 去make menuconfig中搜索HAVE_IMX_RPMSG
12 Symbol: HAVE_IMX_RPMSG [=y]
13 Type : bool

```

```

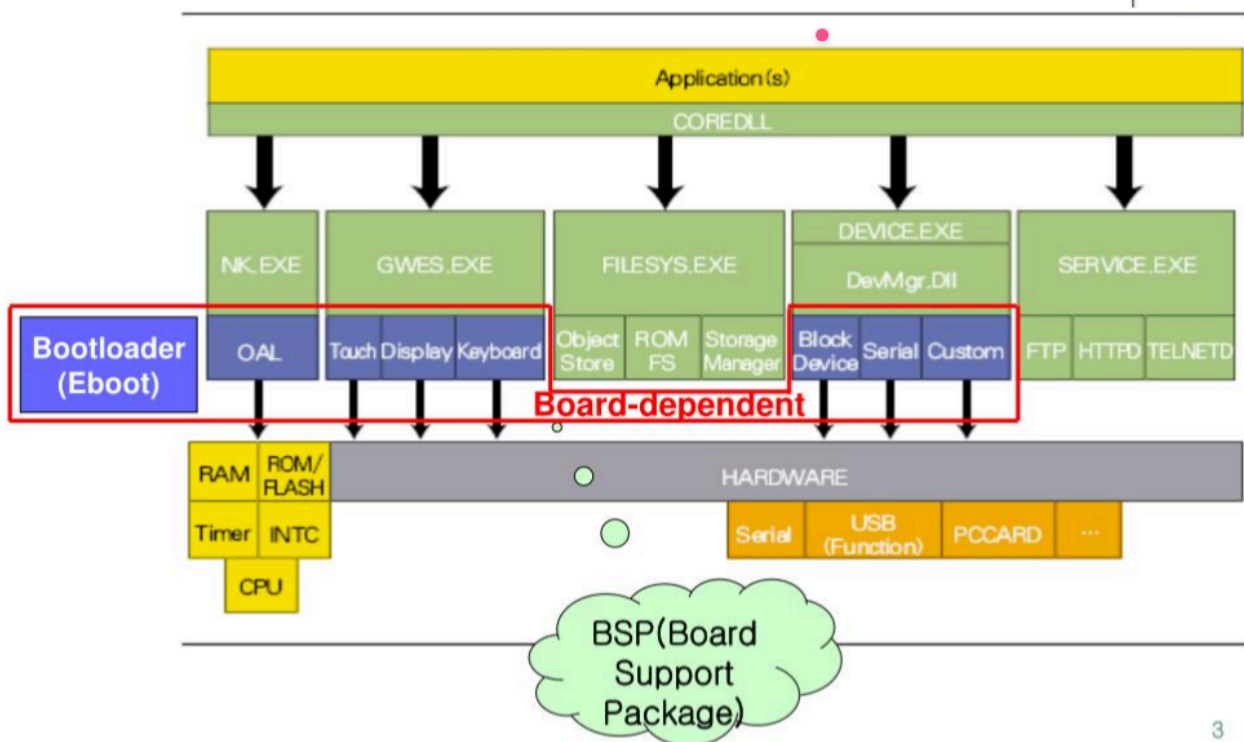
14 Prompt: IMX RPMSG driver on the AMP SOCs
15 Location:
16     -> Device Drivers
17     (1) -> Rpmsg drivers
18     Defined at arch/arm/mach-imx/Kconfig:61
19     Depends on: ARCH_MXC [=y]
20     Selects: RPMSG_VIRTIO [=y] && RPMSG [=y] && RPMSG_VIRTIO [=y] &&
RPMSG_VIRTIO [=y] && RPMSG [=y]
21     Selected by [n]:
22     - SOC_IMX6SX [=n] && ARCH_MXC [=y] && ARCH_MULTI_V7 [=y]
23     - SOC_IMX7 [=n] && ARCH_MXC [=y] && ARCH_MULTI_V7 [=y]
24
25 # 关闭IMX RPMSG driver on the AMP SOCs

```

3.5.4 内核启动 —— 内核板级支持包 (BSP)

板级支持包 (BSP, Board Support Package) 是连接特定硬件开发板与操作系统 (或上层软件) 之间的“胶水软件代码包”。

BSP(Board Support Package) (1)



3

功能维度上BSP包含4大组件:

1. 引导加载程序 (Bootloader)
2. 硬件抽象与板级配置 (Board Configuration / Device Tree) —— 如: 设备树

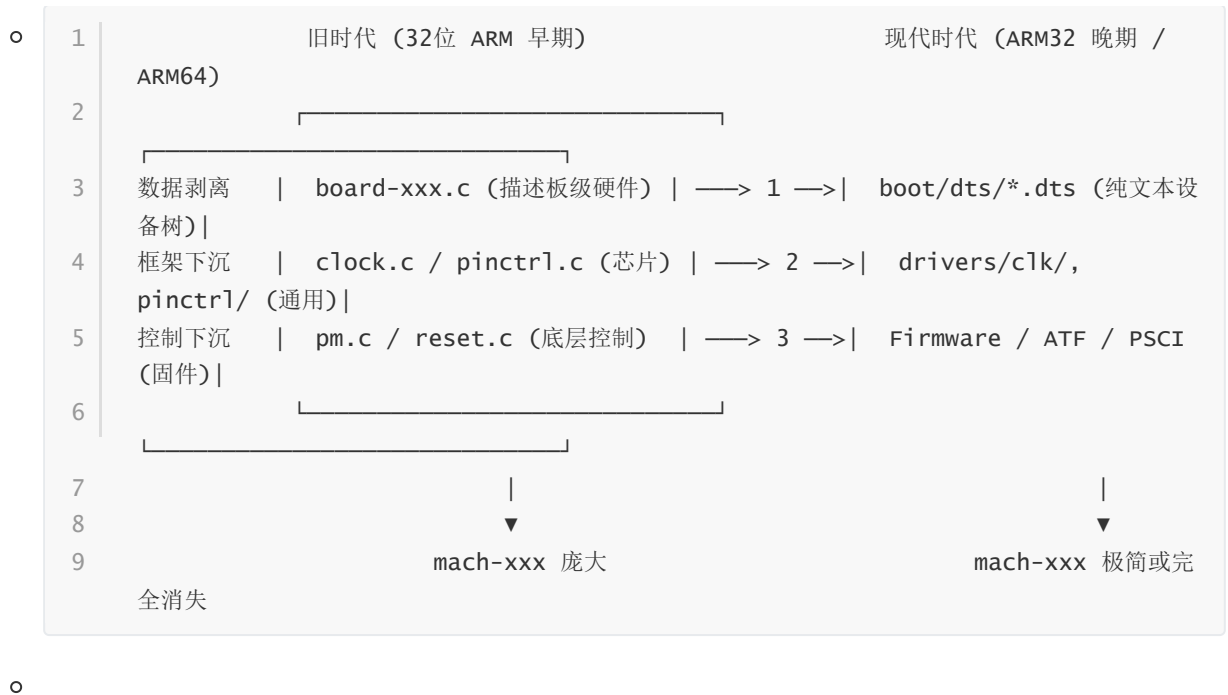
3. 设备驱动程序 (Device Drivers)

4. 操作系统适配层与配置文件 (OS Abstraction Layer & Config)

在内核中：

```
1  |— arch/                                # 【体系结构相关】硬件/芯片/板级目录
2  |   |— arm/                            #   |— 1. 具体 CPU 架构 (ARM 32位)
3  |   |   |— boot/                      #       |— 2. 引导与设备树目录 (BSP核心)
4  |   |   |   |— dts/                  #           |— 3. 板级设备树文件 (.dts/.dtsi)
5  |   |   |— mach-imx/                  #       |— 2. i.MX 系列芯片的 BSP 目录
6  |   |   |   |— mach-imx6ul.c         #           |— 3. i.MX6ULL 单板初始化与平台配置代码
7  |   |   |   |— pm-imx6.c             #           |— 3. i.MX6 系列特定板级的电源管理 (PM)
8  |   |   |   |— clk-imx6ul.c          #           |— 3. i.MX6ULL 芯片时钟树 (Clock) 控制
9  |   |   |   |— mm.c                  #           |— 3. i.MX 芯片内存物理地址与 I/O 映射
10 |   |   |— mach-s3c2410/             #       |— 2. 三星 S3C2410 系列的 BSP 目录
11 |   |
12 |   |— drivers/                       # 【体系结构无关】跨平台的通用驱动框架
13 |       |— net/                      #   |— 1. 网络设备驱动子系统
14 |       |   |— ethernet/             #       |— 2. 以太网控制器通用驱动
15 |       |— char/                     #   |— 1. 字符设备驱动子系统
16 |       |   |— random.c
```

- `arch/CPU体系结构/mach-xxx` —— BSP扩展目录，为了存放“设备树 (DTS) 写不了”且“通用驱动 (drivers/) 装不下”的芯片级 C 代码 (动态逻辑)。**添加硬件驱动，需要在这个目录添加**
- BSP目录的.c本质是：结构体，含有大量的函数指针，启动之前，将各个指针指向正确的函数位置。在“编译时把这些结构体链接进了内核，并在启动初期直接执行结构体里的回调函数。
- 非体系相关的文件，只负责在合适的时间节点，调用板级支持包的结构体里的函数指针
- 现在的改变 (arm64) :
 - 数据剥离：板级硬件描述 → 变成设备树 (DTS)
 - **改变**：以前在 `mach-xxx/board-*.c` 里用 C 代码手动注册的各种外设 (如 UART、网卡、GPIO)，现在**全部改用 .dts 设备树文本文件描述**。
 - **结果**：所有的 `board-*.c` 彻底清理干净。
 - 框架下沉：芯片级控制代码 → 变成通用子系统 (`drivers/`)
 - **改变**：以前写在 `mach-xxx` 里的引脚复用 (Pinctrl)、时钟树 (Clock)、电源管理 (PMIC)、复位 (Reset) 等芯片逻辑，被重构并**强制下沉到内核统一的驱动框架中** (如 `drivers/clk/`、`drivers/pinctrl/`)。
 - **结果**：现代 ARM32 (如 i.MX6/7) 的 `mach-xxx` 目录大幅瘦身，仅剩一两个基本的入口文件 (如 `mach-imx6q.c`)。
 - 控制下沉：底层电源与安全控制 → 变成固件协议 (ATF / PSCI)
 - **改变**：以前在 `mach-xxx/pm.c` 里操作寄存器的低功耗/多核唤醒逻辑，在 ARM64 架构下**直接交给了 Secure World 固件 (如 ATF)**。内核只需通过标准 API (PSCI) 发起请求，不再直接碰硬件。
 - **结果**：ARM64 架构**完全取消了 mach-xxx 目录**，从源头上杜绝了代码膨胀。



内核启动时:

1、先执行汇编程序 `arch/CPU体系架构/kernel/head.S`，进行相关的初始化:

```

1  /* arch/arm/kernel/head.S */
2  ...
3
4  /*
5   * Kernel startup entry point.
6   * -----
7   *
8   * This is normally called from the decompressor code. The requirements
9   * are: MMU = off, D-cache = off, I-cache = dont care, r0 = 0,
10  * r1 = machine nr, r2 = atags or dtb pointer.
11  *
12  * This code is mostly position independent, so if you link the kernel at
13  * 0xc0008000, you call this at __pa(0xc0008000).
14  *
15  * See linux/arch/arm/tools/mach-types for the complete list of machine
16  * numbers for r1.
17  *
18  * We're trying to keep crap to a minimum; DO NOT add any machine specific
19  * crap here - that's what the boot loader (or in extreme, well justified
20  * circumstances, zImage) is for.
21  */
22  .arm
23
24  __HEAD
25  ENTRY(stext)
26  ARM_BE8(setend be ) @ ensure we are in BE8 mode
27
28  THUMB( badr    r9, 1f    ) @ kernel is always entered in ARM.

```

```

29 THUMB( bx r9 ) @ If this is a Thumb-2 kernel,
30 THUMB( .thumb ) @ switch to Thumb now.
31 THUMB(1: )
32
33 #ifdef CONFIG_ARM_VIRT_EXT
34 bl __hyp_stub_install
35 #endif
36 @ ensure svc mode and all interrupts masked
37 safe_svcmode_maskall r9
38
39 mrc p15, 0, r9, c0, c0 @ get processor id
40 bl __lookup_processor_type @ r5=procinfo r9=cpuid
41 movs r10, r5 @ invalid processor (r5=0)?
42 THUMB( it eq ) @ force fixup-able long branch encoding
43 beq __error_p @ yes, error 'p'
44
45 #ifdef CONFIG_ARM_LPAE
46 mrc p15, 0, r3, c0, c1, 4 @ read ID_MMFR0
47 and r3, r3, #0xf @ extract VMSA support
48 cmp r3, #5 @ long-descriptor translation table format?
49 THUMB( it lo ) @ force fixup-able long branch encoding
50 blo __error_lpaes @ only classic page table format
51 #endif
52
53 #ifndef CONFIG_XIP_KERNEL
54 adr r3, 2f
55 ldmia r3, {r4, r8}
56 sub r4, r3, r4 @ (PHYS_OFFSET - PAGE_OFFSET)
57 add r8, r8, r4 @ PHYS_OFFSET
58 #else
59 ldr r8, =PLAT_PHYS_OFFSET @ always constant in this case
60 #endif
61
62 ...

```

arm的启动要求:

```

1 * are: MMU = off, D-cache = off, I-cache = dont care, r0 = 0,
2 * r1 = machine nr, r2 = atags or dtb pointer.
3
4 关闭MMU单元（内存管理单元）、D-cache单元（数据缓存）；I-cache不关心（指令缓存），r0寄存器清理，
r1寄存器等于machine nr，r2寄存器等于atags 或 dtb pointer（启动参数/设备树地址）
5
6 machine nr对应的是板子具体的硬件型号/机器号，后面都是通过设备树进行查找
7 这些都是在跳转到内核入口之前需要准备的工作，也就是bootz命令的执行的内容。

```

2、执行C语言入口函数 `start_kernel()`。Linux 内核体系无关的全局统一通用入口函数（位于 `init/main.c`）

```

1 asmlinkage __visible void __init start_kernel(void)
2 {
3     char *command_line;

```

```

4     char *after_dashes;
5
6     set_task_stack_end_magic(&init_task);
7     smp_setup_processor_id();
8     debug_objects_early_init();
9
10    cgroup_init_early();
11
12    local_irq_disable();
13    early_boot_irqs_disabled = true;
14
15    /*
16     * Interrupts are still disabled. Do necessary setups, then
17     * enable them.
18     */
19    boot_cpu_init();
20    page_address_init();
21    pr_notice("%s", linux_banner); // linux_banner初始化时打印的版本信息
22    setup_arch(&command_line);     // 架构构造
23    /*
24     * Set up the the initial canary and entropy after arch
25     * and after adding latent and command line entropy.
26     */
27    add_latent_entropy();
28    add_device_randomness(command_line, strlen(command_line));
29    boot_init_stack_canary();
30    mm_init_cpumask(&init_mm);
31    setup_command_line(command_line);
32    setup_nr_cpu_ids();
33    setup_per_cpu_areas();
34    smp_prepare_boot_cpu(); /* arch-specific boot-cpu hooks */
35    boot_cpu_hotplug_init();
36
37    build_all_zonelists(NULL);
38    page_alloc_init();
39
40    pr_notice("kernel command line: %s\n", boot_command_line);
41    parse_early_param();
42    after_dashes = parse_args("Booting kernel",
43                             static_command_line, __start__param,
44                             __stop__param - __start__param,
45                             -1, -1, NULL, &unknown_bootoption);
46    if (!IS_ERR_OR_NULL(after_dashes))
47        parse_args("Setting init args", after_dashes, NULL, 0, -1, -1,
48                  NULL, set_init_arg);
49
50    jump_label_init();
51
52    /*
53     * These use large bootmem allocations and must precede
54     * kmem_cache_init()
55     */
56    setup_log_buf(0);

```

```

57     vfs_caches_init_early();
58     sort_main_extable();
59     trap_init();
60     mm_init();
61
62     ftrace_init();
63
64     /* trace_printk can be enabled here */
65     early_trace_init();
66
67     /*
68      * Set up the scheduler prior starting any interrupts (such as the
69      * timer interrupt). Full topology setup happens at smp_init()
70      * time - but meanwhile we still have a functioning scheduler.
71      */
72     sched_init();
73     /*
74      * Disable preemption - early bootup scheduling is extremely
75      * fragile until we cpu_idle() for the first time.
76      */
77     preempt_disable();
78     if (WARN(!irqs_disabled(),
79             "Interrupts were enabled *very* early, fixing it\n"))
80         local_irq_disable();
81     radix_tree_init();
82
83     /*
84      * Set up housekeeping before setting up workqueues to allow the unbound
85      * workqueue to take non-housekeeping into account.
86      */
87     housekeeping_init();
88
89     /*
90      * Allow workqueue creation and work item queueing/cancelling
91      * early. Work item execution depends on kthreads and starts after
92      * workqueue_init().
93      */
94     workqueue_init_early();
95
96     rcu_init();
97
98     /* Trace events are available after this */
99     trace_init();
100
101     if (initcall_debug)
102         initcall_debug_enable();
103
104     context_tracking_init();
105     /* init some links before init_ISA_irqs() */
106     early_irq_init();
107     init_IRQ();
108     tick_init();
109     rcu_init_nohz();

```

```

110     init_timers();
111     hrtimers_init();
112     softirq_init();
113     timekeeping_init();
114     time_init();
115     printk_safe_init();
116     perf_event_init();
117     profile_init();
118     call_function_init();
119     WARN(!irqs_disabled(), "Interrupts were enabled early\n");
120
121     early_boot_irqs_disabled = false;
122     local_irq_enable();
123
124     kmem_cache_init_late();
125
126     /*
127      * HACK ALERT! This is early. We're enabling the console before
128      * we've done PCI setups etc, and console_init() must be aware of
129      * this. But we do want output early, in case something goes wrong.
130      */
131     console_init();
132     if (panic_later)
133         panic("Too many boot %s vars at `%s'", panic_later,
134             panic_param);
135
136     lockdep_init();
137
138     /*
139      * Need to run this when irq's are enabled, because it wants
140      * to self-test [hard/soft]-irqs on/off lock inversion bugs
141      * too:
142      */
143     locking_selftest();
144
145     /*
146      * This needs to be called before any devices perform DMA
147      * operations that might use the SWIOTLB bounce buffers. It will
148      * mark the bounce buffers as decrypted so that their usage will
149      * not cause "plain-text" data to be decrypted when accessed.
150      */
151     mem_encrypt_init();
152
153 #ifdef CONFIG_BLK_DEV_INITRD
154     if (initrd_start && !initrd_below_start_ok &&
155         page_to_pfn(virt_to_page((void *)initrd_start)) < min_low_pfn) {
156         pr_crit("initrd overwritten (0x%08lx < 0x%08lx) - disabling it.\n",
157             page_to_pfn(virt_to_page((void *)initrd_start)),
158             min_low_pfn);
159         initrd_start = 0;
160     }
161 #endif
162     kmemleak_init();

```



```

163     debug_objects_mem_init();
164     setup_per_cpu_pageset();
165     numa_policy_init();
166     acpi_early_init();
167     if (late_time_init)
168         late_time_init();
169     sched_clock_init();
170     calibrate_delay();
171     pid_idr_init();
172     anon_vma_init();
173 #ifdef CONFIG_X86
174     if (efi_enabled(EFI_RUNTIME_SERVICES))
175         efi_enter_virtual_mode();
176 #endif
177     thread_stack_cache_init();
178     cred_init();
179     fork_init();
180     proc_caches_init();
181     uts_ns_init();
182     buffer_init();
183     key_init();
184     security_init();
185     dbg_late_init();
186     vfs_caches_init();
187     pagecache_init();
188     signals_init();
189     seq_file_init();
190     proc_root_init();
191     nsfs_init();
192     cpuset_init();
193     cgroup_init();
194     taskstats_init_early();
195     delayacct_init();
196
197     check_bugs();
198
199     acpi_subsystem_init();
200     arch_post_acpi_subsys_init();
201     sfi_init_late();
202
203     if (efi_enabled(EFI_RUNTIME_SERVICES)) {
204         efi_free_boot_services();
205     }
206
207     /* Do the rest non-__init'ed, we're now alive */
208     rest_init();
209 }

```

- 1 1. 环境硬化与关中断（关中断 / 锁防重入）
- 2 |
- 3 ▼
- 4 2. 架构与 BSP 绑定（setup_arch – 展开设备树 / 匹配 BSP）

```

5      |
6      ▼
7      3. 内存与调度器建立 (mm_init / sched_init)
8      |
9      ▼
10     4. 中断与时钟初始化 (init_IRQ / time_init)
11         | —> 开中断 (local_irq_enable)
12         ▼
13     5. 子系统与文件系统加载 (vfs_caches_init / cgroup_init)
14         |
15         ▼
16     6. 跃迁到 1 号进程 (rest_init - 挂载驱动 / 启动 init)

```

```

1  // setup_arch
2  void __init setup_arch(char **cmdline_p)
3  {
4      const struct machine_desc *mdesc;          // 含有大量初始化函数指针结构体
5
6      setup_processor();
7      mdesc = setup_machine_fdt(__atags_pointer);    // 初始化这个结构体
8      if (!mdesc)
9          mdesc = setup_machine_tags(__atags_pointer, __machine_arch_type);
10     if (!mdesc) {
11         early_printk("\nError: invalid dtb and unrecognized/unsupported machine
ID\n");
12         early_printk("  r1=0x%08x, r2=0x%08x\n", __machine_arch_type,
13                     __atags_pointer);
14         if (__atags_pointer)
15             early_printk("  r2[]=%*ph\n", 16,
16                         phys_to_virt(__atags_pointer));
17         dump_machine_table();
18     }
19
20     machine_desc = mdesc;
21     machine_name = mdesc->name;
22     dump_stack_set_arch_desc("%s", mdesc->name);
23
24     if (mdesc->reboot_mode != REBOOT_HARD)
25         reboot_mode = mdesc->reboot_mode;
26
27     init_mm.start_code = (unsigned long) _text;
28     init_mm.end_code   = (unsigned long) _etext;
29     init_mm.end_data   = (unsigned long) _edata;
30     init_mm.brk        = (unsigned long) _end;
31
32     /* populate cmd_line too for later use, preserving boot_command_line */
33     strlcpy(cmd_line, boot_command_line, COMMAND_LINE_SIZE);
34     *cmdline_p = cmd_line;
35
36     early_fixmap_init();
37     early_ioremap_init();
38

```

```

39     parse_early_param();
40
41     #ifdef CONFIG_MMU
42         early_mm_init(mdesc);
43     #endif
44     setup_dma_zone(mdesc);
45     xen_early_init();
46     efi_init();
47     /*
48      * Make sure the calculation for lowmem/highmem is set appropriately
49      * before reserving/allocating any mmeory
50      */
51     adjust_lowmem_bounds();
52     arm_memblock_init(mdesc);
53     /* Memory may have been removed so recalculate the bounds. */
54     adjust_lowmem_bounds();
55
56     early_ioremap_reset();
57
58     paging_init(mdesc);
59     request_standard_resources(mdesc);
60
61     if (mdesc->restart)
62         arm_pm_restart = mdesc->restart;
63
64     unflatten_device_tree();
65
66     arm_dt_init_cpu_maps();
67     psci_dt_init();
68     #ifdef CONFIG_SMP
69         if (is_smp()) {
70             if (!mdesc->smp_init || !mdesc->smp_init()) {
71                 if (psci_smp_available())
72                     smp_set_ops(&psci_smp_ops);
73                 else if (mdesc->smp)
74                     smp_set_ops(mdesc->smp);
75             }
76             smp_init_cpus();
77             smp_build_mpidr_hash();
78         }
79     #endif
80
81     if (!is_smp())
82         hyp_mode_check();
83
84     reserve_crashkernel();
85
86     #ifdef CONFIG_GENERIC_IRQ_MULTI_HANDLER
87         handle_arch_irq = mdesc->handle_irq;
88     #endif
89
90     #ifdef CONFIG_VT
91     #if defined(CONFIG_VGA_CONSOLE)

```

```

92     conswitchp = &vga_con;
93     #elif defined(CONFIG_DUMMY_CONSOLE)
94         conswitchp = &dummy_con;
95     #endif
96 #endif
97
98     if (mdesc->init_early)
99         mdesc->init_early();
100 }

```

板级支持包定义:

```

1 // arch/arm/include/asm/mach/arch.h
2 // 定义了nr机器号
3 #define DT_MACHINE_START(_name, _namestr)      \
4 static const struct machine_desc __mach_desc_##_name      \
5     __used                                         \
6     __attribute__((__section__(".arch.info.init"))) = {    \
7     .nr      = ~0,                                     \
8     .name     = _namestr,                             \
9
10 #endif
11
12 // arch/arm/mach-imx/mach-imx6il.c
13 DT_MACHINE_START(IMX6UL, "Freescale i.MX6 UltraLite (Device Tree)")
14     .map_io      = imx6ul_map_io,
15     .init_irq    = imx6ul_init_irq,
16     .init_machine = imx6ul_init_machine,
17     .init_late   = imx6ul_init_late,
18     .dt_compat   = imx6ul_dt_compat,
19 MACHINE_END

```